EXEO
Digital Solutions

システムエンジニア（Java開発）
5days体験セミナー

エクシオ・デジタルソリューションズ株式会社

ダウンロード資材、環境（OS）の確認

- Eclipseのダウンロード、起動
- Windows10 or 11（eclipseが起動する環境であればOK）
- java教材（EDS2026.zipの着弾確認）
- 資料（パワポの着弾確認）



Eclipseインストール

- はじめに
  インストールができていない人は以下から最新版のダウンロードをお願いします
- ※事前送付済み

- 「https://ftp.jaist.ac.jp/pub/mergedoc/pleiades/2025/pleiades-2025-12-java-win-64bit-jre_20260117.exe」

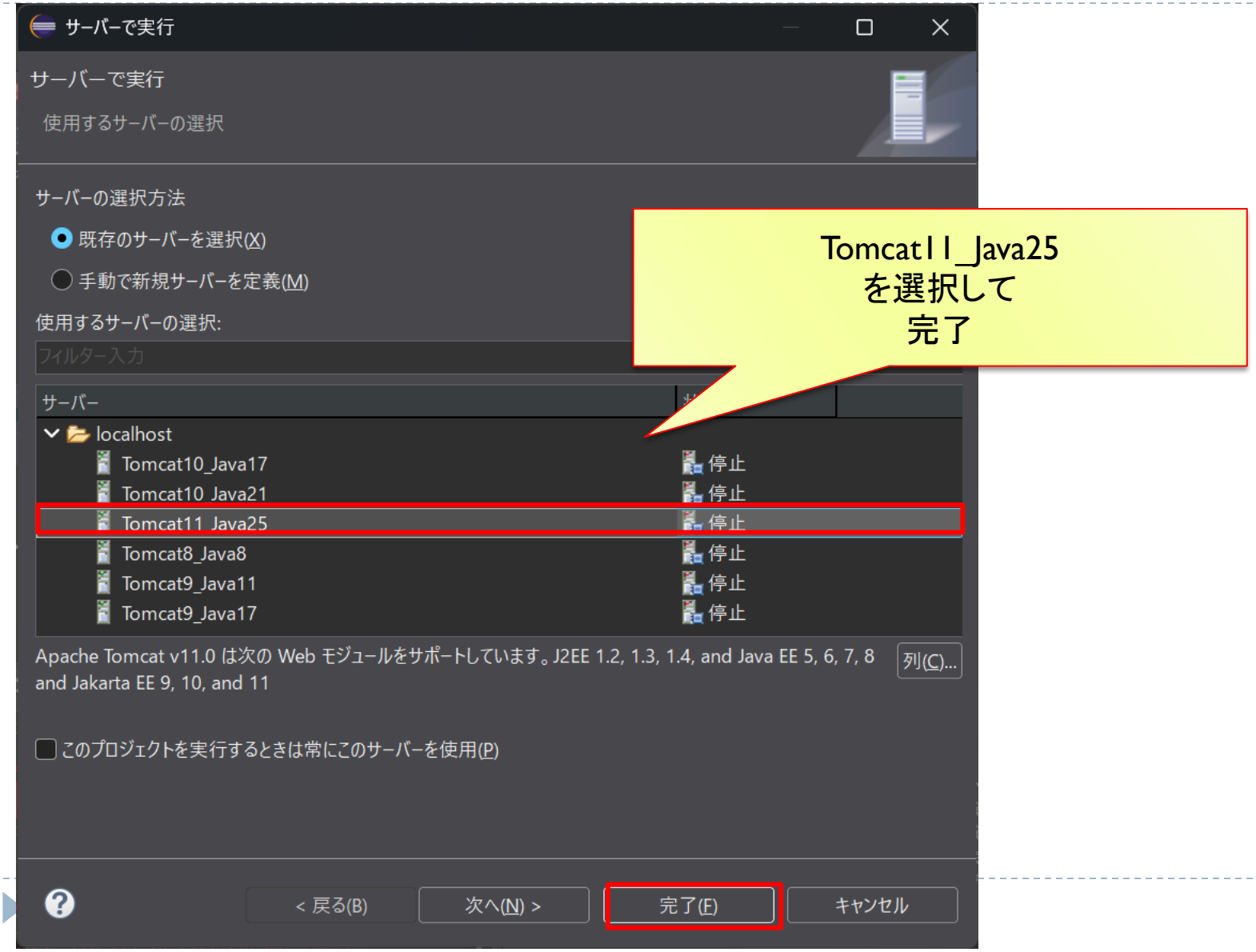


Eclipseでのjsp起動手順

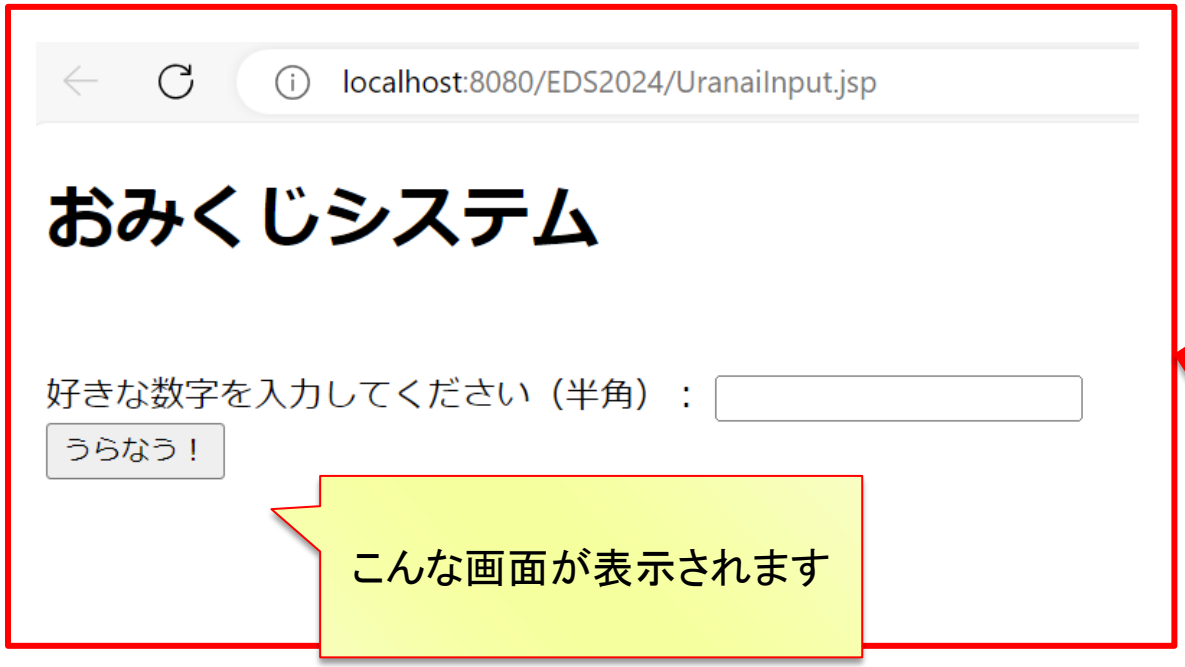
- 起動後の画面



Eclipseでのjsp起動手順

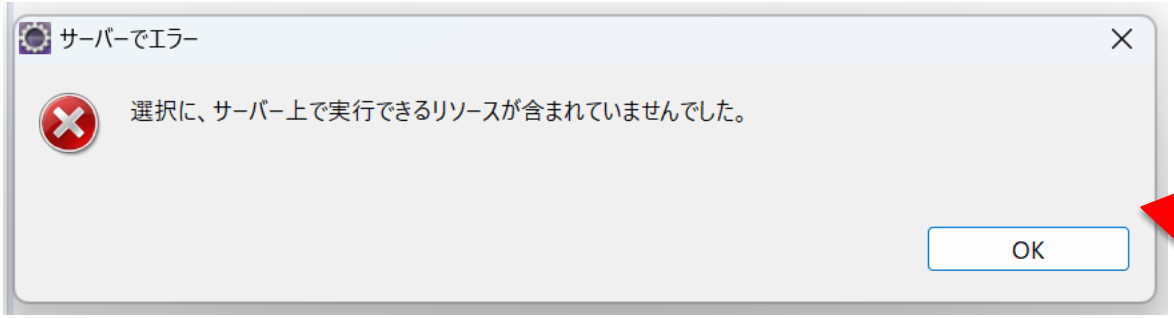


Eclipseでのjsp起動手順



こんな画面が表示されます

ダウンロードしたeclipseでこの画面が出なかった人は教えてください



この画面が出たりした方も同上の対応をお願いします

導入(Java)

- Javaって？

①大規模な開発で用いられることが多い

・セキュリティが強固（堅牢性（けんろうせい））
OSを操作するようなコマンドを実行出来ない、
型によるメモリ確保（後述）等

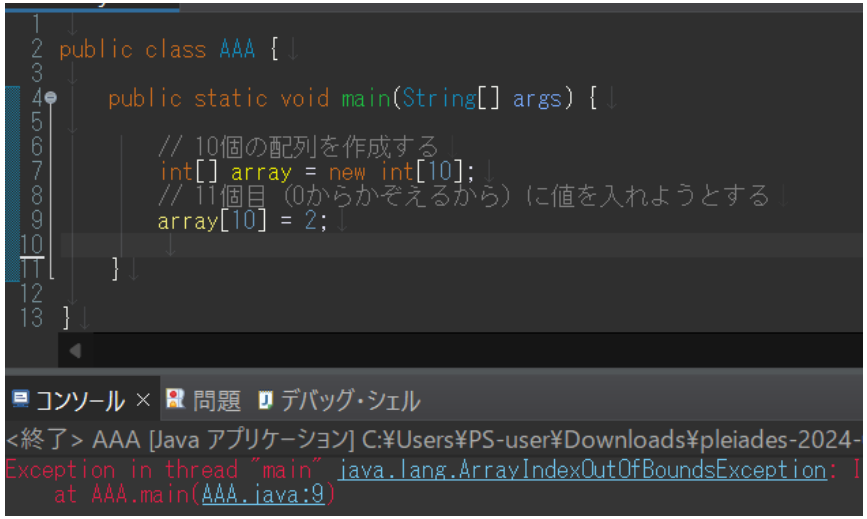
・動作が早い（インタープリタに比べて）
コンパイル（事前にマシン語に変換）してから動く

・使い倒されている
未知の動作などが（ほぼ）無い

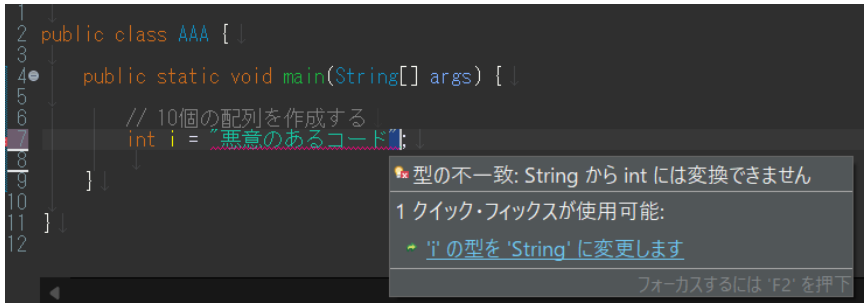


※Javaの型安全性

- 固定サイズの配列: 配列は作成時にサイズが固定され、範囲外アクセスが発生すると自動的に例外がスロー



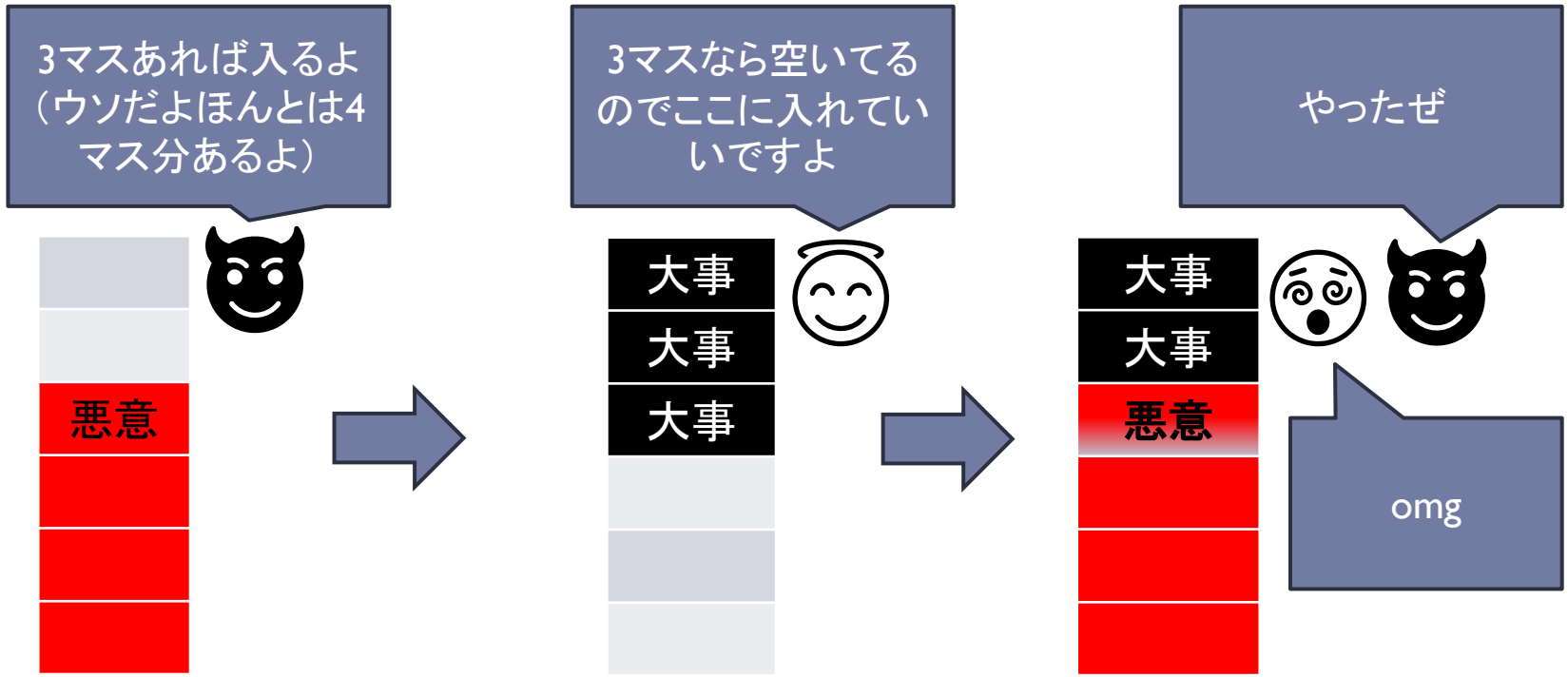
- 型チェック: すべての変数の型がコンパイル時にチェックされ、不適切な型の操作はコンパイルエラー





※Javaの型安全性

■：必ず実行する情報　　■：格納する情報



事前のメモリ確保によりこの手の攻撃を防ぐ

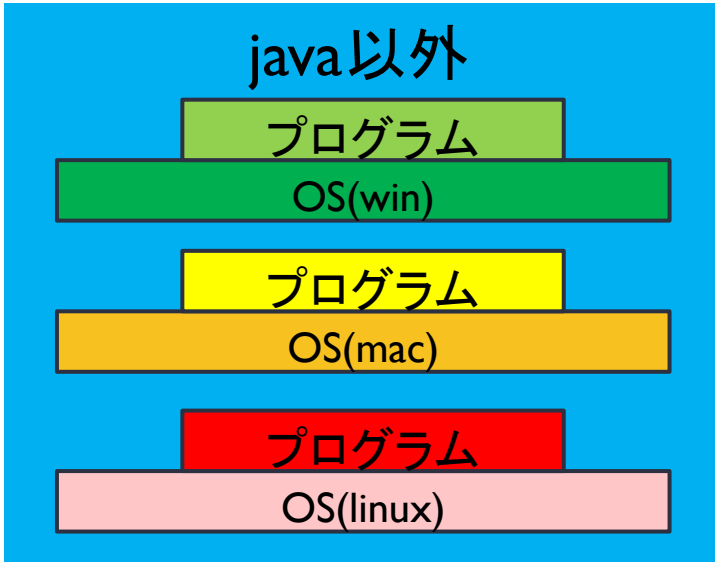
導入(Java)

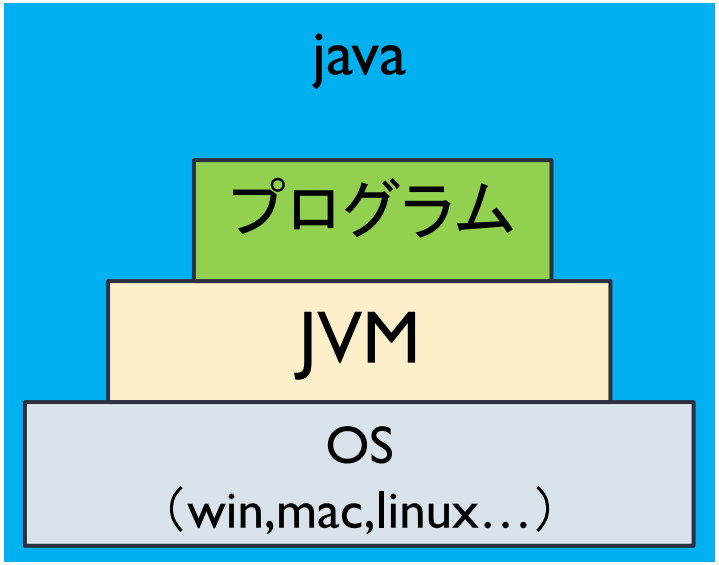
- Javaって？

②どこでも動く

OSとプログラムの間にJVMという実行環境を挟むことで

OSに依存しない開発 + 動作が可能





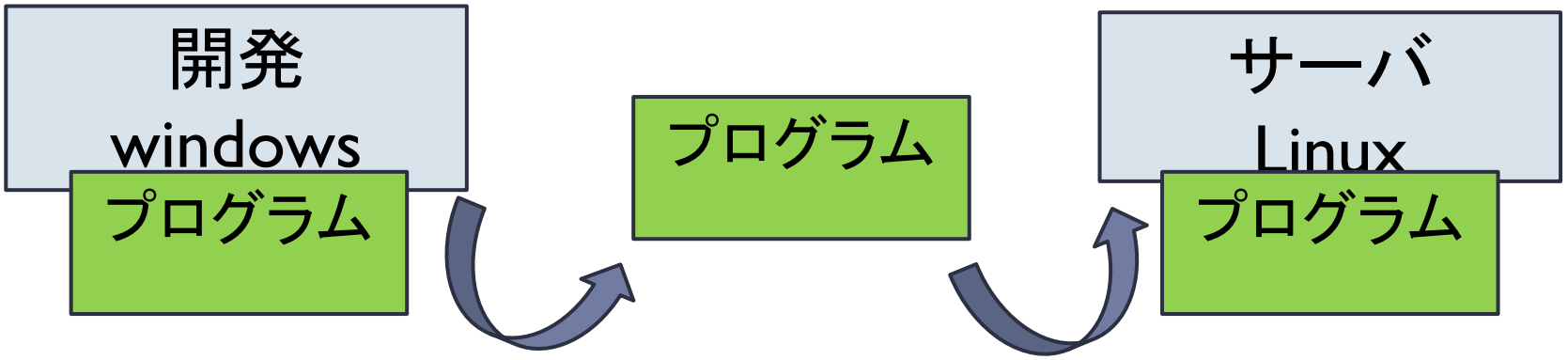


導入(Java)

- Javaって？

③どこでも動くとどうなる？

windowsで作って、それをlinuxサーバに置いて動かす、
といったOSに依存しない開発が可能になる





プログラミング基礎

- 開発環境（JavaEclipse）について
- クラス/パッケージ
- 変数
- メソッド
- フィールド/ローカル変数
- キャスト演算子
- 制御文（繰り返し文、分岐文）
- 配列



確認事項

- 本日コーディングに使用するのは「eclipse」です。

☆ショートカットキー☆

・ctrl+c : コピー

・ctrl+v : ペースト

・ctrl+s : 上書き保存

・ctrl+shift+F

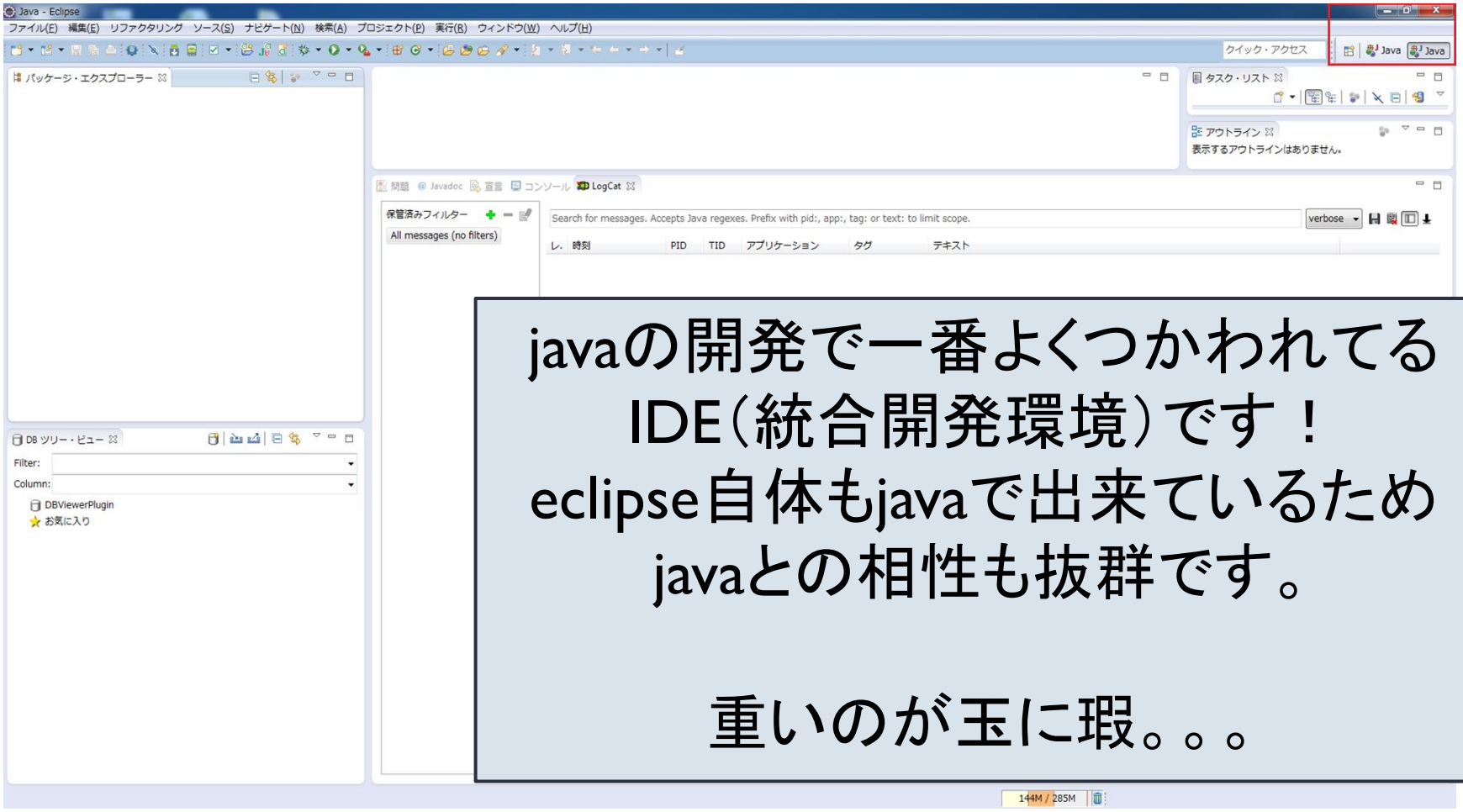
・ctrl+スペース : コードの予測変換

・シートのドラッグアンドドロップ : シートの分割表示

→下の3つは実演します



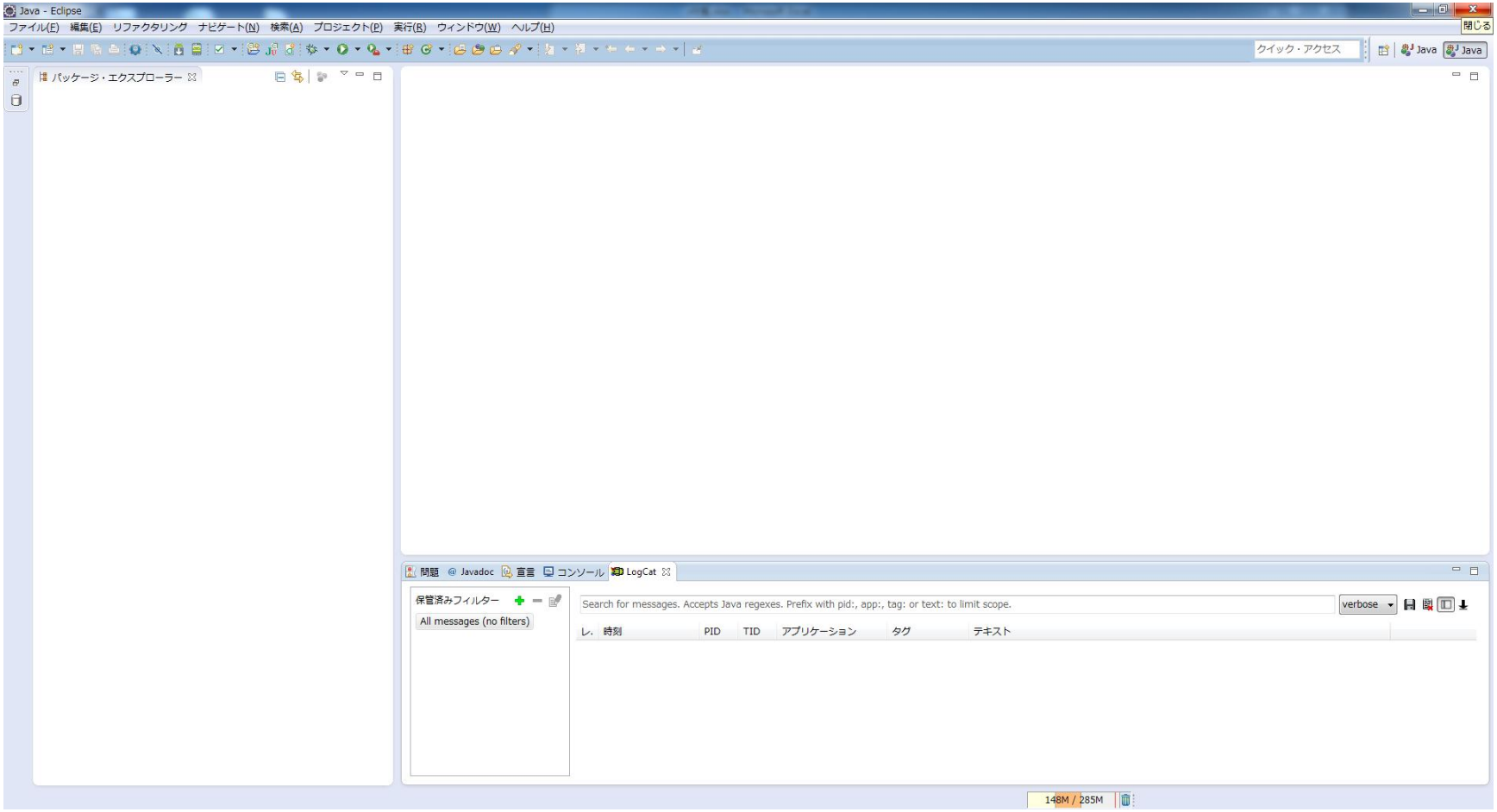
開発環境（JavaEclipse）について



javaの開発で一番よくつかわれてる
IDE（統合開発環境）です！
eclipse自体もjavaで出来ているため
javaとの相性も抜群です。

重いのが玉に瑕。。。

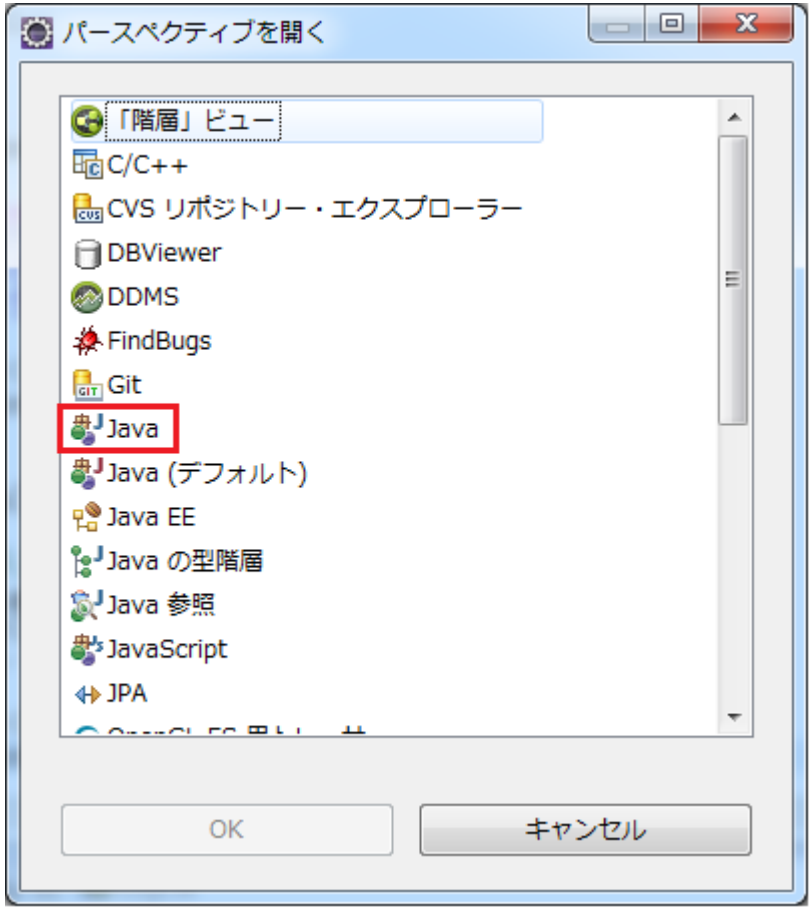
JavaEclipseの使い方



JavaEclipseの使い方

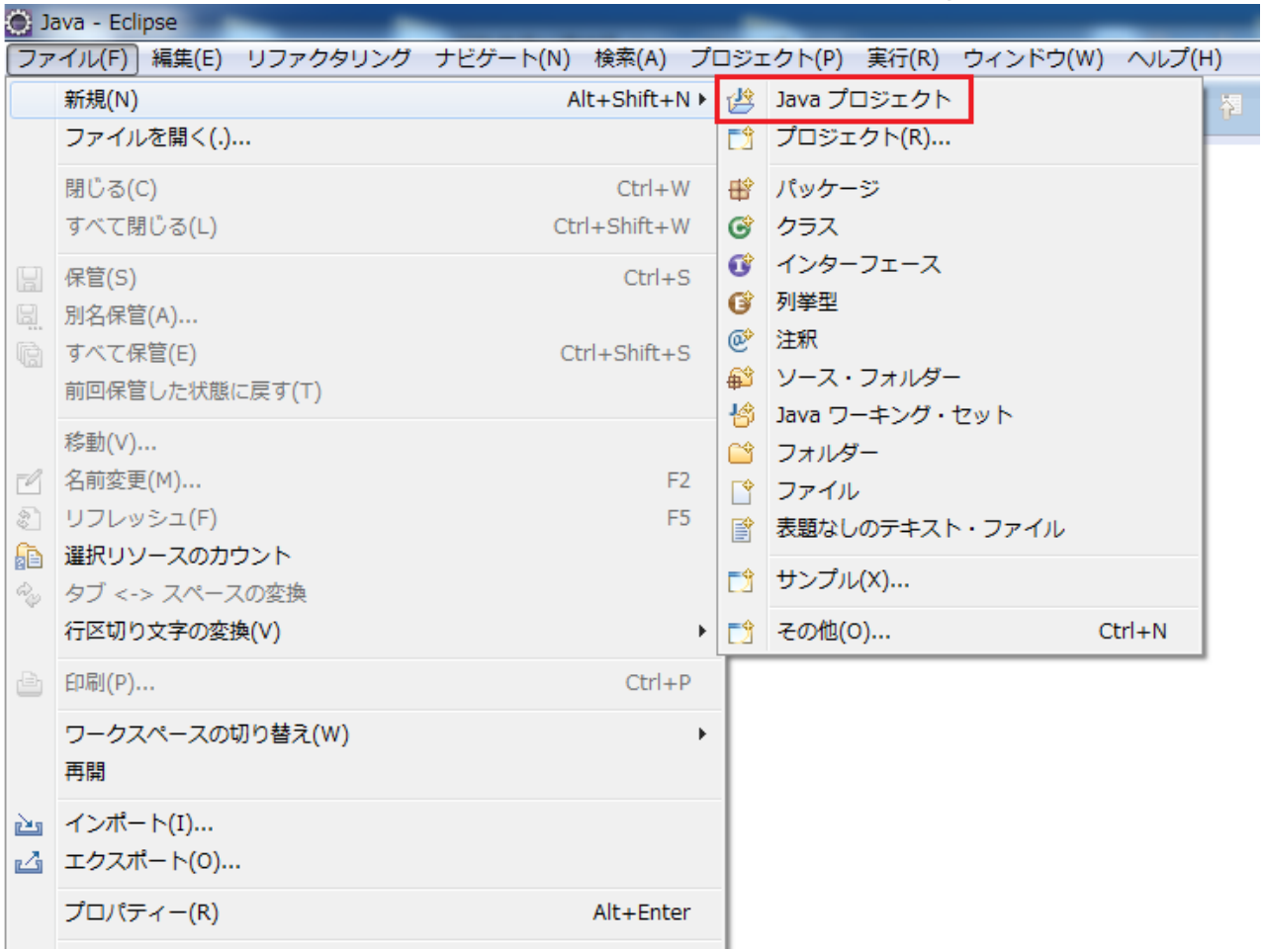


- パースペクティブ



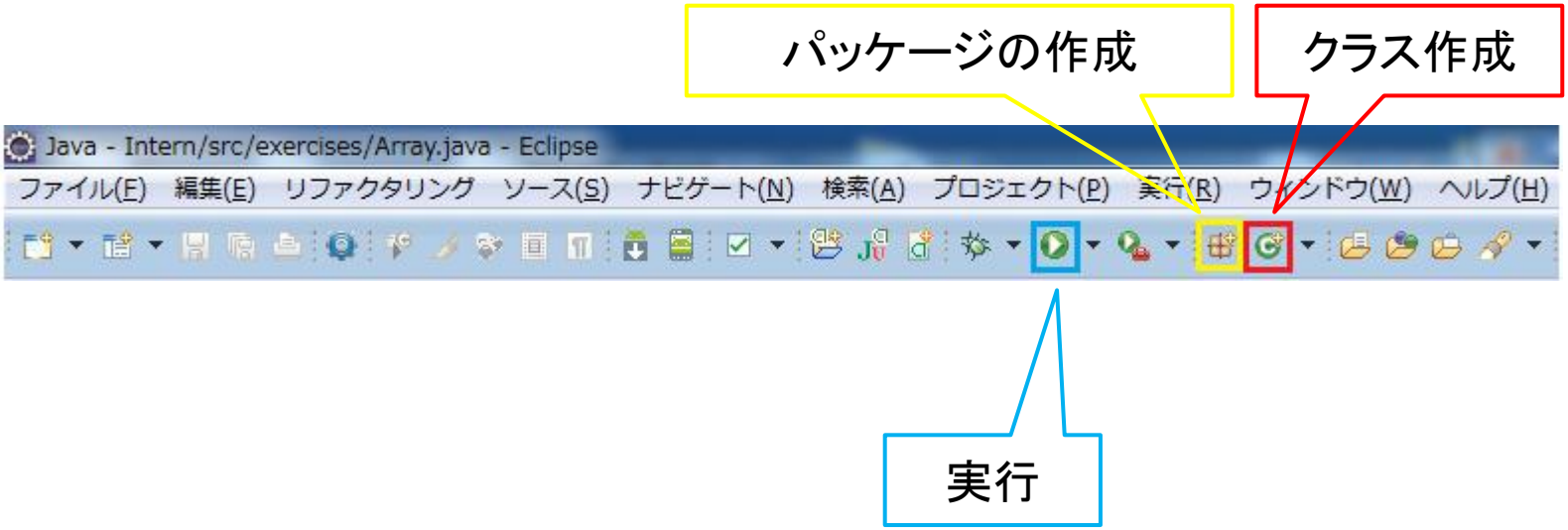
JavaEclipseの使い方

- プロジェクト作成　ファイル→新規→Javaプロジェクト



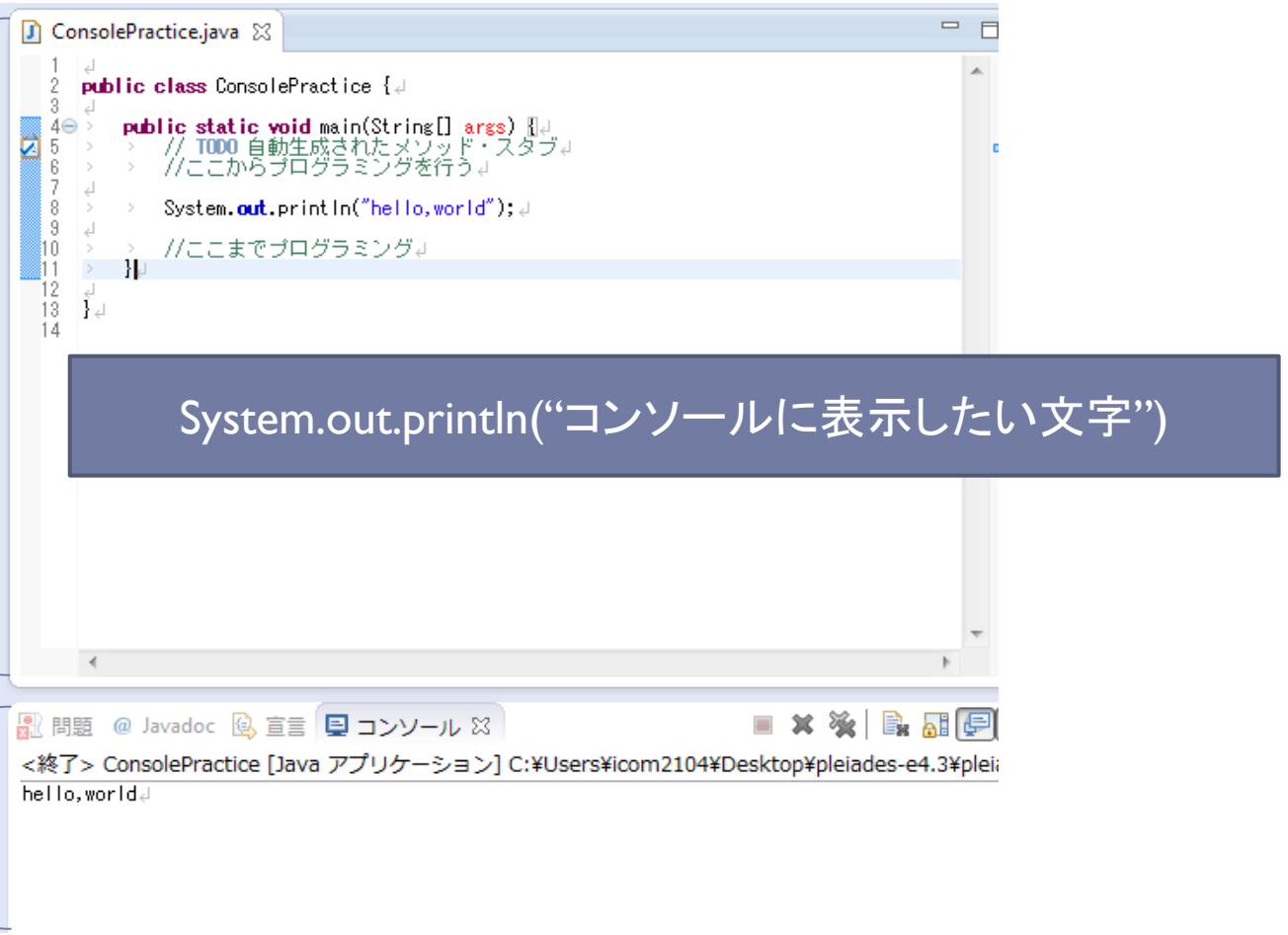
JavaEclipseの使い方

- パッケージの作成
- クラスの作成
- 実行



JavaEclipseの使い方

- Eclipseのコンソール画面へ文字を表示する



クラス/パッケージ

- クラス

→変数やメソッドをまとめて定義できる単位

（クラス名は原則として頭文字を大文字とする）

- パッケージ

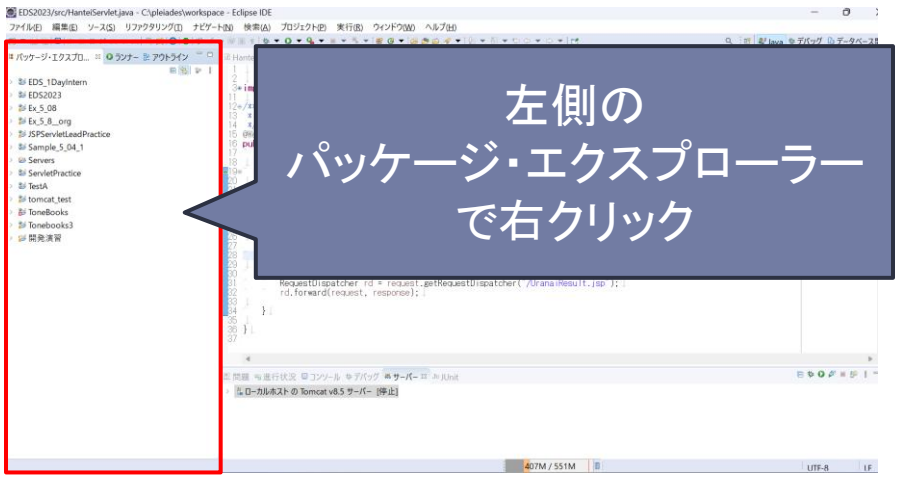
→複数のクラスを分けて管理するための単位

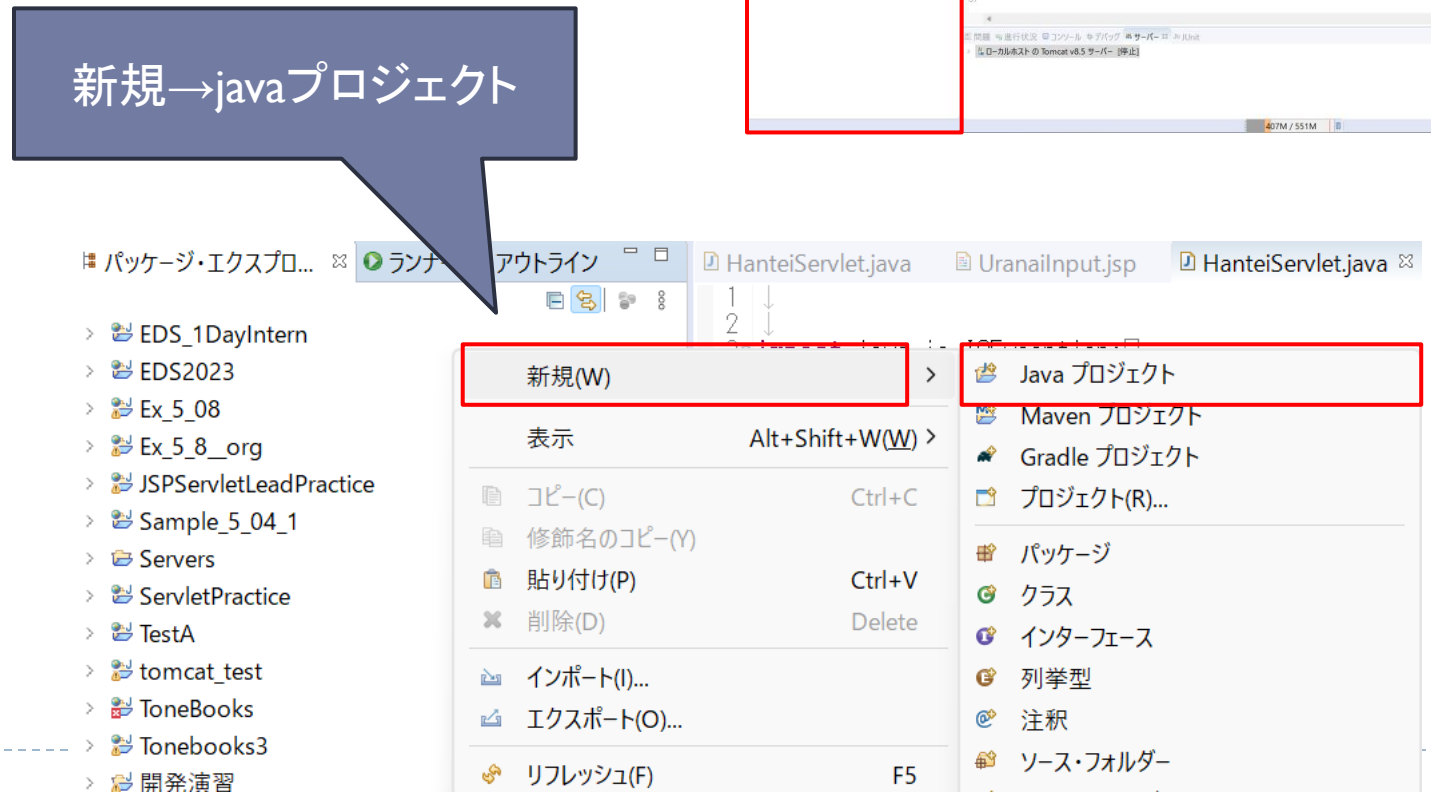
（パッケージ名は原則として頭文字を小文字とする）



パッケージ作成①

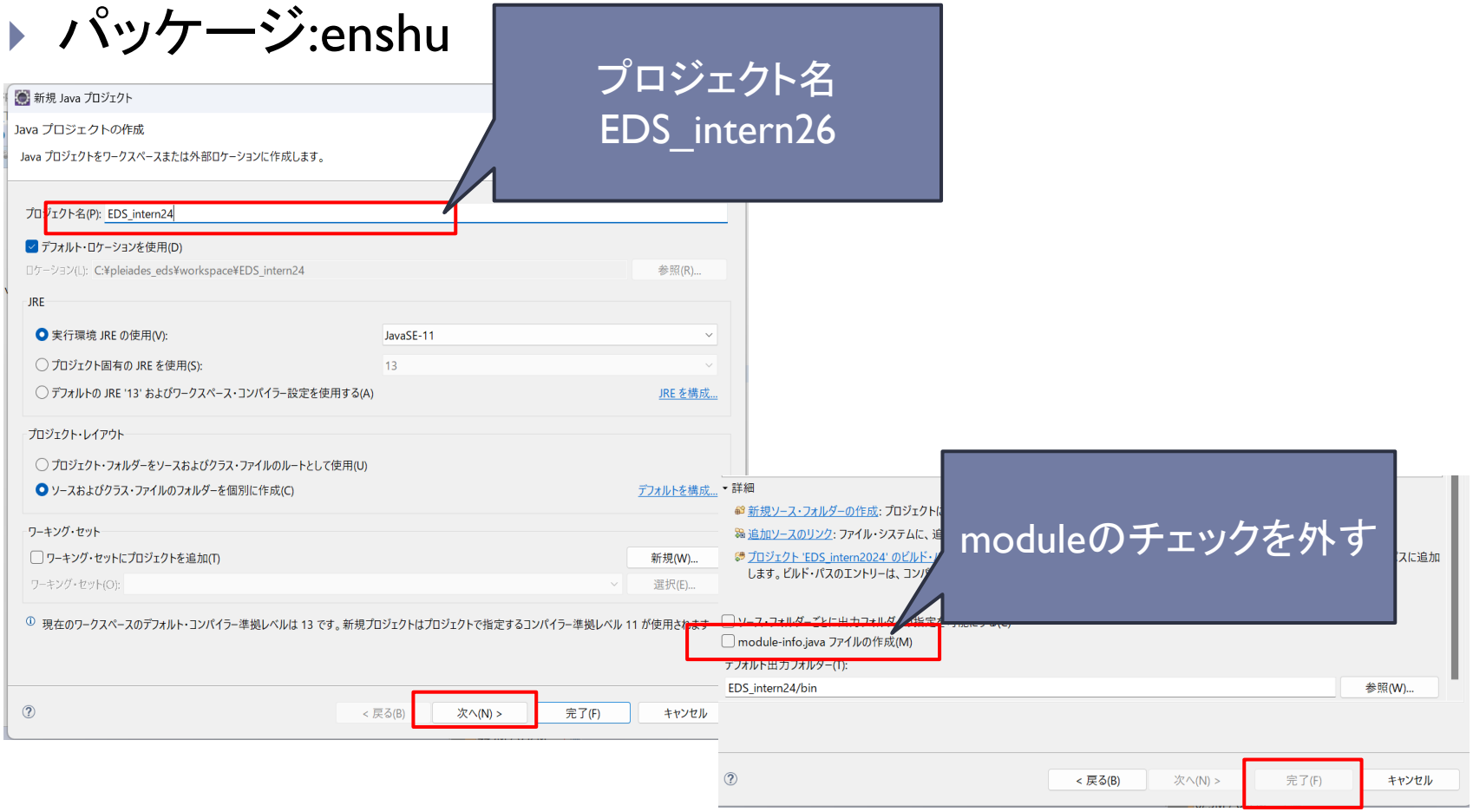
- プロジェクト:EDS_intern26
- パッケージ:enshu





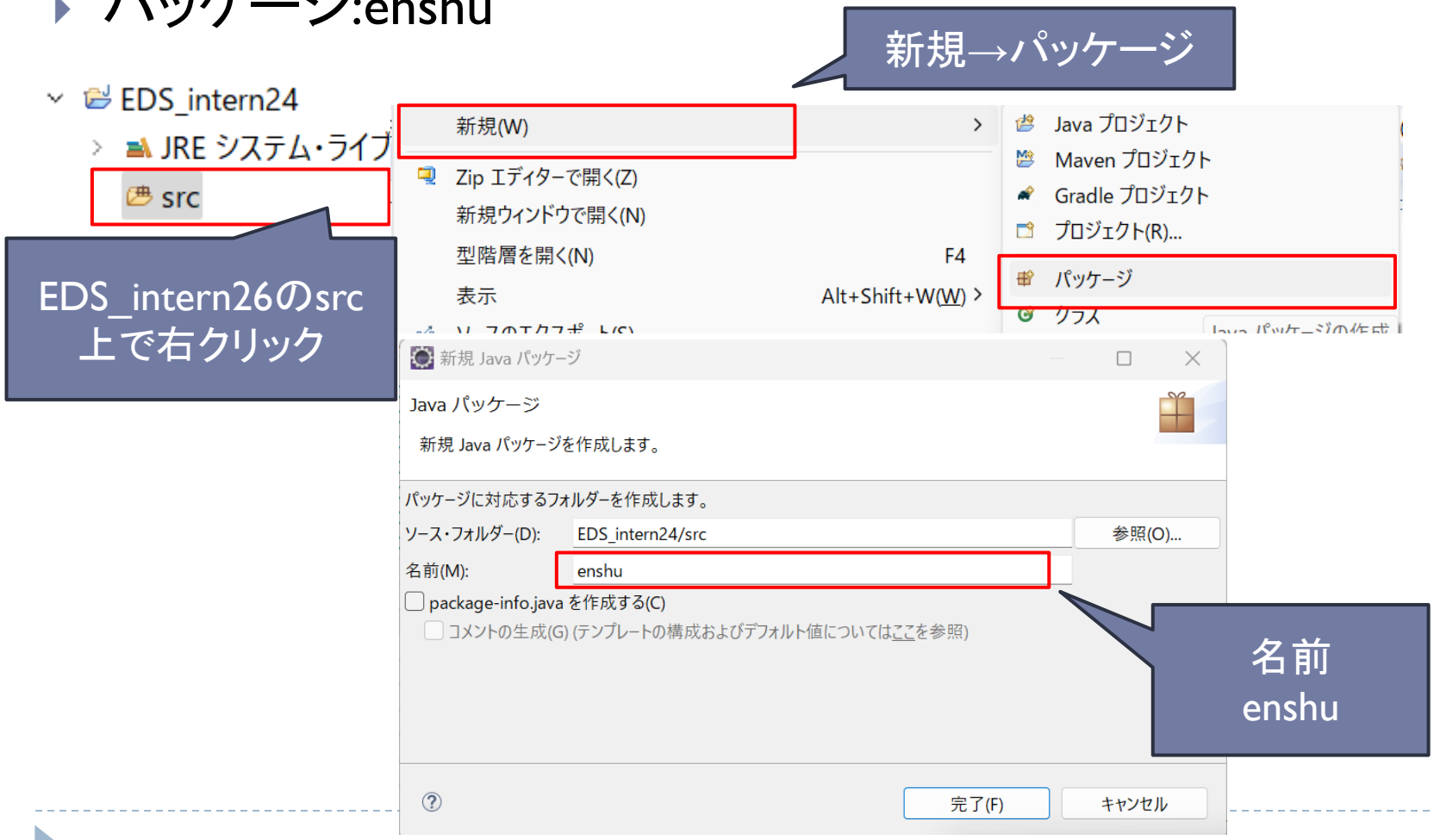
パッケージ作成②

- プロジェクト:EDS_intern26
- パッケージ:enshu



パッケージ作成③

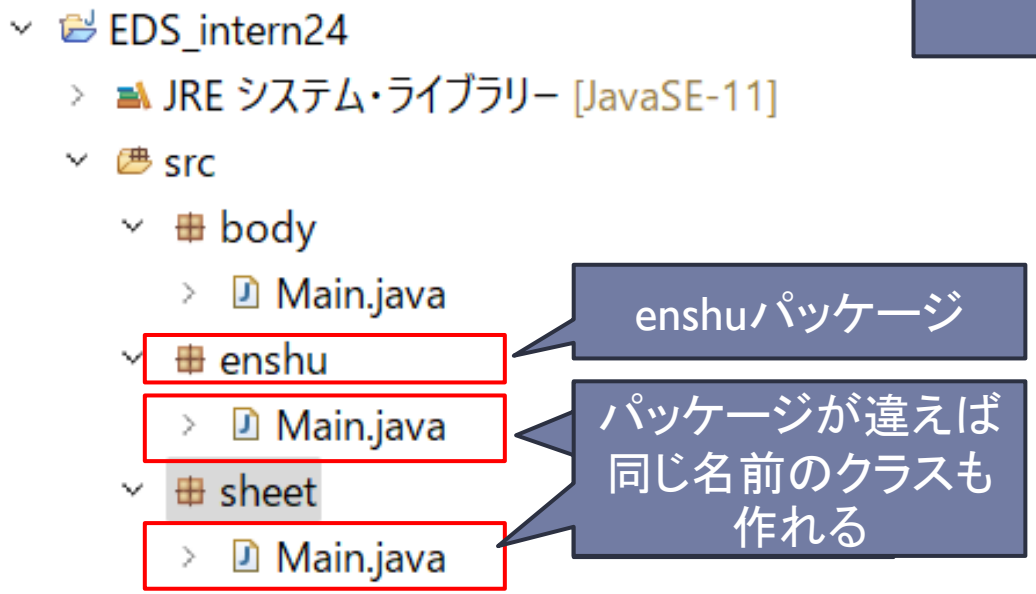
- プロジェクト:EDS_intern26
- パッケージ:enshu



パッケージ作成④

- プロジェクト:EDS_intern26
- パッケージ:enshu

明確な決まりはないがある程度の部品、
機能単位でパッケージを作る。





Javaの基本構成

Variable.java

```
1  package exercises;
2
3  public class Variable {
4
5      public static void main(String[] args) {
6
7          int tax=8;
8          System.out.println("消費税は"+tax+"%です。");
9
10         String name="山田";
11         System.out.println("私の名前は"+name+"です。");
12
13     }
14
15 }
16
```

①

②

③

④

①パッケージ宣言（前述）
②クラス（～.javaが一つのクラス）
③メソッド（後述）
④処理（1行単位で記載する。末尾に;をつける）



変数

- Javaの変数

型名　変数名　（＝初期値）;

例：

Variable.java

```
1  package exercises;
2
3  public class Variable {
4
5      public static void main(String[] args) {
6
7          int tax=8;
8          System.out.println("消費税は"+tax+"%です。");
9
10         String name="山田";
11         System.out.println("私の名前は"+name+"です。");
12
13     }
14
15 }
16
```

問題　コンソール　LogCat

<終了> Variable [Java アプリケーション] C:¥pleiades¥ecli

```
消費税は8%です。
私の名前は山田です。
```

データ型	型名	内容
整数型	int	32ビットの整数を格納
実数型	double	64ビットの実数を格納
文字型	String	「“ ”」で囲んだ文字列の格納



メソッド

・メソッド　クラスが持つ機能を定義するためのもの

→ { から } までが範囲となる

```
戻り値の型　メソッド名（引数）{
            実行したい処理
}
```

・mainメソッド

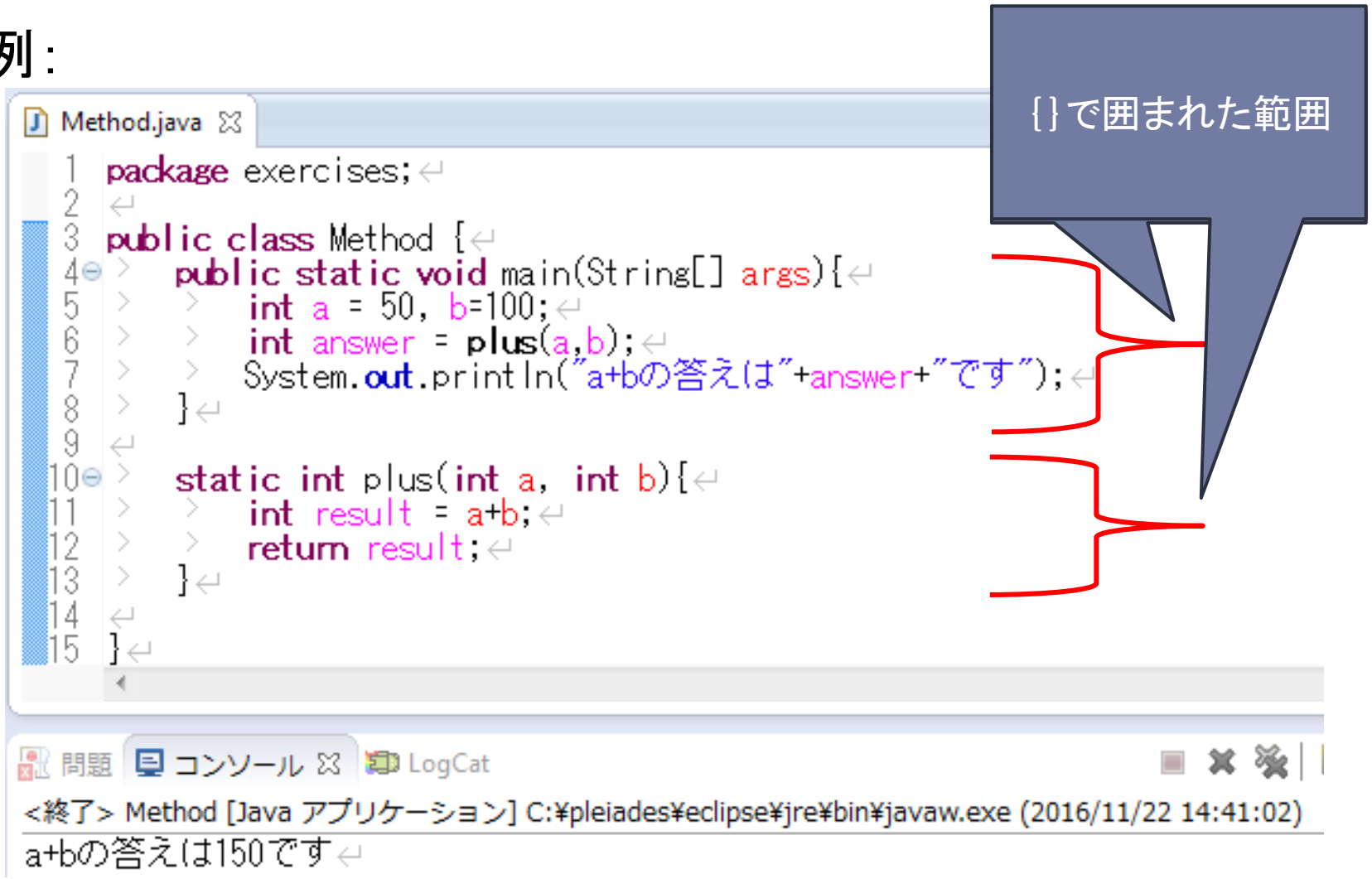
→プログラムを実行したときに最初に実行されるメソッド（一つのクラスに一つまで）

```
public  static void main(String[] args){
            実行したい処理
}
```



メソッド　例

例：


{}で囲まれた範囲
Method.java
1 package exercises;
2
3 public class Method {
4 public static void main(String[] args){
5 int a = 50, b=100;
6 int answer = plus(a,b);
7 System.out.println("a+bの答えは"+answer+"です");
8 }
9
10 static int plus(int a, int b){
11 int result = a+b;
12 return result;
13 }
14
15 }
問題
コンソール
LogCat
<終了> Method [Java アプリケーション] C:¥pleiades¥eclipse¥jre¥bin¥javaw.exe (2016/11/22 14:41:02)
a+bの答えは150です

フィールドとローカル変数

- フィールド
  - クラス全体で定義される変数

- ローカル変数
  - メソッド内で定義される変数



フィールドとローカル変数　例

例：

フィールド

ローカル変数

宣言した{}外では使えない

Field_Local.java

```
1  package exercises;
2
3  public class Field_Local {
4      String name = "山田";
5
6      public static void main(String[] args) {
7          int shoes = 3, sandals = 2;
8          int total = plus(shoes, sandals);
9          System.out.println("山田は靴とサンダルを合計" + total + "足持っている。");
10     }
11
12     static int plus(int a, int b) {
13         int total = a + b;
14         return total;
15
16     }
17 }
```

問題　コンソール　LogCat

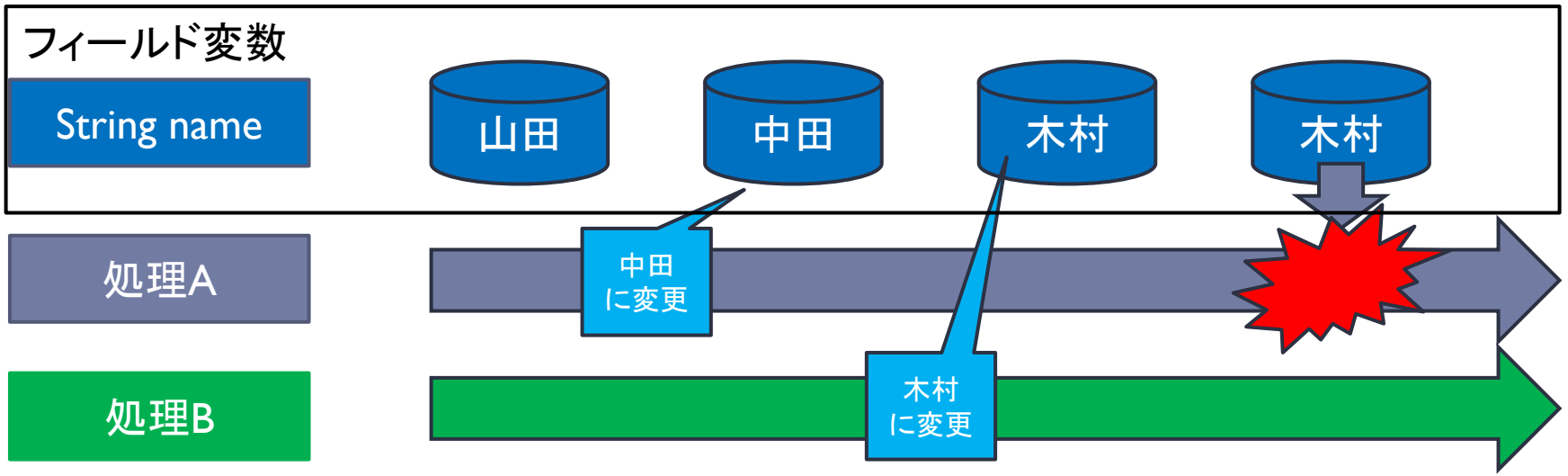
<終了> Field_Local [Java アプリケーション] C:¥pleiades¥eclipse¥jre¥bin¥javaw.exe (2016/11/22 14:42:32)

山田は靴とサンダルを合計5足持っている。



フィールドとローカル変数　例

なんでもフィールド変数にすると・・・




```
package exercises;

public class Field_Local {
    String name = "山田";
```

スレッド（並行）処理で実行した場合、書き換えられて予期せぬ値になってしまう事がある

キャスト演算子

- キャスト・・・変数の型を別の型に変換する演算子
- 暗黙のキャスト
  - 精度の低いものから高いものへのキャスト演算子は省略できる

```
1  package exercises;
2
3  public class Cast {
4
5      public static void main(String[] args) {
6
7          int a = 100;
8
9          double b = a;
10
11         System.out.println("a=" + a);
12         System.out.println("b=" + b);
13
14         double c = 3.1415;
15
16         int d = (int) c;
17
18         System.out.println("c=" + c);
19         System.out.println("d=" + d);
20
21     }
22
23 }
```

マーカー　プロパティー　サーバー　データ・ソース・エクスプローラー　スニペット　コンソール

```
<終了> Cast [Java アプリケーション] C:¥pleiades¥eclipse¥jre¥bin¥javaw.exe (2016/11/22 17:13:23)
a=100
b=100.0
c=3.1415
d=3
```

省略できる	省略できない
int → double	double → int

演算子	内容	表記例
(型名)値	指定された値を、()内に記述された型に変換する。	(int) a

制御文

- Javaにおける制御文
  - 分岐文
    - if文
  - 繰り返し文
    - for文
    - while文、do～while文



分岐文

- if文

```
if （条件式1){
        文1;
} else if （条件式2){
        文2;
} else {
        文3;
}
```

例：

If.java

```
1  package exercises;
2
3  public class If {
4
5      public static void main(String[] args) {
6
7          int a = 0, b = 2;
8
9          if (a > b) {
10             System.out.println("aはbより大きい。");
11         } else if (a < b) {
12             System.out.println("aはbより小さい。");
13         } else {
14             System.out.println("aとbは同じである。");
15         }
16     }
17 }
```

0 > 2なので
False(偽)

0 < 2なので
true(真)

問題　コンソール　LogCat

<終了> If [Java アプリケーション] C:¥pleiades¥eclipse¥jre¥bin¥javaw.exe (2016/11/22 15:15:22)

aはbより小さい。



文字列の比較問題①

以下を実行したら結果はどうなるでしょう？

```java
public static void main(String[] args) {
    // 作成したリストをコピーする
    String aaa = "エヴァ";
    String bbb = "エヴァ";
    String ccc = aaa;
    String ddd = new String(aaa);
    System.out.println(aaa);
    System.out.println(bbb);
    System.out.println(ccc);
    System.out.println(ddd);

    if (aaa == bbb) {
        System.out.println("aaa == bbb");
    }
    if (aaa == ccc) {
        System.out.println("aaa == ccc");
    }
    if (aaa == "エヴァ") {
        System.out.println("aaa == エヴァ");
    }
    if (aaa == ddd) {
        System.out.println("aaa == ddd");
    }
}
```

```
<終了> Aaa [Java アプリケーション] C:¥ple
エヴァ
エヴァ
エヴァ
エヴァ
aaa == bbb
aaa == ccc
aaa == エヴァ
```

aaa～ddd
すべて「エヴァ」だが
aaa == ddd
だけはtrueにならない
（aaa == dddが出力されない）



文字列の比較問題②

以下を実行したら結果はどうなるでしょう？

```
public static void main(String[] args) {
    // 作成したリストをコピーする
    String aaa = "エヴァ";
    String bbb = "エヴァ";
    String ccc = aaa;
    String ddd = new String(aaa);
    System.out.println(aaa);
    System.out.println(bbb);
    System.out.println(ccc);
    System.out.println(ddd);

    if (aaa.equals(bbb)) {
        System.out.println("aaa == bbb");
    }
    if (aaa.equals(ccc)) {
        System.out.println("aaa == ccc");
    }
    if (aaa.equals("エヴァ")) {
        System.out.println("aaa == エヴァ");
    }
    if (aaa.equals(ddd)) {
        System.out.println("aaa == ddd");
    }
}
```

```
<終了> Aaa [Java アプリケーション] C
エヴァ
エヴァ
エヴァ
エヴァ
aaa == bbb
aaa == ccc
aaa == エヴァ
aaa == ddd
```

文字列の比較は
equalsメソッド
を使うと正しく行える
（aaa == dddが出力される）

繰り返し文

- for文

  ループする回数が決まっているとき有効

例：

For.java

```
1  package exercises;
2
3  public class For {
4
5      public static void main(String[] args) {
6
7          for (int i = 0; i < 10; i++) {
8              System.out.println(i);
9              if (i > 4) {
10                 break;
11             }
12         }
13     }
14
15 }
```

```
for （カウント変数宣言; 条件式 ; カウント){
        文1;
        文2;
        ・・・
}
```

break文
制御を中断する
　　反復処理を途中で中断して終了

問題　コンソール　LogCat

<終了> For (1)　　e¥bin¥javaw.exe (2016/11/22 14:35:27)

```
0
1
2
3
4
5
```

for文が0～9までループ
If文で4より大きくなったら
Breakするため5まで

繰り返し文

- while文

  繰り返す回数が決まっていない場合

  主に**バッチ処理**（後述）で使用

```
while （条件式）{
        文1;
        文2;
        ・・・
}

// 条件式がtrueの間はずっと実行される
```

flgが「false」になるまでの
5回実行された

test.java

```
1
2  public class test {
3
4      public static void main(String[] args) {
5          String local = "こんばんわ";
6          int i = 0;
7          boolean flg = true;
8          while (flg) {
9              i++; // 1ずつカウントする
10             System.out.println(local);
11             if (i > 5) {
12                 flg = false;
13             }
14         }
15     }
16 }
17
```

問題　進行状況　コンソール　デバッグ　サーバー　JUnit

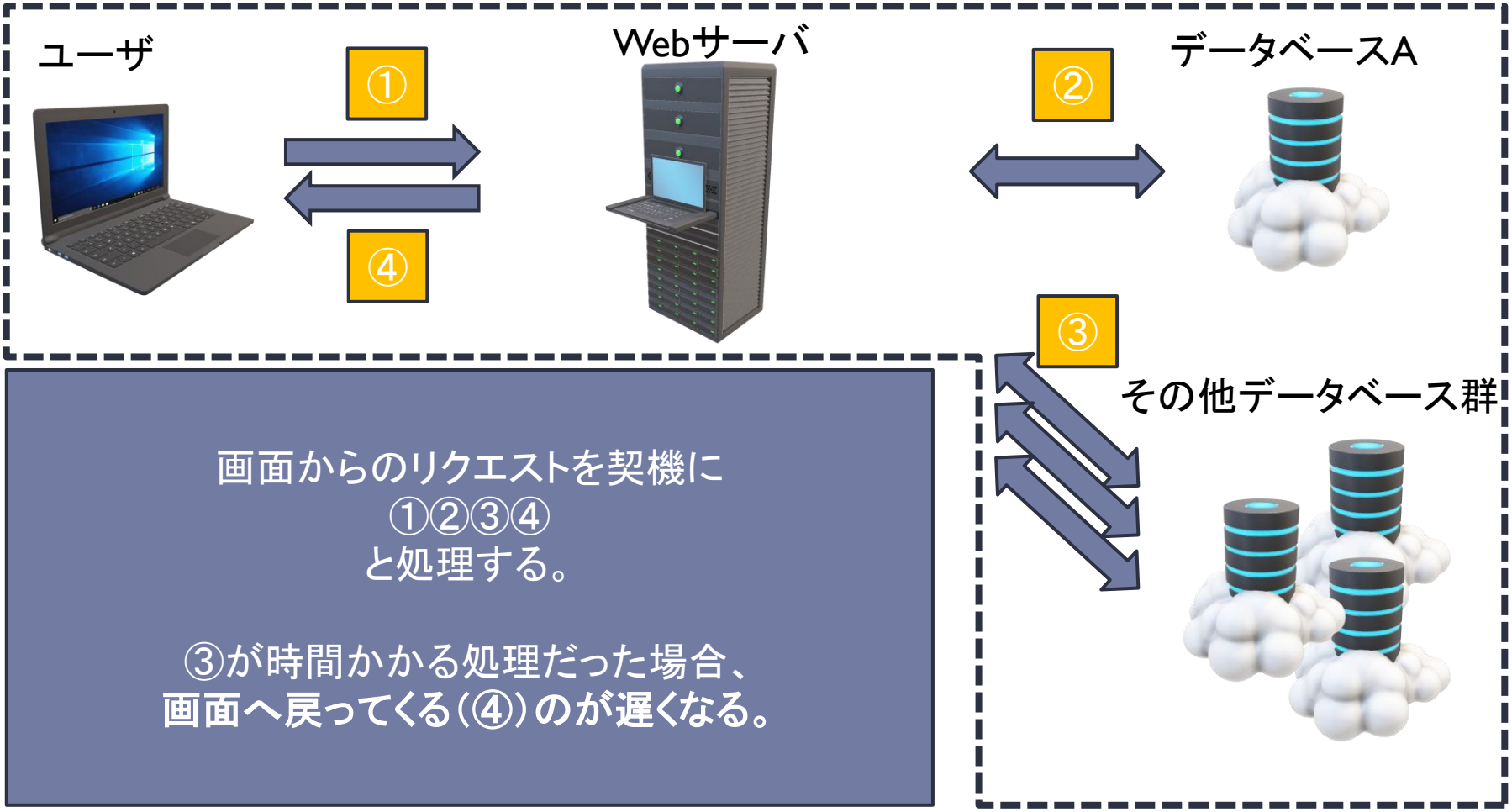
<終了> test (1) [Java アプリケーション] C:¥pleiades¥java¥8¥bin¥javaw.e

```
こんばんわ
こんばんわ
こんばんわ
こんばんわ
こんばんわ
こんばんわ
```



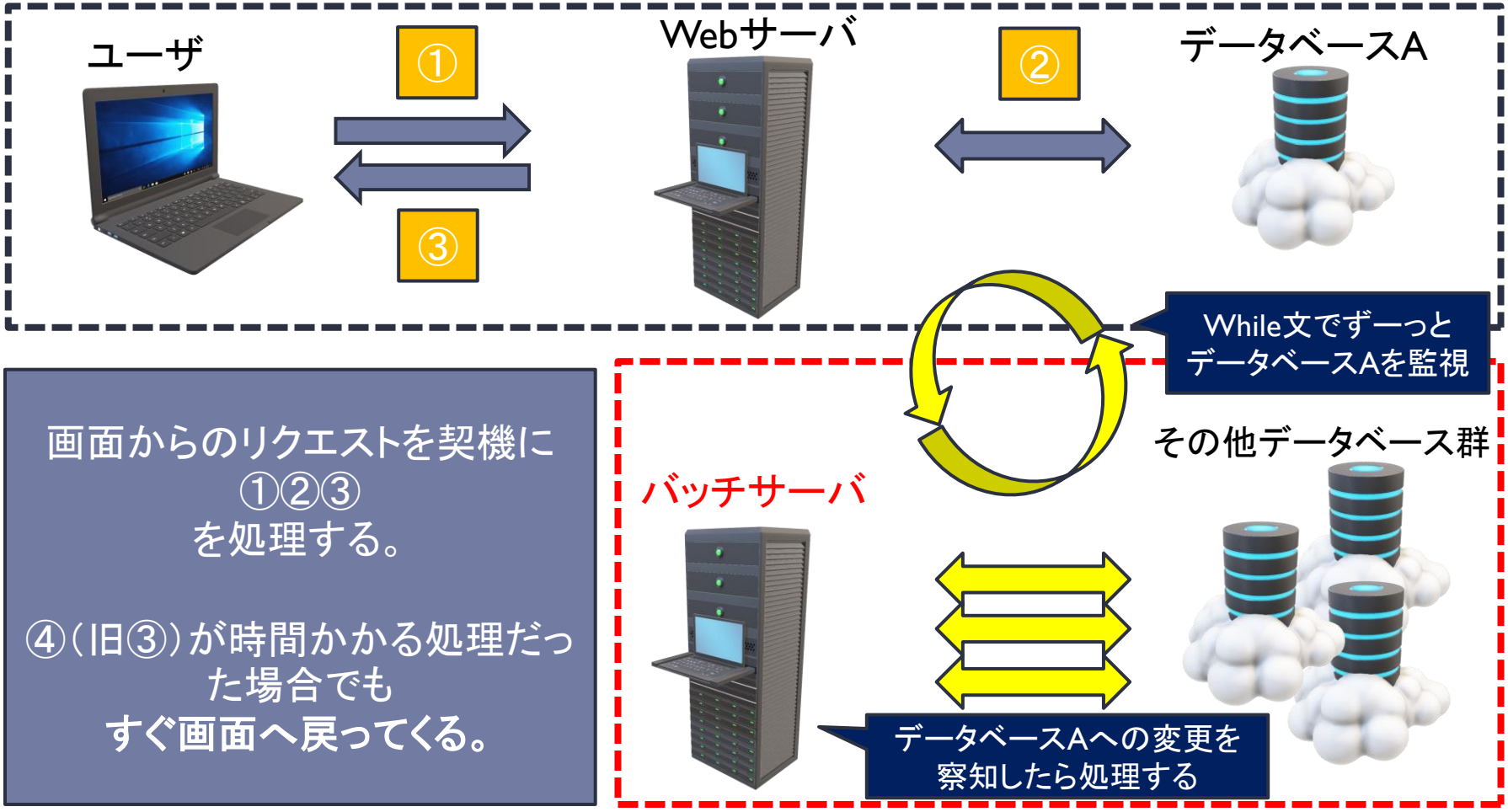
繰り返し文

- while文の使い道（バッチ処理なし、とある登録処理）



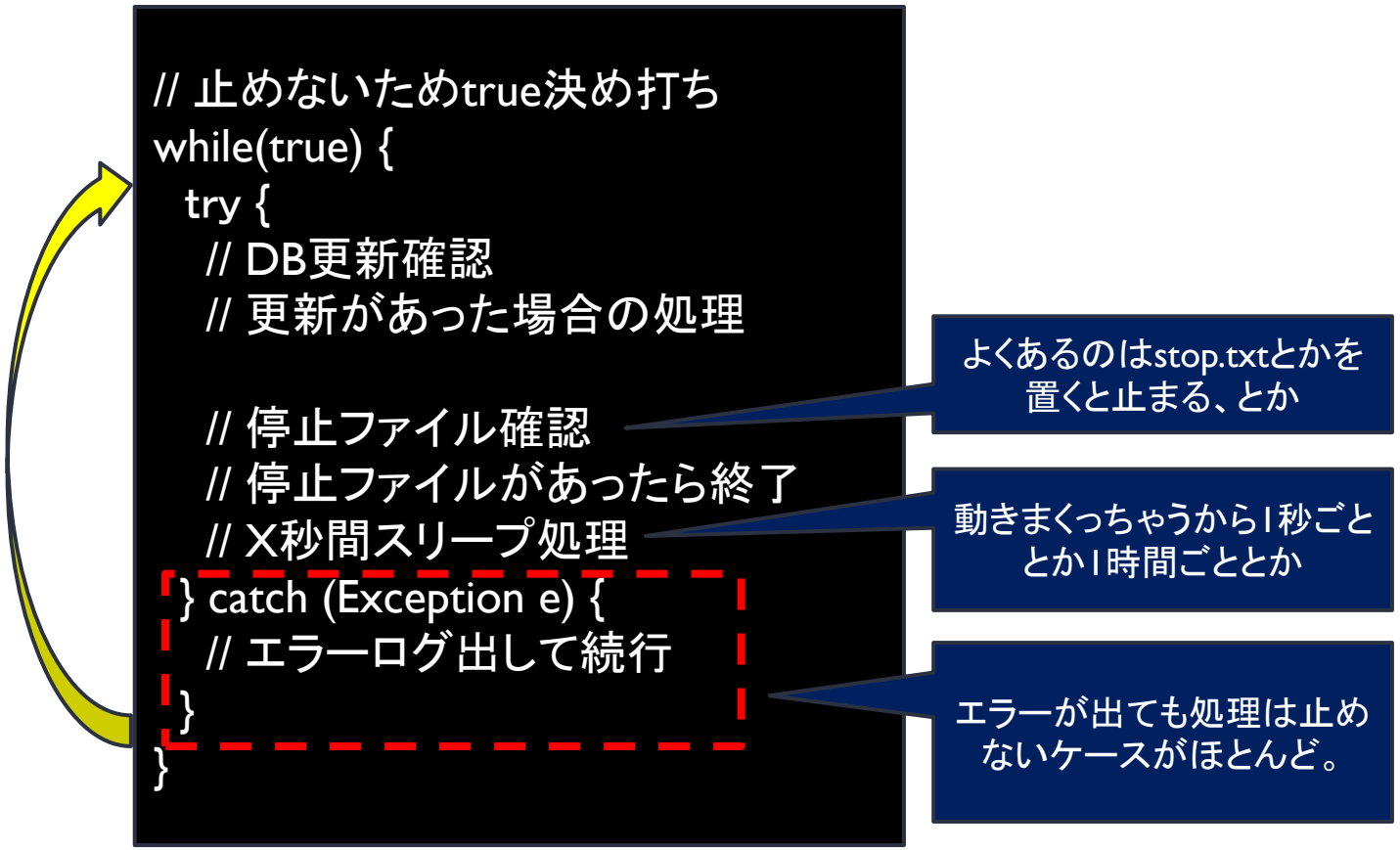
繰り返し文

- while文の使い道（バッチ処理あり、とある登録処理）



繰り返し文

- while文の使い道（バッチ処理あり）
  ソースイメージ（WhileTest.java参照）



繰り返し文

- do～while文

  少なくとも一度は
  ループを行う場合

```
do {
        文1;
        文2;
        ・・・
} while(条件式);
```

例：

Do_While.java

```
1  package exercises;
2
3  public class Do_While {
4
5      public static void main(String[] args) {
6          int i = 5;
7          do {
8              System.out.println("i=" + i);
9              i = i - 3;
10         } while (i > 0);
11         System.out.println("0を下回ったので終了です");
12
13         System.out.println("======================");
14
15         i = 2;
16         do {
17             System.out.println("i=" + i);
18             i = i - 3;
19         } while (i > 0);
20         System.out.println("0を下回ったので終了です");
21     }
22
23 }
```

問題　コンソール　LogCat

```
<終了> Do_While [Java アプリケーション] C:¥pleiades¥eclipse¥jre¥bin¥javaw.exe (2016/11/22 15:19:22)
i=5
i=2
0を下回ったので終了です
======================
i=2
0を下回ったので終了です
```



配列

- Javaの配列

型名[]　配列名　＝{要素1,要素2・・・};

例：

Array.java

```
1  package exercises;
2
3  public class Array {
4
5      public static void main(String[] args) {
6          String[] week = { "月", "火", "水", "木", "金", "土", "日" };
7
8          for (int i = 0; i < week.length; i++) {
9              System.out.println(week[i] + "曜日");
10         }
11     }
12
13 }
```

問題　コンソール　LogCat

<終了> Array [Java アプリケーション] C:¥pleiades¥eclipse¥jre¥bin¥java

```
月曜日
火曜日
水曜日
木曜日
金曜日
土曜日
日曜日
```

week[0] → 月
week[1] → 火
week[2] → 水
week[3] → 木
week[4] → 金
week[5] → 土
week[6] → 日

配列の繰り返し文

- 拡張for構文

→配列の中の要素を変数定義に繰り返し表示する文

Extension_For.java

```
1  package exercises;
2
3  public class Extension_For {
4
5      public static void main(String[] args) {
6
7          int[] age = { 22, 23, 21, 50 };
8
9          int result = 0;
10         for (int a : age) {
11             result += a;
12             System.out.println("年齢は" + a + "である");
13         }
14         System.out.println("4人の年齢の合計は" + result + "である");
15     }
16 }
```

```
for （変数定義 : 配列名){
        文1;
        文2;
        ・・・
}
```

問題　コンソール　LogCat

<終了> Extension_For [Java アプリケーション] C:¥pleiades¥eclipse¥jre¥bin¥javaw.exe (2016/11/22 15:45:23)

```
年齢は22である
年齢は23である
年齢は21である
年齢は50である
4人の年齢の合計は116である
```

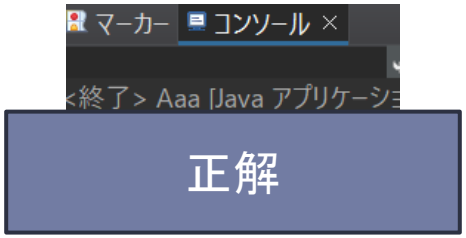
age[0] → 22
age[1] → 23
age[2] → 21
age[3] → 50



配列の問題

以下を実行したら結果はどうなるでしょう？

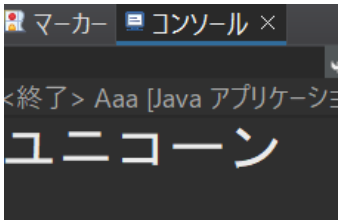
```
public static void main(String[] args) {
    // 作成した配列をコピーする
    String[] gundams1 = { "ZZ", "SEED", "F91" };
    String[] gundams2 = gundams1;
    // 片方の値を変更する
    gundams1[0] = "ユニコーン";
    // 実行結果はどうなる？
    System.out.println(gundams2[0]);
}
```



配列の問題

以下を実行したら結果はどうなるでしょう？

```
public static void main(String[] args) {
    // 作成した配列をコピーする
    String[] gundams1 = { "ZZ", "SEED", "F91" };
    String[] gundams2 = gundams1;
    // 片方の値を変更する
    gundams1[0] = "ユニコーン";
    // 実行結果はどうなる？
    System.out.println(gundams2[0]);
}
```

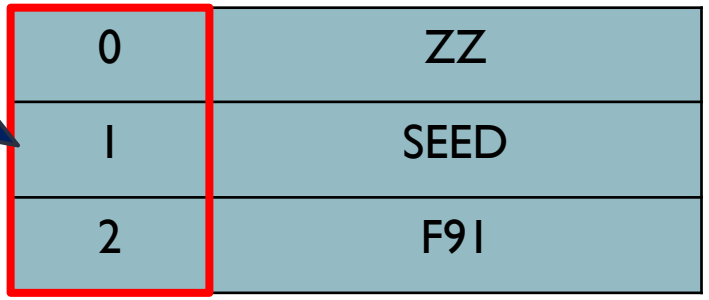




配列の問題

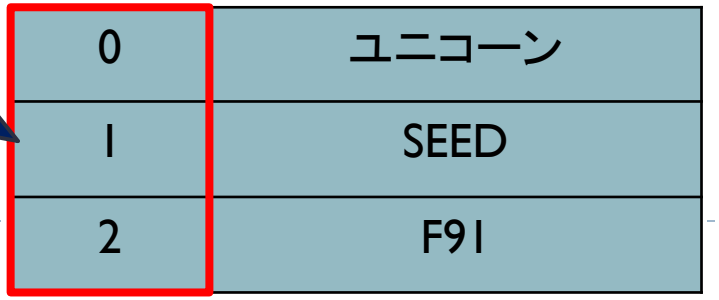
```
// 作成した配列をコピーする
String[] gundams1 = { "ZZ", "SEED", "F91" };
String[] gundams2 = gundams1;
```

配列の変数には、配列のアドレス情報(0.1.2)が保管されている。
配列の情報自体(ZZ,SEED,F91)は別の場所で保管されている。
→gundams1もgundams2も同じ配列を参照している。




```
// 片方の値を変更する
gundams1[0] = "ユニコーン";
// 実行結果はどうなる？
System.out.println(gundams2[0]);
```

同じ配列を参照しているので、gundams1を変更すると
同じ配列を参照していているgundams2も変更される



ArrayList（可変長配列）

- ArrayListとは

→定義後に大きさを自由に変更できる配列

- ArrayListの特徴
  - 格納できるデータがインスタンスに限る！

ArrayList<クラス名> ArrayList名 ＝ new ArrayList<クラス名>();

例：

```
ArrayList<Person> yamadaList = new ArrayList<Person>();
ArrayList<Person> tanakaList = new ArrayList<Person>(25);
```



ArrayListクラスのメソッド

- add(Object)
  - リストの末尾に引数の内容を追加
- set(int , Object)
  - リストのintで指定した箇所に追加
- get(int)
  - リストの指定した箇所の要素の値を返す
- remove(int)
  - リストのintで指定した要素を削除
- size()
  - リストのサイズ(要素の数)を返す



ArrayList　例

例：

RobotList.java

```
1  package advance;
2
3  import java.util.ArrayList;
4
5  public class RobotList {
6
7      public static void main(String[] args) {
8
9          Robot r1 = new Robot("R3-D3", 50);
10         Robot r2 = new Robot("C2PO", 70);
11         Robot r3 = new Robot("どら衛門", 110);
12
13         ArrayList<Robot> list = new ArrayList<Robot>();
14         list.add(r1);
15         list.add(r2);
16         list.add(r3);
17
18         for (Robot ri : list) {
19             ri.selfintroduction();
20         }
21
22     }
23
24 }
```

Robot.java

```
1  package advance;
2
3  public class Robot {
4      private String name;
5      private int weight;
6
7      public Robot(String name, int weight) {
8          this.name = name;
9          this.weight = weight;
10     }
11
12     void selfintroduction() {
13         System.out.println("機体名は" + name + "、重量は" + weight + "kgです");
14     }
15
16     public String getName() {
17         return name;
18     }
19
20     public void setName(String name) {
21         this.name = name;
22     }
23
24     public int getWeight() {
25         return weight;
26     }
27
28     public void setWeight(int weight) {
29         this.weight = weight;
30     }
31
32 }
```

問題　コンソール　LogCat

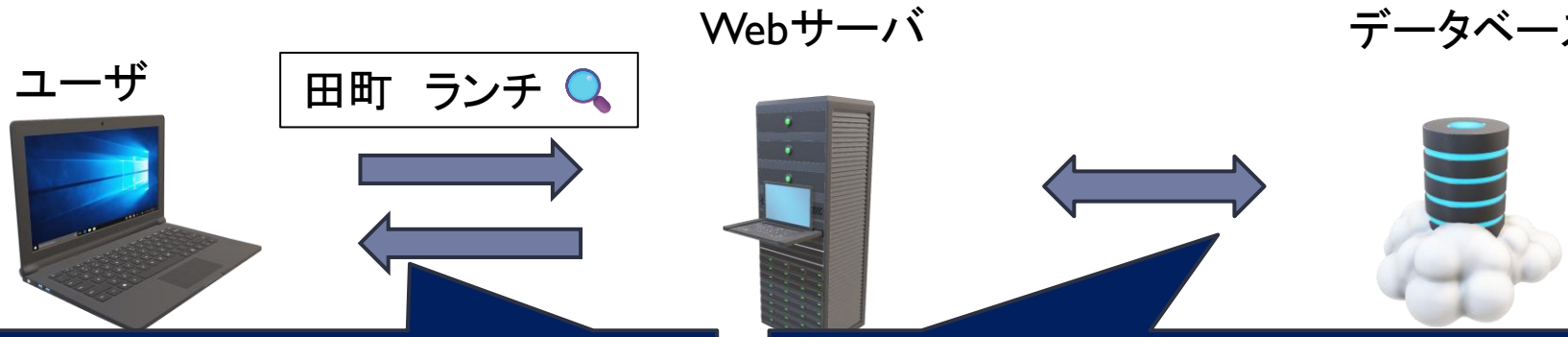
<終了> RobotList [Java アプリケーション] C:¥pleiades¥eclipse¥jre¥bin¥javaw.exe (2016/11/22 16:32:05)

```
機体名はR3-D3、重量は50kgです
機体名はC2PO、重量は70kgです
機体名はどら衛門、重量は110kgです
```



ArrayListと配列の使用例

可変のものはArrayList（データベースから取得したデータとか）
固定のものは配列（1:女性　2:男性とか）がいいかな



画面表示

■えくしお
東京都田町　ランチ　禁煙

■デジタル
東京都田町　ランチ　喫煙

■EDS
東京都田町　ランチ　喫煙

※フラグを文字列に変換して表示

条件に合う件数がわからないので**ArrayList**で取得する
※下の図は三件

店名	住所	ランチ	喫煙フラグ
えくしお	東京都田町	1	0
デジタル	東京都田町	1	1
EDS	東京都田町	1	1

```
String tabako[] = {“禁煙”,”喫煙”};
// 上記のように定義して画面表示用に変換する
```

Listの問題

以下を実行したら結果はどうなるでしょう？

```
public static void main(String[] args) {
    // 作成したリストをコピーする
    ArrayList<String> gundams1 = new ArrayList<String>();
    gundams1.add("ZZ");
    gundams1.add("SEED");
    gundams1.add("F91");
    ArrayList<String> gundams2 = gundams1;
    gundams1.add("00");
    // 片方の値を変更する
    gundams1.set(0, "ユニコーン");
    // 実行結果はどうなる？
    System.out.println(gundams2.get(0));
    System.out.println(gundams2.get(3));
}
```

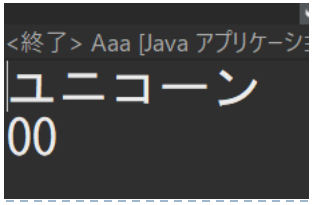
正解



Listの問題

以下を実行したら結果はどうなるでしょう？

```
public static void main(String[] args) {
    // 作成したリストをコピーする
    ArrayList<String> gundams1 = new ArrayList<String>();
    gundams1.add("ZZ");
    gundams1.add("SEED");
    gundams1.add("F91");
    ArrayList<String> gundams2 = gundams1;
    gundams1.add("00");
    // 片方の値を変更する
    gundams1.set(0, "ユニコーン");
    // 実行結果はどうなる？
    System.out.println(gundams2.get(0));
    System.out.println(gundams2.get(3));
}
```
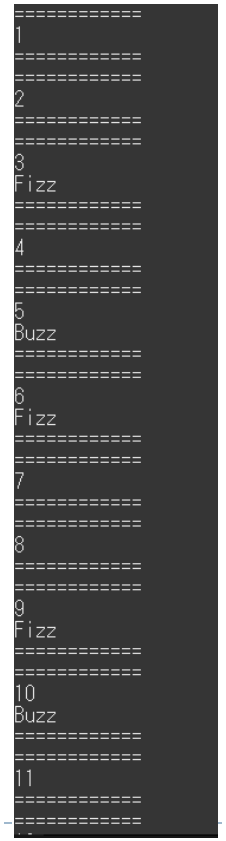


(ユニコーン,SEED,F91,00)



Java練習問題

- 練習１～６に取り組んでみましょう。 ※それぞれコンソールに表示
- 【練習１】 5,12を変数a,bに格納し、四則演算の式と結果を表示する
- 【練習２】 if文を使って0と12の四則演算結果の
  ”正”、”0”、”負”を判定し、式と答えと結果を表示する
- 【練習３】 1～20の数字をfor文を使って表示する
- 【練習４】 練習３に関してその数が
  3で割り切れるなら“Fizz“
  5で割り切れるなら“Buzz“
  両方で割り切れるなら“FizzBuzz”
  が数字の下に表示されるようにする





Java練習問題

- 【練習5】 コンソールで各種紙幣（二千円札は除く）と500円玉、100円玉の入力を受け付け
  ①その総額を表示
  消費税の入力を受け付け
  ②総額に対しての税込み価格を表示

```
一万円札を何枚入れますか？
1
五千円札を何枚入れますか？
3
千円札を何枚入れますか？
2
五百円玉を何枚入れますか？
7
百円玉を何枚入れますか？
1
消費税は何%ですか？
10
税抜金額は ： 30600
税込金額は ： 33660
```



Java練習問題

- 【練習６】 練習５に対して支払い金額を入力し、
  お釣りを出力する処理を追加する

```
一万円札を何枚入れますか？
1
五千円札を何枚入れますか？
1
千円札を何枚入れますか？
1
五百円玉を何枚入れますか？
1
百円玉を何枚入れますか？
1
消費税は何%ですか？
10
税抜金額は ： 16600
税込金額は ： 18260
支払い金額を入力してください。
20000
お釣りは ： 1740
```

```
一万円札を何枚入れますか？
1
五千円札を何枚入れますか？
1
千円札を何枚入れますか？
1
五百円玉を何枚入れますか？
1
百円玉を何枚入れますか？
1
消費税は何%ですか？
10
税抜金額は ： 16600
税込金額は ： 18260
支払い金額を入力してください。
10000
支払い金額が足りません！
```

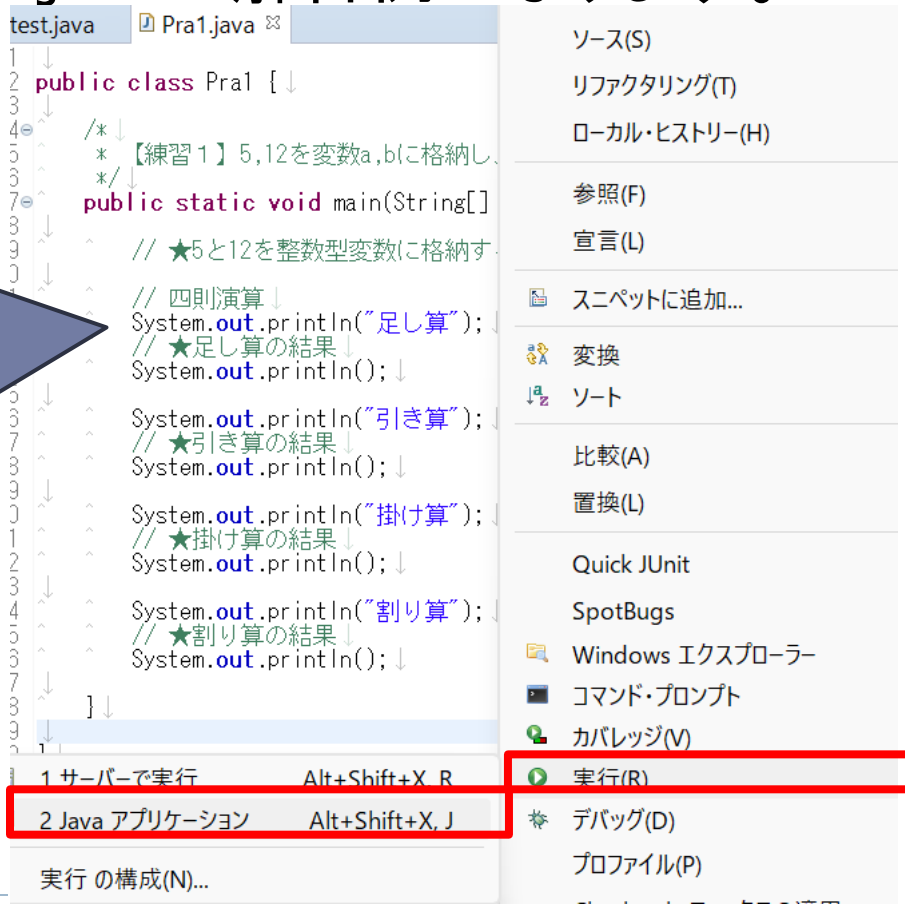


Java練習問題

- 練習問題1～6がPra1～6に対応してます。
  PraAnsの中にあるPra〇Ans.javaが解答例になります。

この辺で右クリック
「実行」「javaアプリケーション」
で実行出来るので動かしながら
やってみて下さい！

「必要なプロジェクトでエラー」
って出ますが
「続行」
で大丈夫です！



答え合わせ　練習１

- 【練習１】 5,12を変数a,bに格納し、四則演算の式と結果を表示する

pra1.java

```
public static void main(String[] args) {
    // 5と12を整数型変数に格納する
    int a = 5;
    int b = 12;

    // 四則演算
    System.out.println("足し算");
    System.out.println(a + b);
    System.out.println("引き算");
    System.out.println(a - b);
    System.out.println("掛け算");
    System.out.println(a * b);
    System.out.println("割り算");
    System.out.println(a / b); // 0になってしまう
    System.out.println((double) a / b); // 小数点も表示する

}
```

問題　進行状況　コンソール　デバッグ　サーバー　JUnit

```
<終了> pra1 [Java アプリケーション] C:¥pleiades¥java¥8¥bin¥javaw.exe (2023/07/24 15:51:31)
足し算
17
引き算
-7
掛け算
60
割り算
0
0.4166666666666667
```

＜ポイント＞
①コンソール表示は”System.out.println()”
②割り算結果はdouble型にキャスト

答え合わせ　練習２

- 【練習2】 if文を使って0と12の四則演算結果の”正”、”0”、”負”を判定し、式と答えと結果を表示する

```
public static void main(String[] args) {

    int a = 0;
    int b = 12;

    // 四則演算
    // ★足し算の結果を格納する
    int aTASUb = a + b;
    // 式を表示する（改行無）
    System.out.print("a + b = " + aTASUb);
    // if文を使って判定結果を表示する
    if (aTASUb == 0) { // ★足し算の結果が0かを判定
        System.out.println("  判定結果：0");
    } else if (aTASUb > 0) { // ★足し算の結果が0より大きいかを判定
        System.out.println("  判定結果：正");
    } else if (aTASUb < 0) { // ★足し算の結果が0より小さいかを判定
        System.out.println("  判定結果：負");
    }

    // ★引き算の結果を格納する
    int aHIKUb = a - b;
    // 式を表示する（改行無）
    System.out.print("a - b = " + aHIKUb);
    // if文を使って判定結果を表示する
    if (aHIKUb == 0) { // ★引き算の結果が0かを判定
        System.out.println("  判定結果：0");
    } else if (aHIKUb > 0) { // ★引き算の結果が0より大きいかを判定
        System.out.println("  判定結果：正");
    } else if (aHIKUb < 0) { // ★引き算の結果が0より小さいかを判定
        System.out.println("  判定結果：負");
    }


    // ★掛け算の結果を格納する
    int aKAKERUb = a * b;
    // 式を表示する（改行無）
    System.out.print("a * b = " + aKAKERUb);
    // if文を使って判定結果を表示する
    if (aKAKERUb == 0) { // ★引き算の結果が0かを判定
        System.out.println("  判定結果：0");
    } else if (aKAKERUb > 0) { // ★引き算の結果が0より大きいかを判定
        System.out.println("  判定結果：正");
    } else if (aKAKERUb < 0) { // ★引き算の結果が0より小さいかを判定
        System.out.println("  判定結果：負");
    }

    // ★割り算の結果を格納する
    int aWARIb = a / b;
    // 式を表示する（改行無）
    System.out.print("a / b = " + aWARIb);
    // if文を使って判定結果を表示する
    if (aWARIb == 0) { // ★引き算の結果が0かを判定
        System.out.println("  判定結果：0");
    } else if (aWARIb > 0) { // ★引き算の結果が0より大きいかを判定
        System.out.println("  判定結果：正");
    } else if (aWARIb < 0) { // ★引き算の結果が0より小さいかを判定
        System.out.println("  判定結果：負");
    }
```

＜ポイント＞
①””で区切る位置の確認
②printlnとprintの使い分け
③共通して表示する部分はif文の外に
④見やすさを考えた表示をする

```
a + b = 12  判定結果：正
a - b = -12  判定結果：負
a * b = 0  判定結果：0
a / b = 0  判定結果：0
```

割り算なので他の数字が与えられた時のことを考えてdouble型でもOK

答え合わせ　練習３

- 【練習3】 1～20の数字をfor文を使って表示する

pra1.jsp　pra2.jsp　pra3.jsp　pra4.jsp　HanteiServlet.java

```
1
2  public class Pra3 {
3
4      public static void main(String[] args) {
5
6          for(int i=1;i<=20;i++){
7
8              System.out.println(i);
9
10         }
11
12     }
13
14 }
15
```

＜ポイント＞
①for文の使い方を確認
②「i<21」でもOK



答え合わせ　練習４

- 練習3に関してその数が3で割り切れるなら“Fizz“、5で割り切れるなら“Buzz“
- 　　両方で割り切れるなら“FizzBuzz”が数字の下に表示されるようにする

```
package PraAns;

public class Pra4Ans {

    /*
     * 練習３に関してその数が
     * ・3で割り切れるなら “Fizz “、
     * ・5で割り切れるなら “Buzz “
     * ・両方で割り切れるなら “FizzBuzz”
     * が数字の下に表示されるようにする
     */
    public static void main(String[] args) {

        // 1～20の数だけループする処理
        for (int i = 1; i <= 20; i++) {
            System.out.println("============"); // 見やすいように線
            System.out.println(i); // カウントアップした変数を出力

            if (i % 3 == 0 && i % 5 == 0) { // ★両方で割り切れるな
                System.out.println("FizzBuzz");
            } else if (i % 3 == 0) {
                System.out.println("Fizz"); // ★3で割り切れるなら “Fizz “
            } else if (i % 5 == 0) {
                System.out.println("Buzz"); // ★5で割り切れるなら “Buzz “
            }
            System.out.println("============"); // 見やすいように線を入れてます

        }

    }

}
```

```
============
1
============
============
2
============
============
3
Fizz
============
============
4
============
============
5
Buzz
============
============
6
Fizz
============
============
7
============
============
8
============
============
9
Fizz
============
============
10
Buzz
============
============
11
============
============
```

15の倍数判定からしないと3や5の倍数で引っかかってしまう。
※「i%n」で、iをnで割った余りを示す

＜ポイント＞
①倍数判定の順番
②if文の最後をelseにして、改行したり
「その他」という判定項目を設けることも可能

答え合わせ　練習５

- 【練習5】コンソールで各種紙幣（二千円札は除く）と500円玉、100円玉の入力を受け付け
  ①その総額を表示
  消費税の入力を受け付け
  ②総額に対しての税込み価格を表示

```
import java.util.Scanner;

public class pra5Ans {

    /*
     * 【練習5】コンソールで各種紙幣（二千円札は除く）と500円玉、100円玉の入力を受け付け
     *   ①その総額を表示
     *   消費税の入力を受け付け
     *   ②総額に対しての税込み価格を表示
     */
    public static void main(String[] args) {

        Scanner scan = new Scanner(System.in);

        // 一万円札を計算
        System.out.println("一万円札を何枚入れますか？");
        int manYen = scan.nextInt();

        // 五千円札を計算
        System.out.println("五千円札を何枚入れますか？");
        int gosenYen = scan.nextInt();

        // 千円札を計算
        System.out.println("千円札を何枚入れますか？");
        int senYen = scan.nextInt();

        // 五百円玉を計算
        System.out.println("五百円玉を何枚入れますか？");
        int gohyakuYen = scan.nextInt();

        // 百円玉を計算
        System.out.println("百円玉を何枚入れますか？");
        int hyakuYen = scan.nextInt();

        System.out.println("消費税は何%ですか？");
        int syouhiZei = scan.nextInt();

        int sum = (manYen * 10000) + (gosenYen * 5000) + (senYen * 1000) + (gohyakuYen * 500) + (hyakuYen * 100);
        double sumZei = sum * ((double) (100 + syouhiZei) / 100);
        System.out.println("税抜金額は：" + sum);
        System.out.println("税込金額は：" + (int) sumZei); // intにして小数点を丸めた

        scan.close();

    }
}
```

入力された枚数を紙幣ごとに変数に格納する

枚数に対して紙幣の金額をかけ、合計する

入力された税率に100%を足して、合計値にかける。
※小数点考慮でdouble

答え合わせ　練習６

【練習６】練習５に対して支払い金額を入力し、お釣りを出力する処理を追加する

```
42        // 支払い金額を受け付ける
43        int shiharai = scan.nextInt();
44
45        // お釣りを計算する
46        int oturi = shiharai - (int) sumZei;
47
48        // お釣りが0だった場合は不足とする
49        if (oturi < 0) {
50            System.out.println("支払い金額が足りません！");
51        } else {
52            System.out.println("お釣りは ： " + oturi);
53        }
54
55        scan.close();
56
```

支払い金額を格納する

支払い金額と総額を引き算

支払い金額が総額より
「少ないか」
を判定する



Java練習問題(追加)

- 追加練習(Add)に取り組んでみましょう。

- 【追加練習１】 整数を 10 回入力し、平均値を表示する
- ※計算は整数で行い、小数点以下は切り捨ててよい

- 【追加練習２】 数値を繰り返し入力し、合計が 100 を超えたら入力を止めて合計を表示する

```
1
2
3
4
5
6
7
8
9
10
平均は5
```

```
50
30
10
20
合計は 110
```



答え合わせ　追加練習１

- 【追加練習1】 整数を 10 回入力し、平均値を表示する

```
public class Add1 {
    public static void main(String[] args) throws IOException {
        BufferedReader br = new BufferedReader(
                new InputStreamReader(System.in));
        int sum = 0;

        for (int i = 0; i < 10; i++) {
            int value = Integer.parseInt(br.readLine());
            sum += value;
        }

        System.out.println("平均は" + (sum / 10));
    }
}
```

sumにvalueを足していく

sumを10で割って平均値を求める



答え合わせ　追加練習2

- 【追加練習2】 数値を繰り返し入力し、合計が 100 を超えたら入力を止めて合計を表示する

```
class Add2 {
    public static void main(String[] args) throws NumberFormatException, IOException {
        BufferedReader br = new BufferedReader(
            new InputStreamReader(System.in));
        int sum = 0;

        while (sum <= 100)
            sum += Integer.parseInt(br.readLine());

        System.out.println("合計は " + sum);
    }
}
```

sumが100以下であれば入力値をsumに足していく



Javaについて

- オブジェクト指向とは
- インスタンス/コンストラクタ
- カプセル化
- APIについて

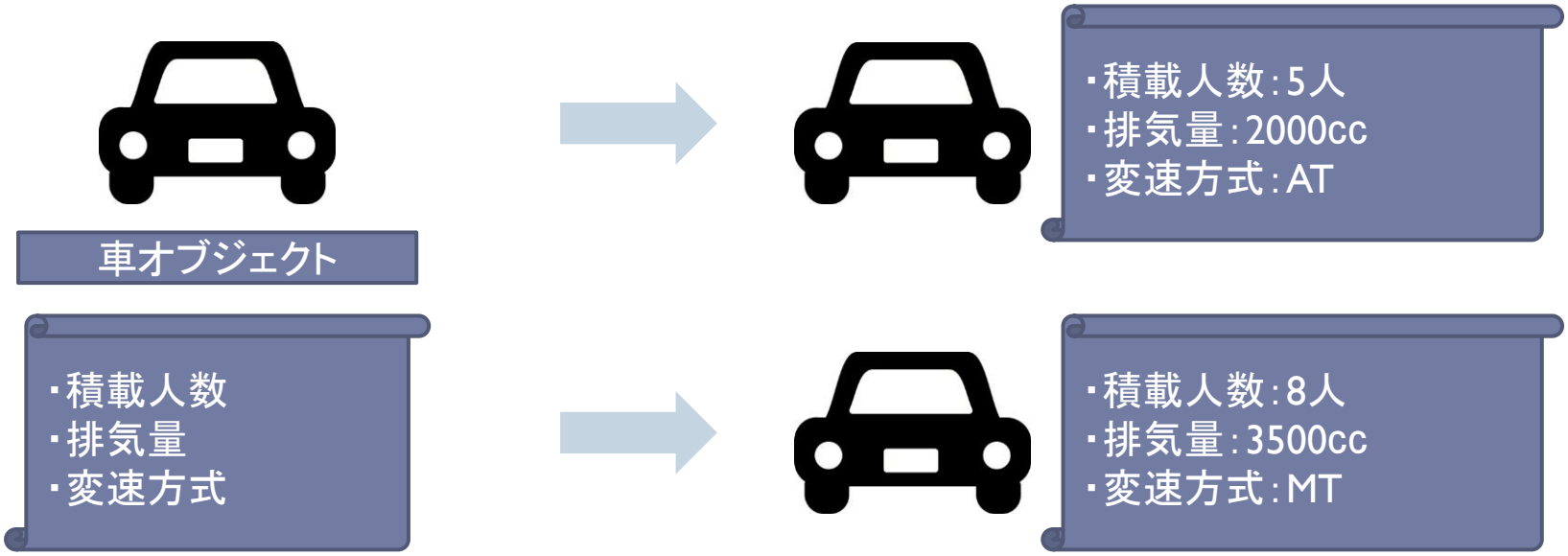


オブジェクト指向とは

- オブジェクト（Object）・・・あらゆる事物のこと

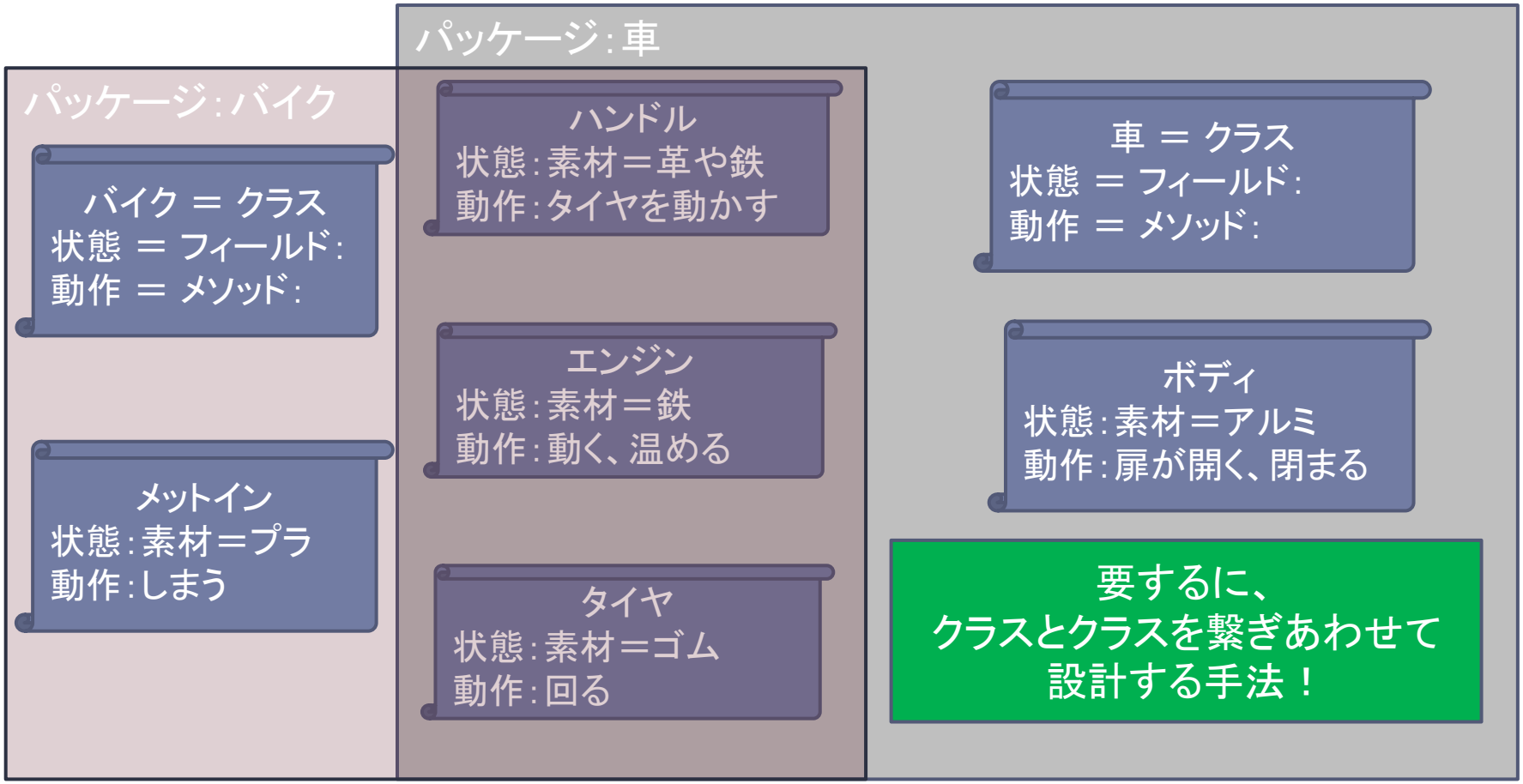
（概念的なものや、実体のあるものなどは問わず対象となる）

- オブジェクト指向・・・なるべくそのままの（概念や実体を活かした）構造で表現しようとするプログラミング手法



オブジェクト指向とは

- Javaコーディングにおけるオブジェクト＝クラス



インスタンス

- インスタンス

→他のクラスの機能を使うための変数

- インスタンスの生成方法

```
クラス名　インスタンス名　＝　new コンストラクタ指定;
```

- インスタンスの活用方法

```
インスタンス名.メソッド名（引数）;
```

```
インスタンス名.フィールド名;
```



コンストラクタ

- コンストラクタ

→インスタンス生成時に自動的に行われるメソッド

特徴として戻り値がない（基本は引数・処理文もなし）

```
public クラス名（引数）{
        文1;
}
```



インスタンス/コンストラクタ　例

例：

コンストラクタ

インスタンス

Person.java

```
1  package advance;
2
3  public class Person {
4      int age;
5
6      public Person() {
7          age = 20;
8      }
9
10     public Person(int a) {
11         age = a;
12     }
13
14 }
```

People.java

```
1  package advance;
2
3  public class People {
4
5      public static void main(String[] args) {
6
7          Person yamada = new Person();
8          System.out.println("山田の年齢は" + yamada.age + "歳です");
9
10         Person tanaka = new Person(25);
11         System.out.println("田中の年齢は" + tanaka.age + "歳です");
12
13     }
14
15 }
```

問題　コンソール　LogCat

<終了> People [Java アプリケーション] C:¥pleiades¥eclipse¥jre¥bin¥javaw.exe (2016/11/22 15:48:28)

山田の年齢は20歳です
田中の年齢は25歳です

カプセル化

- クラスが持つフィールドの内容を外部から保護する（勝手に変更・参照）ための仕組み

public
どのクラスからでもアクセス可能

private
自身のみがアクセス可能で、外部からはアクセス不可

getter/setter（ゲッター/セッター）
カプセル化されたフィールドへアクセスする場合に使う

```
private 型 フィールド名;
public 型 get+フィールド名(){
        return フィールド名;
}
public void set+フィールド名(型 変数) {
        this.フィールド名 = フィールド名;
}
```

Physical.java

```
1  package advance;
2
3  public class Physical {
4      private String name;
5      private int hight;
6      private int weight;
7
8      public String getName() {
9          return name;
10     }
11
12     public void setName(String name) {
13         this.name = name;
14     }
15
16     public int getHight() {
17         return hight;
18     }
19
20     public void setHight(int hight) {
21         this.hight = hight;
22     }
23
24     public int getWeight() {
25         return weight;
26     }
27
28     public void setWeight(int weight) {
29         this.weight = weight;
30     }
31 }
```

APIについて

- API（Application Programming Interface）とは

→頻繁に使われる機能を予めクラスとして提供されているもの

例：

Mathクラス・・・数学的な演算を行うメソッドや定数が定義されている

Systemクラス・・・コンソール画面への入力や出力を行うメソッドや定数が定義されている

Scannerクラス・・・入力されたデータを読み取るメソッドや定数が定義されている



APIについて

- System.outクラス（java.langパッケージ）
  - Print(Object)メソッド
    - コンソール画面に指定されたオブジェクトを表示
    - 表示後の改行は行わない
  - Println(Object)メソッド
    - コンソール画面に指定されたオブジェクトを表示
    - 表示後に改行を行う

- Scannerクラス（java.utilパッケージ）
  - Scanner(System.in).nextInt()メソッド
    - コンソール画面に入力された数値データを受け取るメソッド
  - Scanner(System.in).nextLine()メソッド
    - コンソール画面に入力された文字列データを受け取るメソッド



APIについて

- Mathクラス（java.langパッケージ）
  - Max(),min()メソッド
    - 2値のうちで大きい方（小さい方）の値を返す
  - Random()メソッド
    - 0以上1未満のdouble型の乱数を発生させる

その他のAPIについて

→ http://docs.oracle.com/javase/jp/7/api/



APIについて

- Dateクラス（java.utilパッケージ）
- Calendarクラス（java.utilパッケージ）
- GregorianCalendarクラス（java.utilパッケージ）

DateTest.java

```
1  package advance;
2
3  import java.util.Calendar;
6
7  public class DateTest {
8
9      public static void main(String[] args) {
10         Date d = new Date();
11         Calendar c = new GregorianCalendar();
12         c.setTime(d);
13         c.add(Calendar.DATE, 90);
14
15         System.out.print(c.get(Calendar.YEAR) + "年");
16         System.out.print((c.get(Calendar.MONTH) + 1 + "月"));
17         System.out.println(c.get(Calendar.DATE) + "日");
18     }
19
20 }
```

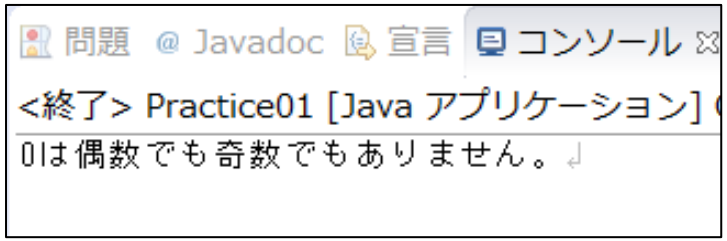
問題　コンソール　LogCat

```
<終了> DateTest (2) [Java アプリケーション] C:¥pleiades¥eclipse¥jre¥bin¥javaw.exe (2016/11/22 16:22:37)
2017年2月20日
```

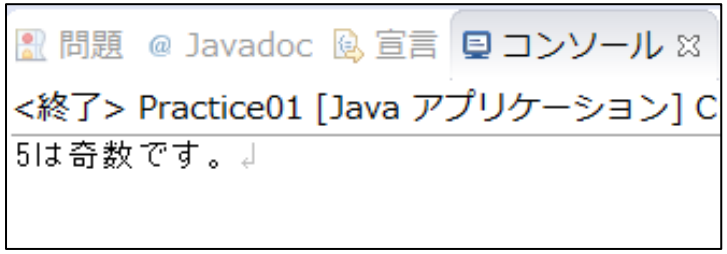
練習 7

- if文を使って偶数と奇数、0を判別するプログラムを作成してください。

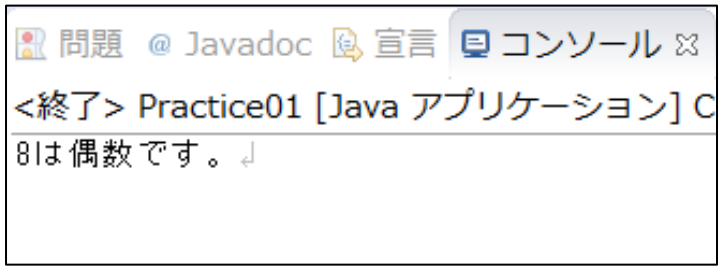
0の場合



5の場合



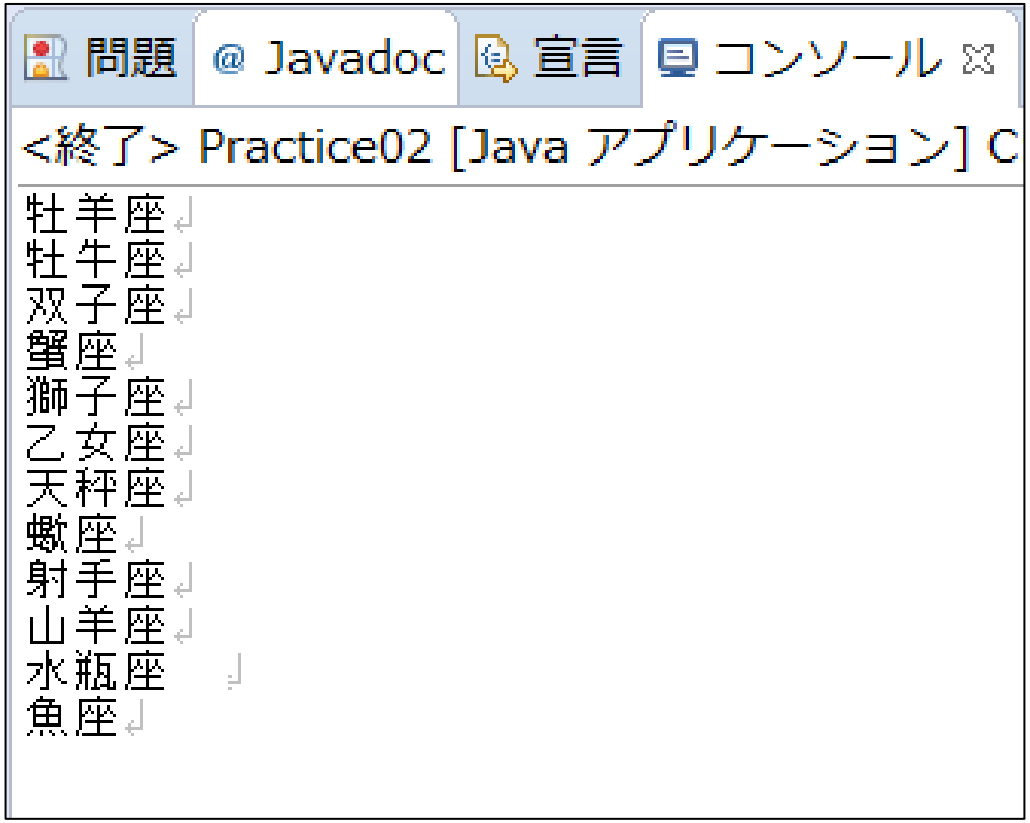
8の場合





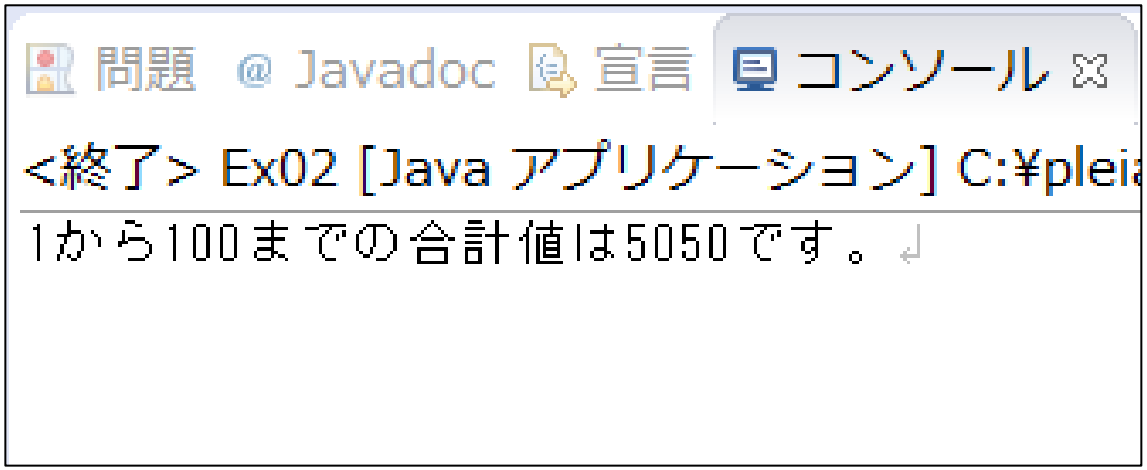
練習 8

- for文を使って12星座を出力するプログラムを作成してください。



練習 9

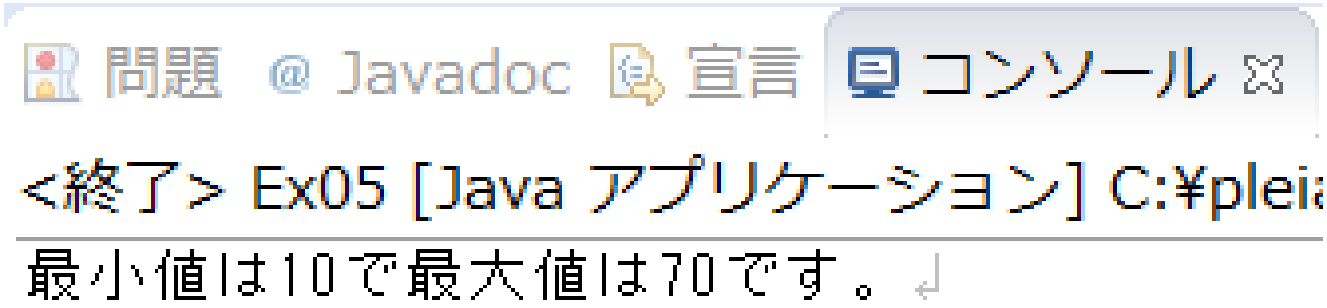
・1から100まで足した合計を出力するプログラムを作成してください。



練習１０

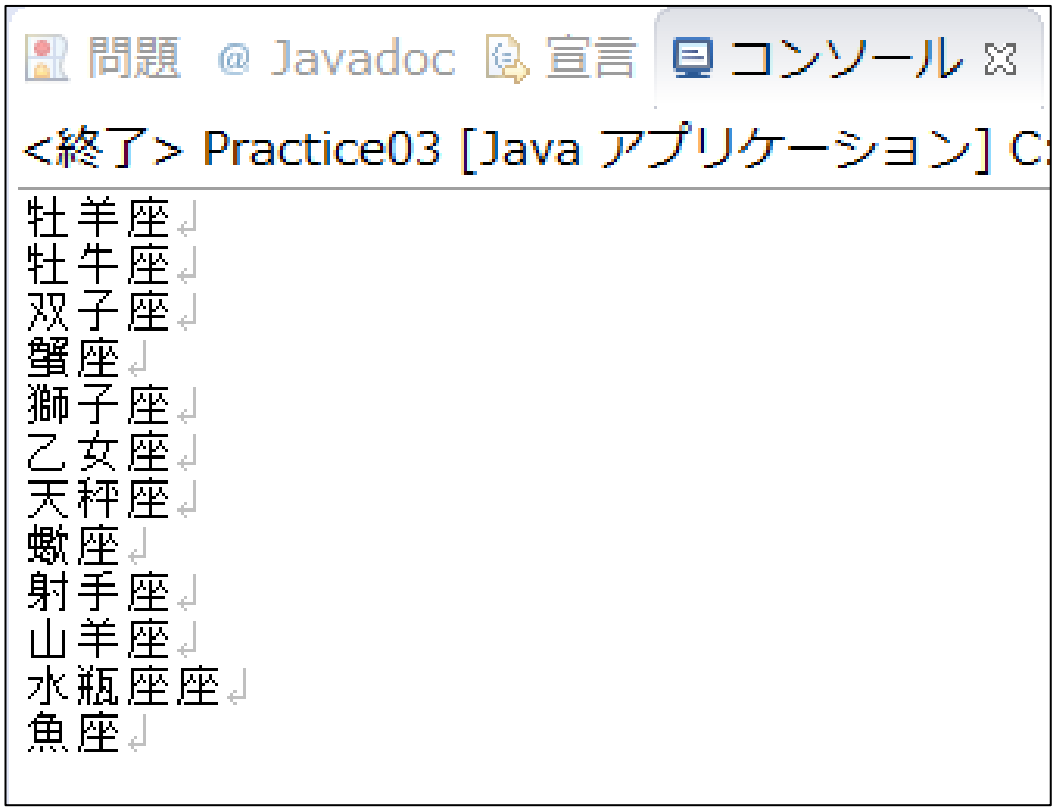
・配列に{20,30,70,10,50,40}を設定し、
この中から最小値と最大値を出力する
プログラムを作成してください。

解答例



練習１１

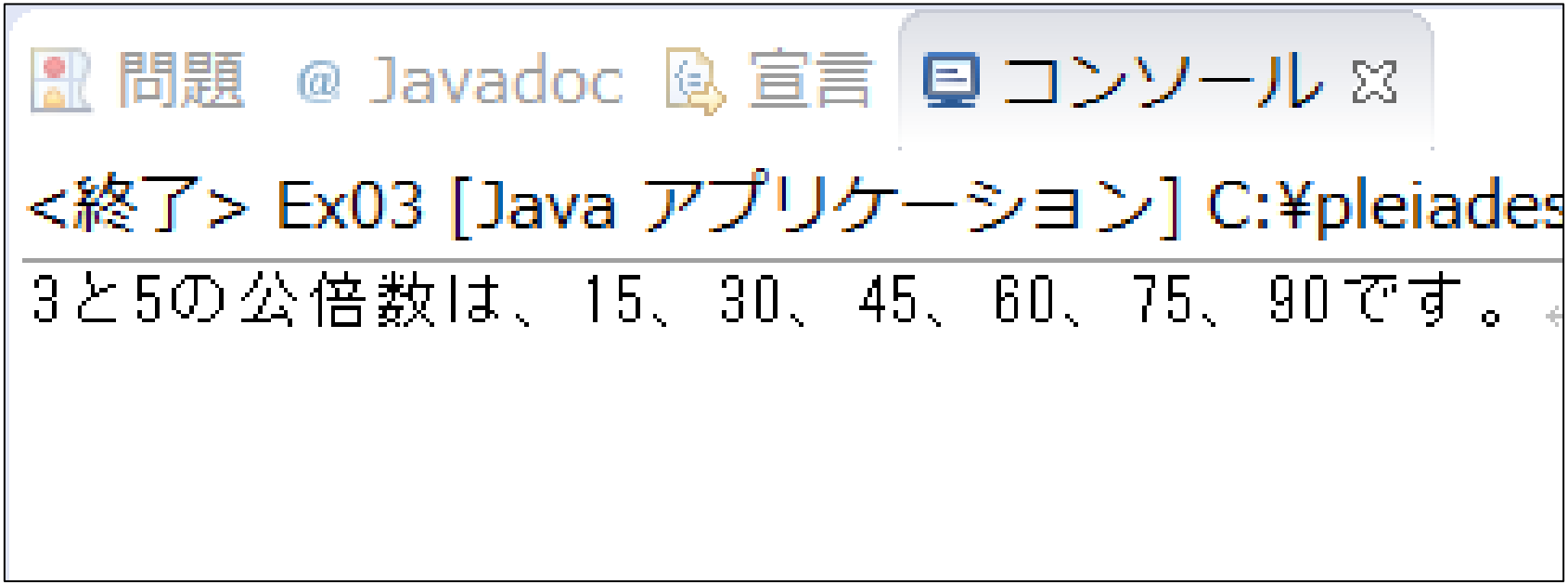
- while文を使って12星座を出力するプログラムを作成してください。



練習１２

・変数a,bを作成し、aとbの公倍数を6個表示するプログラムを作成してください。

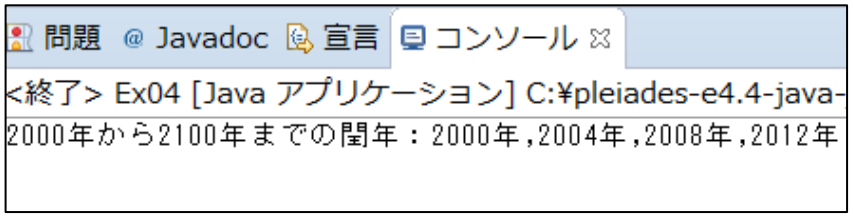
※例ではa,bは3,5として実行



練習１３

- 2000年から2100年までの閏(うるう)年を出力する
  プログラムを作成してください。
  - 閏年は4で割り切れる年。
    ただし400で割り切れない100の倍数年は平年(閏年じゃない)
    (例えば1900年、2100年は平年で、2500年は「閏年」です)

解答例



2000年から2100年までの閏年

2000年,2004年,2008年,2012年,2016年
2020年,2024年,2028年,2032年,2036年
2040年,2044年,2048年,2052年,2056年
2060年,2064年,2068年,2072年,2076年
2080年,2084年,2088年,2092年,2096年



練習７（解答例）

・if文を使って偶数と奇数を判別するプログラムを作成してください。

Practice01.java

```
1  package enshu;
2
3  public class Practice01 {
4
5      public static void main(String[] args) {
6
7          int kazu = 0; // ★ここを変える
8
9          // 0じゃない、かつ偶数でも無ければ奇数とする
10         if (kazu == 0) {
11             System.out.println(kazu + "は偶数でも奇数でもありません。");
12         } else if ((kazu % 2) == 0) {
13             System.out.println(kazu + "は偶数です。");
14         } else {
15             System.out.println(kazu + "は奇数です。");
16         }
17     }
18 }
```

問題　進行状況　コンソール　デバッグ　サーバー　JUnit

<終了> Practice01 [Java アプリケーション] C:¥pleiades¥java¥8¥bin¥javaw.exe (2023/07/24 16:46:54)

0は偶数でも奇数でもありません。

if（0かどうか）
else if（偶数かどうか）
else（それ以外）



練習８（解答例）

- for文を使って12星座を出力するプログラムを作成してください。

Practice01.java　*Practice02.java

```
1  package enshu;
2
3  public class Practice02 {
4
5      public static void main(String[] args) {
6          // 12星座を配列にする
7          String[] seizaList = { "牡羊座", "双子座", "獅子座", "天秤座", "射手座",
8                  "水瓶座", "牡牛座", "蟹座", "乙女座", "蠍座", "山羊座", "魚座" };
9          // for文で一つずつ配列から文字列を取り出す
10         for (String seiza : seizaList) {
11             System.out.println(seiza);
12         }
13     }
14 }
```

問題　進行状況　コンソール　デバッグ　サーバー　JUnit

<終了> Practice02 [Java アプリケーション] C:¥pleiades¥java¥8¥bin¥javaw.exe (2023/07/24 16:57:32)

```
牡羊座
双子座
獅子座
天秤座
射手座
水瓶座
牡牛座
蟹座
乙女座
蠍座
山羊座
魚座
```

配列じゃなくてArrayListでもOK

普通のfor文で12回回す、でもOK

練習９（解答例）

・1から100まで足した合計を出力するプログラムを作成してください。

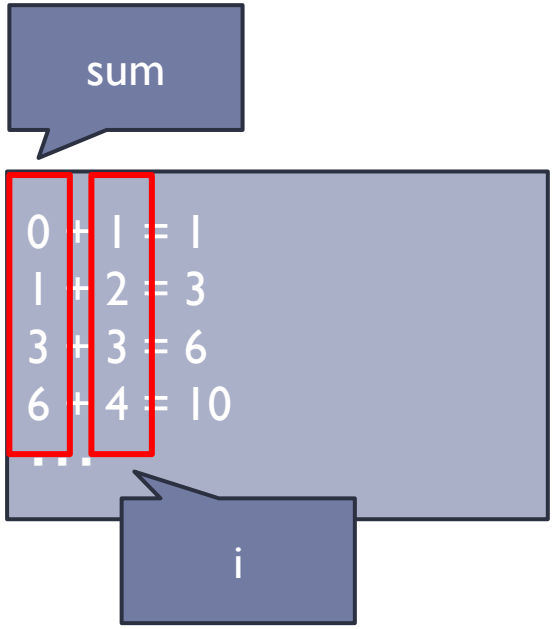
Practice01.java　Practice02.java　Ex01.java　Ex02.java

```
1  package enshu;
2
3  public class Ex02 {
4
5      public static void main(String[] args) {
6          // 数値を足していく用の変数
7          int sum = 0;
8          // 1～100まで100回ループさせてどんどん足していく
9          for (int i = 1; i <= 100; i++) {
10             sum = sum + i;
11         }
12         System.out.println("1から100までの合計値は：" + sum);
13     }
14 }
15
```

問題　進行状況　コンソール　デバッグ　サーバー　JUnit

<終了> Ex02 [Java アプリケーション] C:¥pleiades¥java¥8¥bin¥javaw.exe (2023/07/24 17:1

1から100までの合計値は：5050



練習１０（解答例）

・配列に{20,30,70,10,50,40}を設定し、この中から最小値と最大値を出力するプログラムを作成してください。

Practice01.java　Practice02.java　Ex01.java　Ex02.java　Ex03.java

```
1  package enshu;
2
3  public class Ex03 {
4
5      public static void main(String[] args) {
6          // TODO 自動生成されたメソッド・スタブ
7          int[] hairetu = { 20, 30, 70, 10, 50, 40 };
8          // 最大値を入れる変数
9          int maxSuti = 0;
10         // 最小値を入れる変数
11         int miniSuti = 100;
12
13         // 配列の数だけ数値を取り出してループ処理を行う
14         for (int suti : hairetu) {
15             // 最大値より取り出した数値が大きければ置き換える
16             if (maxSuti < suti) {
17                 maxSuti = suti;
18             }
19             // 最小値より取り出した数値が小さければ置き換える
20             if (miniSuti > suti) {
21                 miniSuti = suti;
22             }
23         }
24         System.out.println("最大値は：" + maxSuti);
25         System.out.println("最小値は：" + miniSuti);
26     }
27 }
```

問題　進行状況　コンソール　デバッグ　サーバー　JUnit

<終了> Ex03 [Java アプリケーション] C:¥pleiades¥java¥8¥bin¥javaw.exe (2023/07/24 17:31:3

```
最大値は：70
最小値は：10
```

以下の処理を配列文実施する
For文の中にif文を入れて処理

if（暫定最大値 < 取り出した数値）
　暫定最大値を置き換える
if（暫定最小値 > 取り出した数値）
　暫定最小値を置き換える

練習１１（解答例）

・while文を使って12星座を出力するプログラムを作成してください。

```
public class Pra11Ans {

    /*
     * while文を使って12星座を出力するプログラムを作成してください。
     */
    public static void main(String[] args) {
        // 12星座を配列にする
        String[] seizaList = { "牡羊座", "双子座", "獅子座", "天秤座", "射手座",
                "水瓶座", "牡牛座", "蟹座", "乙女座", "蠍座", "山羊座", "魚座" };

        // while文の実行をカウント + 順番に取り出す用の変数
        int i = 0;

        // while文で一つずつ配列から文字列を取り出す
        while (i < seizaList.length) {
            System.out.println(seizaList[i]);
            // iをカウントしていく
            i = i + 1; // i++でもOK
        }
    }
}
```

配列で星座を定義して、配列の数だけwhile分でループ処理を行う。
カウント用の変数でループ回数を制御する。



練習１２（解答例）

・変数a,bを作成し、aとbの公倍数を6個表示するプログラムを作成してください。

```
public static void main(String[] args) {
    // ★変数a,bを作成する
    int a = 8;
    int b = 9;
    // 公倍数のカウント用変数
    int c = 0;
    // カウントアップ用変数（0は公倍数とはしないので1から）
    int d = 1;
    System.out.print(a + "と" + b + "の公倍数は");
    // 公倍数が6つになるまで出力
    while (c < 6) { // ★while文で公倍数が6つになるまで実行
        // ★公倍数を判定する処理
        if (d % a == 0 && d % b == 0) {
            // 両方で割り切れるため公倍数として出力
            System.out.print(d);
            // 公倍数のカウントを一つ増やす
            c = c + 1; // c++でもOK

            // 6つ目の数字の後には「,」をつけないようにする
            if (c < 6) {
                System.out.print(",");
            }
        }
        // カウントアップする
        d = d + 1; // d++でもOK
    }
    System.out.println("です。");
}
```

変数cを作成し、カウントして公倍数が6回出力されるまで繰り返すwhile処理。

その中に変数a,bの両方で割り切れる数（公倍数）を出力する処理を入れる。



練習１３（解答例）

・2000年から2100年までの閏（うるう）年を出力するプログラムを作成してください。

```
public static void main(String[] args) {
    // ★範囲を指定する変数を作成する
    int start = 2000;
    int end = 2100;

    System.out.print(start + "年から" + end + "年の閏年：");

    // ★範囲の分、処理を行う
    for (int i = start; i <= end; i++) {

        if (i % 400 == 0) { // 400で割り切れたら閏年
            System.out.print(i + "年、");
        } else if (i % 4 == 0 && i % 100 != 0) { // 4で割り切れ

        }

    }

}
```

400で割り切れるってことは4でも割り切れるため、まずは400で割り切れるものを閏年と判定。
その後、4で割り切れて、かつ100で割り切れないものを閏年と判定する処理。

※最後に「、」がついちゃうのが嫌な人は一回全部文字列に格納して最後にまとめて出力してもいいです

追加練習3

・while文を使用して、6の目がでるまでサイコロをふり続けるプログラムを作成してください。
・サイコロを振るたびに出目を画面に表示してください。
・6の目が出たら「6が出たので終了します」と表示してください。
※1～6までのランダムなint型の数値（出目）を取得する方法 ：

1 + (int)(Math.random() * 6.0)

```
5
6
「6が出たので終了します」
```

```
1
2
5
4
5
5
5
4
3
1
4
6
「6が出たので終了します」
```



追加練習4

・試験と補習の結果で評価を表示してください。
・キーボードからテストの点数を受け付け、int型に変換し int型変数に代入します。
・60点未満の場合には、キーボードから補習受講の有無を ”yes”/”no”で受付け、String型変数に代入します。
・評価表を見て結果を以下のように表示してください。

```
点数を入力してください > 40
補習を受けましたか？（yes/no） > yes
結果：C評価
```

点数	補習	評価
80点以上		A評価
60−79点		B評価
60点未満	"yes"	C評価
	″no″	D評価



追加練習3（解答例）

・while文を使用して、6の目がでるまでサイコロをふり続けるプログラムを作成してください。

```
class Add3 {
    public static void main (String[] args) {

        int dice = 0 ;          //サイコロの目(初期値:0)

        //サイコロを振る(1～6までのランダムな値を格納)
        dice =  1 + (int)( Math.random() * 6.0 ) ;

        //サイコロの目を表示
        System.out.println(dice) ;

        //---------------------------------
        //6の目がでるまでサイコロをふり続ける
        //---------------------------------
        while( dice != 6 ){

            //サイコロを振る(1～6までのランダムな値を格納)
            dice =  1 + (int)(Math.random() * 6.0) ;

            //サイコロの目を表示
            System.out.println(dice) ;

        }

        //終了メッセージを表示
        System.out.println("「6が出たので終了します」") ;

    }
}
```

まずサイコロを振ってその出目を表示する
その後while文で6以外であれば出目を表示する
6が出れば終了

追加練習4（解答例）

・試験と補習の結果で評価を表示してください。

```
public static void main(String[] args) {
    // 変数宣言
    String result;      // 評価結果
    int score;          // 点数
    String addStudy = null; // 補習を受けた ("yes"/"no")

    // 点数入力
    System.out.print("点数を入力してください > ");
    String str = new java.util.Scanner(System.in).nextLine();
    score = Integer.parseInt(str);

    // 60点未満の時に補習の有無を聞く
    if (score < 60) {
        System.out.print("補習を受けましたか？ (yes/no) > ");
        addStudy = new java.util.Scanner(System.in).nextLine();
    }

    // 判定
    if (80 <= score) {
        result = "A評価";
    } else if (60 <= score) {
        result = "B評価";
    } else if (addStudy.equals("yes")) {
        result = "C評価";
    } else {
        result = "D評価";
    }

    // 結果の表示
    System.out.println("結果:" + result);
}
```

If(80点以上の場合)　A
else if(60点以上の場合)　B
else if(補習がyes の場合)　C
それ以外　D

演習

- mainメソッドを持つクラス「Uranai」とmainメソッドを持たないクラス「Hantei」を作成します。
- Uranaiクラスではint型の変数「num」を定義します。
- Hanteiクラス内ではString型の変数「unsei」をフィールドに定義します。
- Hanteiクラス内では引数として与えられた数を3で割った余りによって大吉or中吉or小吉のいずれかを生成し、それをunseiに代入するコンストラクタを作成します。
- Uranaiクラス内でHanteiクラスのインスタンスを生成し、numを与えることでunseiを受け取り、それを占い結果としてコンソールに表示する式を作成します。



演習 テスト編

- ソースコードは出来上がったらテストします
  単体テストとかユニットテスト、と言われます。

- 以下、ChatGPT先生からの説明です

- 単体テスト（ユニットテスト）は、プログラミングにおいてソフトウェアの最小単位（通常は関数やメソッド）が正しく動作するかを確認するテストのことです。このテストは、ソフトウェア開発の初期段階で行われ、各機能が想定通りに動作するかを個別に検証します。

- 単体テストの主な特徴は以下の通りです：

- - **独立性**：各テストは他の部分に依存せずに実行されるべきです。
- - **自動化**：テストは自動化され、繰り返し実行可能であるべきです。
- - **小規模**：小さな機能単位でテストを行い、特定の関数やメソッドの動作を検証します。
- - **速度**：テストは迅速に実行され、即座にフィードバックを提供することが望ましい。

- 単体テストは、バグの早期発見、コードのリファクタリング時のリスク軽減、ソフトウェアの品質向上に貢献します。また、テストケースの作成と実行は、開発プロセスの一部として重要な役割を果たします。



演習 テスト編

```
public int warizan(int a, int b) {
    int kekka = a / b;
    return kekka;
}
```

```
@Test
public void testWarizan1() {
    BBB bbb = new BBB();
    assertEquals(0, bbb.warizan(0, 1));
}

@Test
public void testWarizan2() {
    BBB bbb = new BBB();
    assertEquals(0, bbb.warizan(1, 0));
}
```

一つ目は
0÷1が0になる事を確認

2つ目は
1÷0が0になる事を確認

0除算としてエラーになる
iPhoneの電卓などでも
「エラー」になります

assertEquals(A, B);
A:期待される値
B:テスト対象から得られる値
AとBを比較して等しければ成功、異なる場合は失敗



演習（解答例①）

- mainメソッドを持つクラス「Uranai」とmainメソッドを持たないクラス「Hantei」を作成します。
- Uranaiクラスではint型の変数「num」を定義します。
- Uranaiクラス内でHanteiクラスのインスタンスを生成し、numを与えることでunseiを受け取り、それを占い結果としてコンソールに表示する式を作成します。

Uranai.java　Hantei.java

```
1  package enshu;
2
3  public class Uranai {
4
5      public static void main(String[] args) {
6          // Uranaiクラスではint型の変数「num」を定義します。
7          int num = 0;
8          // Uranaiクラス内でHanteiクラスのインスタンスを生成し、
9          // numを与えることでunseiを受け取り、
10         // それを占い結果としてコンソールに表示する式を作成します。
11         Hantei hantei = new Hantei(num);
12         System.out.println(hantei.unsei);
13     }
14 }
```



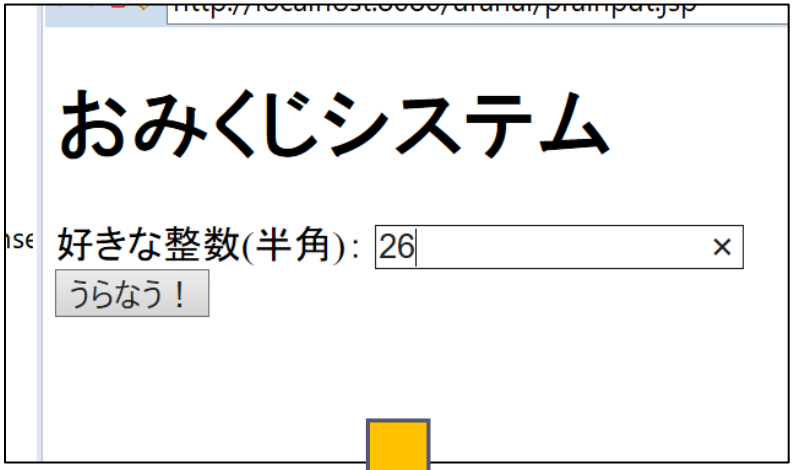
演習（解答例②）

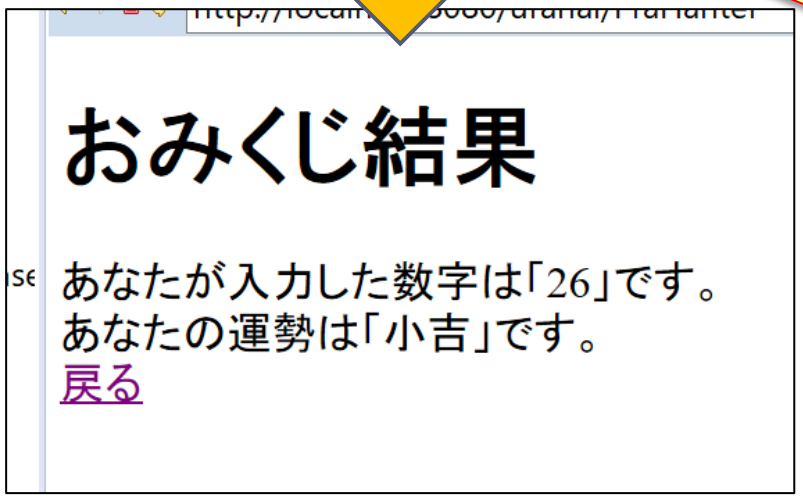
- mainメソッドを持つクラス「Uranai」とmainメソッドを持たないクラス「Hantei」を作成します。
- Hanteiクラス内ではString型の変数「unsei」をフィールドに定義します。
- Hanteiクラス内では引数として与えられた数を3で割った余りによって大吉or中吉or小吉のいずれかを生成し、それをunseiに代入するコンストラクタを作成します。

Uranai.java　Hantei.java

```
1  package enshu;
2
3  public class Hantei {
4      // Hanteiクラス内ではString型の変数「unsei」をフィールドに定義します。
5      String unsei = "";
6
7      // Hanteiクラス内では引数として与えられた数を3で割った
8      // 余りによって大吉or中吉or小吉のいずれかを生成し、
9      // それをunseiに代入するコンストラクタを作成します。
10     Hantei(int kuji) {
11         // 余りによって運勢を判断する
12         if (kuji % 3 == 0) {
13             unsei = "大吉";
14         } else if (kuji % 3 == 1) {
15             unsei = "中吉";
16         } else if (kuji % 3 == 2) {
17             unsei = "小吉";
18         }
19     }
20 }
21
```

作ったものをweb化する

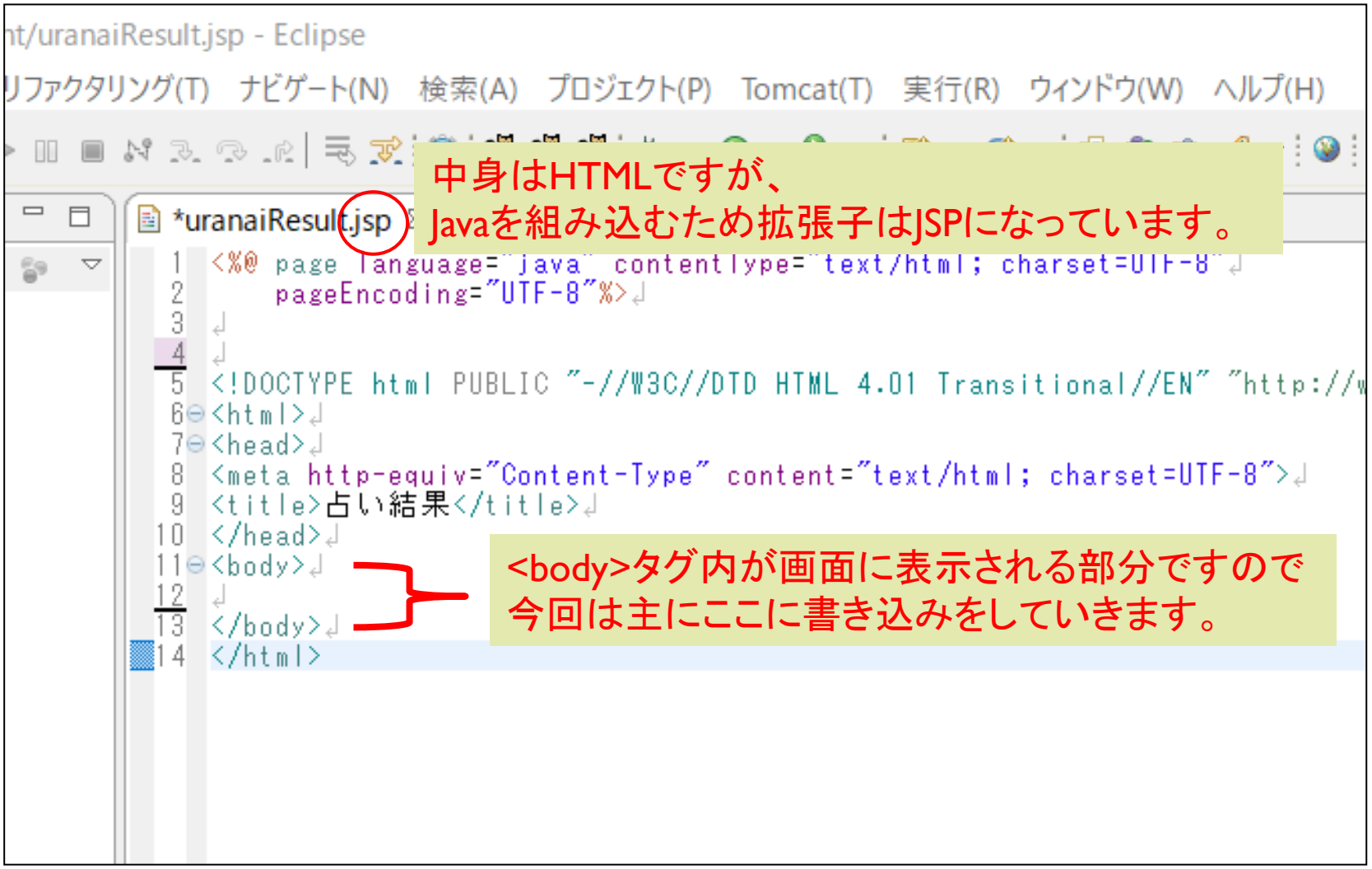




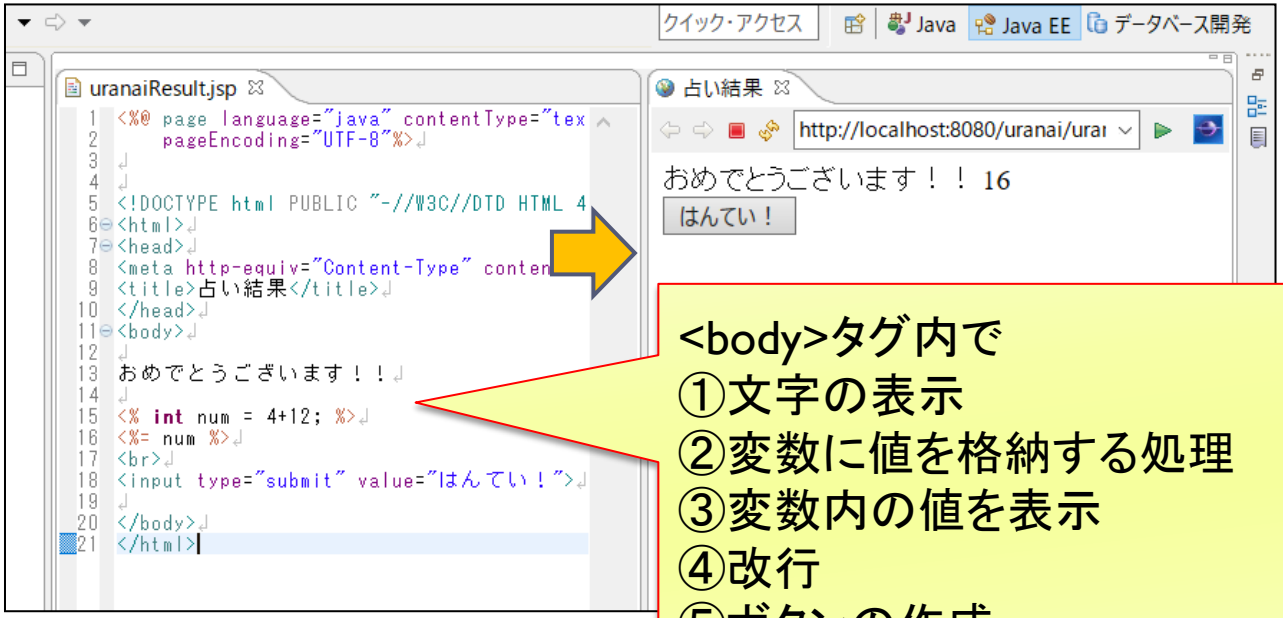
ここまでは演習の中で作ります。

この先機能をどう増やしていくか、今回演習する内容を使ってなにができるかを考えながら取り組んでみましょう。

HTMLについて

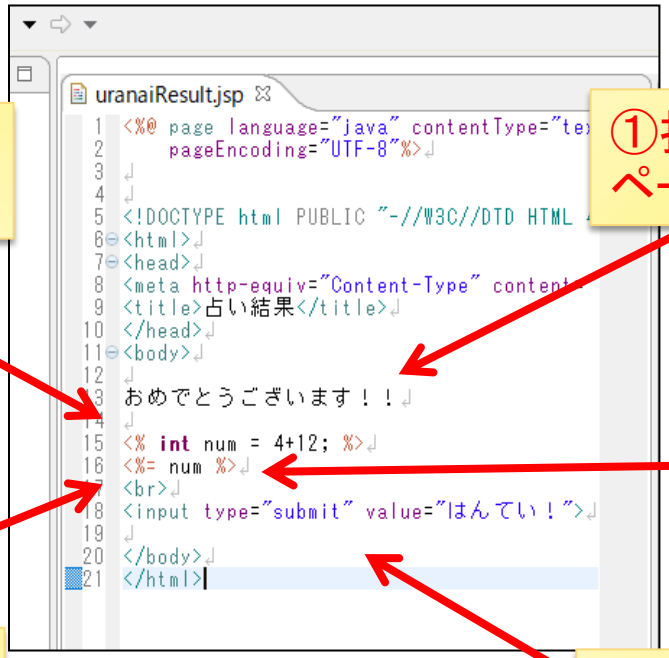

ht/uranaiResult.jsp - Eclipse
リファクタリング(T) ナビゲート(N) 検索(A) プロジェクト(P) Tomcat(T) 実行(R) ウィンドウ(W) ヘルプ(H)
*uranaiResult.jsp
中身はHTMLですが、
Javaを組み込むため拡張子はJSPになっています。
<%@ page language="java" contentType="text/html; charset=UTF-8"
pageEncoding="UTF-8"%>
<!DOCTYPE html PUBLIC "-//W3C//DTD HTML 4.01 Transitional//EN" "http://w
<html>
<head>
<meta http-equiv="Content-Type" content="text/html; charset=UTF-8">
<title>占い結果</title>
</head>
<body>
</body>
</html>
<body>タグ内が画面に表示される部分ですので
今回は主にここに書き込みをしていきます。

JSPについて



どこが①～⑤に対応しているかわかりますか？

答え合わせ



②<% %>で囲った部分の処理が実行されます。

①打った文字がそのままページに表示されます。

③<%= %>で囲った部分の処理内容が表示されます。

④
で改行です。通常の改行では反映されません。

⑤ボタンの作成とボタンの文字を指定します。

JSPコード例

```
<title>JSP見本</title>
</head>
<body>

簡易税率計算ページ
<br><br>

<!-- 食品or酒類を選ぶ -->
<% String category = "酒類"; %>

<!-- 値段を自由に決める -->
<% int price = 1200; %>

<!-- 分類と値段の画面表示 -->
分類…<%= category %>
<br>
値段…<%= price%>円
<br>

<!-- 分類で見分けて税率計算 -->
<%
    double ans = price;
    if(category.equals("食品"){
        ans = price*1.08;
    }else if(category.equals("酒類"){
        ans = price*1.1;
    }
%>

<!-- 結果の画面表示 -->
<br>
税込みで
<%= ans %>
円となります。

</body>
</html>
```

一例です

＜酒類のとき＞

簡易税率計算ページ

分類…酒類
値段…1200円

税込みで 1320.0 円となります。

＜食品のとき＞

簡易税率計算ページ

分類…食品
値段…1200円

税込みで 1296.0 円となります。

プログラミング演習　JSP

- JSPの練習をします。

★練習で使う知識★

①<% %>で囲むとJavaのコードを実行できる

②<%= %>で囲むと変数の表示が可能

③for文やif文も使用できる（①の応用）

④改行は

⑤ブラウザ表示は「実行」ボタンから

→実際のコード例は次のページで



JSP練習問題

☆画面表示を意識してコーディングしてみましょう☆

(webapp配下のPraです)

【練習１】 5,12を変数a,bに格納し、四則演算結果を表示する

【練習２】 1～20の数字をfor文を使って表示する

【練習３】 練習２に関して

その数が3で割り切れるなら“Fizz“

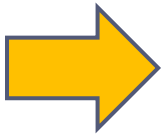
5で割り切れるなら“Buzz“

両方で割り切れるなら“FizzBuzz"と表示する



答え合わせ　練習１

```
 7 <title>練習1</title>
 8 </head>
 9 <body>
10
11 <!-- a,bの画面表示 -->
12 a=5
13 b=12
14 <br>
15
16 <!-- 任意の数a,bの定義 -->
17 <%
18  int a=5;
19  int b=12;
20 %>
21
22 <!-- 画面表示と計算 -->
23 足し算 : <%= a+b %>
24 <br>
25 引き算 : <%= a-b %>
26 <br>
27 掛け算 : <%= a*b %>
28 <br>
29 割り算 : <%= a/b %>
30
31 </body>
32 </html>
```



a=5 b=12
足し算:17
引き算:-7
掛け算:60
割り算:0

改行の場所と
見間違いに注意しましょう！

答え合わせ　練習2

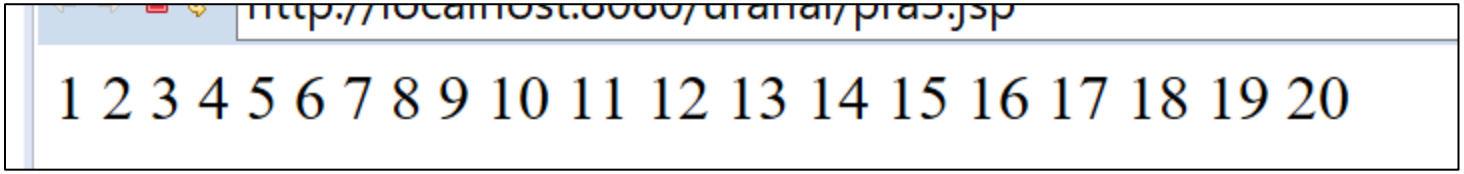
```
 7 <title>練習3</title>
 8 </head>
 9 <body>
10
11 <!-- for文の処理 -->
12 <%
13     for(int i=1;i<21;i++){
14 %>
15
16 <!-- 変数iの表示 -->
17     <%= i %>
18
19 <!-- for文の閉じ括弧を認識させる -->
20 <%
21     }
22 %>
23
24 </body>
25 </html>
```

＜ポイント＞

①for文の中に表示の処理があるため、
その部分だけ<%= %>で囲む必要がある

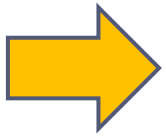
②残る閉じ括弧もJavaとして認識させないと
for文が成立しない

③
を入れて改行し縦に表示しても良い



答え合わせ　練習3

```
 7 <title>練習4</title>
 8 </head>
 9 <body>
10
11 <%for(int i=1;i<21;i++){%>
12     <%=i %>
13
14     <%if(i%15==0){%>
15         :FizzBuzz
16     <%}else if(i%5==0){ %>
17         :Buzz
18     <%}else if(i%3==0){ %>
19         :Fizz
20     <%} %>
21
22     <br>
23 <%} %>
24
25 </body>
26 </html>
```



```
1
2
3 :Fizz
4
5 :Buzz
6 :Fizz
7
8
9 :Fizz
10 :Buzz
11
12 :Fizz
13
14
15 :FizzBuzz
16
17
18 :Fizz
19
20 :Buzz
```

＜ポイント＞
「for文」「if文」
「<% %>」「<%=%>」
「直書き」
の五要素が絡み合っているため囲む範囲などに注意する



JSP追加練習問題

☆画面表示を意識してコーディングしてみましょう☆

【追加練習5】 boolean型を使用し、1000以上 かつ 2000以下の場合true、それ以外をfalseを表示する



答え合わせ　追加練習5

```
<title>追加練習5</title>
</head>
<body>
    <!-- 変数宣言 -->
    <%
        int price = 1500;
        boolean result;
    %>
    <!-- 改行コード -->
    <br>
    <!-- result 設定 -->
    <%
        result = (1000 <= price) && (price <= 2000);
    %>
    <!-- 結果表示 -->
    <%
        out.println("結果:" + result);
    %>
</body>
</html>
```

＜ポイント＞
画面表示の仕方注意
out.println



サーブレット 練習

- ◎動作の流れ
- ①数値を入力する（UranaiInput.jsp） ※作成済
- ②数値を3で割り、余りの数によって大吉/中吉/小吉を決める（HanteiServlet.java）
- ③数値と運勢を表示する（UranaiResult.jsp） ※作成済

- 今回練習するのは、②の部分です。

- ◎作る機能
- ・入力された数値を受け取る
- ・数値を3で割った余りの値を変数に格納する
- ・if文を使い、変数によって運勢を決める
- ・運勢の値を結果表示画面に送る



①UranaiInput.jspの解説

・jspページ上で入力した値はaction属性で指定したパスにparameterとして送られる

（<form action="パス指定" method="post">）

・複数のパラメータが送られる場合もあるのでパラメータ名をname属性に付与し区別する

（<input type="text" name="パラメータ名">）

```
 8 <title>おみくじシステム</title>
 9 </head>
10 <body>
11 <h1>おみくじシステム</h1>
12
13 <%-- -------- 入力フォーム -------- --%>
14 <form action="PraHantei" method="post">
15     好きな整数(半角):
16     <input type="text" name="num">
17     <br>
18     <input type="submit" value="うらなう！">
19 </form>
20
21 </body>
22 </html>
```

UranaiInput.jsp



②HanteiServlet.javaの解説

・servletでは暗黙オブジェクト「request」に対しgetParameterメソッドを実行し、パラメータ名を指定し戻り値を受け取る

（request.getParameter(“パラメータ名”)） ※String型

例：String year = request.getParameter(“age”);

・jspに送りたい値を格納するため、暗黙オブジェクト「request」に対しsetAttributeメソッドを実行する

（request.setAttribute(“オブジェクト名”, 格納する値);）

例： request.setAttribute(“result”, unsei);



③UranaiResult.jspの解説

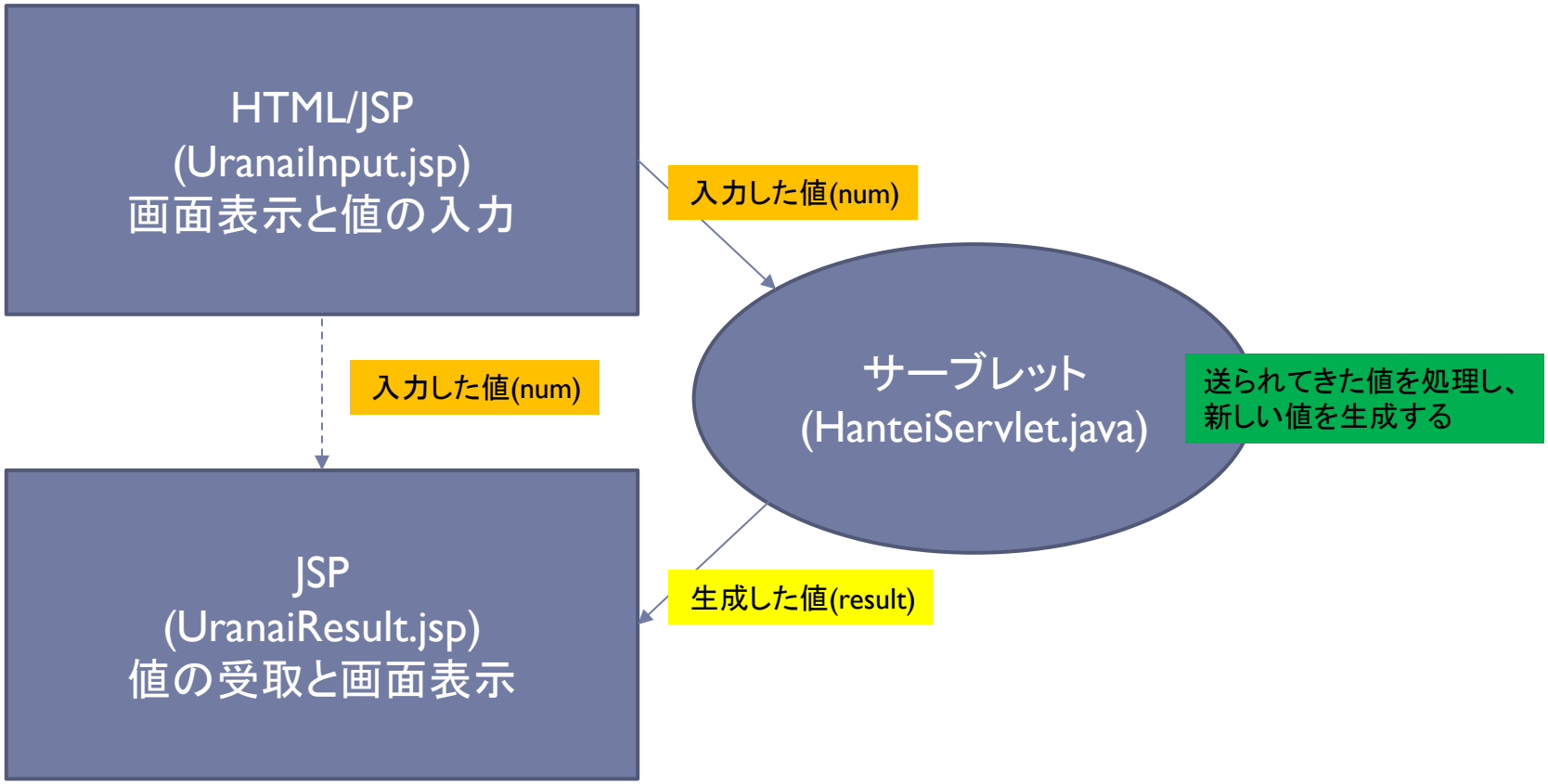
- UranaiInputで格納したパラメータを取り出す
  （request.getParameter(“パラメータ名”)） ※String型

- HanteiServletで格納したパラメータを取り出す
  （ request.getAttribute(“オブジェクト名”) ）※オブジェクト型

UranaiResult.jsp

```
 9 <title>おみくじ結果</title>
10 </head>
11 <body>
12 <h1>おみくじ結果</h1>
13
14 <%-- -------- リクエストから各値を取得 -------- --%>
15 <%
16     String result = (String) request.getAttribute("result");
17     String num = request.getParameter("num");
18 %>
19 <%-- -------- 取得した各値を使って、占い結果の表示 -------- --%>
20 あなたが入力した数字は「<%=num %>」です。
21 <br>
22 あなたの運勢は「<%=result %>」です。
23 <br>
24
25 <a href="<%=request.getContextPath()%>/praInput.jsp">戻る</a>
26 </body>
27 </html>
```

サーブレット　図解


HTML/JSP
(UranaiInput.jsp)
画面表示と値の入力
入力した値(num)
サーブレット
(HanteiServlet.java)
送られてきた値を処理し、
新しい値を生成する
入力した値(num)
JSP
(UranaiResult.jsp)
値の受取と画面表示
生成した値(result)

値のやり取り

- formから送られた値の受取り

  ★request.getParameter(“パラメータ名”)　※String型

  例：String year = request.getParameter(“age”);

- 明示的に値を送るとき

  ★request.setAttribute(“パラメータ名”, 格納する値);

  例：request.setAttribute(“result”, unsei);

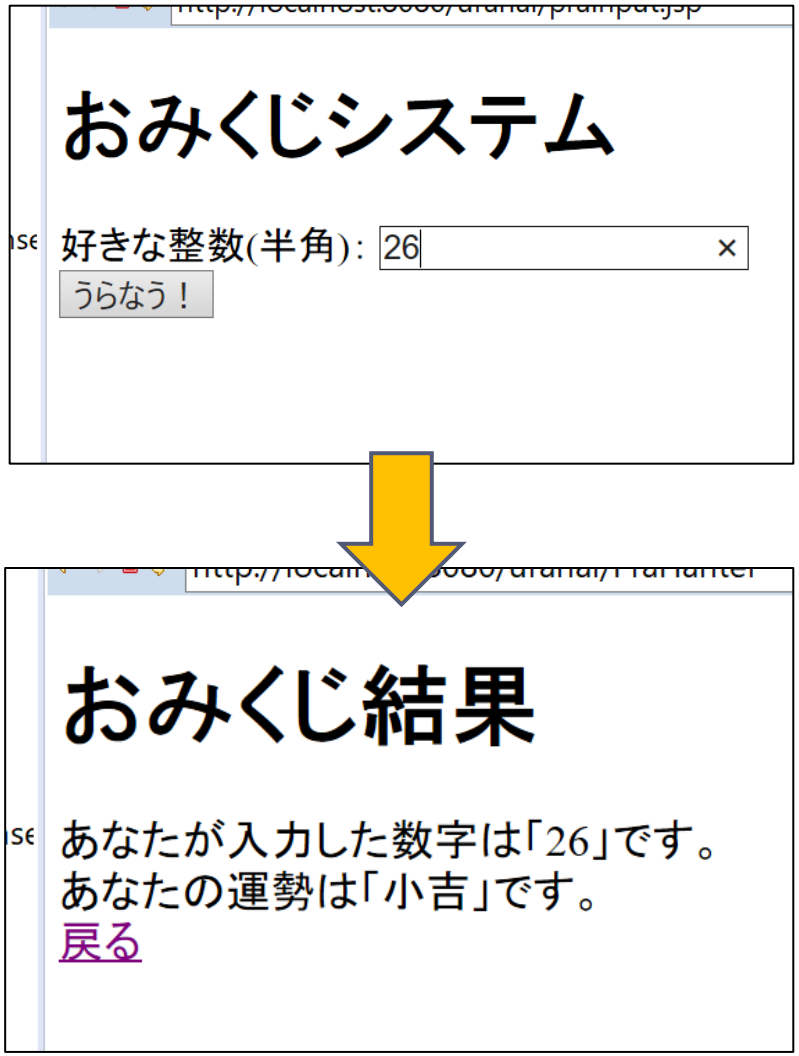
- setAttributeで送られた値の受取り　※オブジェクト型

  ★request.getAttribute(“パラメータ名”)

  例：String unsei = (String)request.getAttribute(“result”) ;



答え合わせ　サーブレット練習　画面



答え合わせ　サーブレット練習　コード例

```
19    protected void doPost(HttpServletRequest request, HttpServletResponse response) thr
20
21        // -------- 文字コード設定 --------
22        request.setCharacterEncoding("UTF-8");
23
24        // -------- 占い結果生成  --------
25        String kekka = "仮";
26        int number = Integer.parseInt(request.getParameter("num"));
27        number = number%3;
28
29        if (number == 0) {
30            kekka = "大吉";
31        } else if (number == 1) {
32            kekka = "中吉";
33        } else if (number == 2) {
34            kekka = "小吉";
35        }
36
37        // -------- リクエストに占い結果を追加 (setAttribute) --------
38        request.setAttribute("result", kekka);
39
40        // -------- JSPへ転送 (forward) --------
41        RequestDispatcher rd = request.getRequestDispatcher("/praResult.jsp");
42        rd.forward(request, response);
43
44    }
```

パラメータ名等は一例です。
型変換に注意！



CSSの使い方

①CSSファイルの作成

②CSSを読み込む側に

```
<link rel="stylesheet" href="style.css">
```

を挿入する　※(“style.css”)は一例なので変更

③CSSファイルに指定するタグと内容を記載する

例:h1タグ内を赤くする

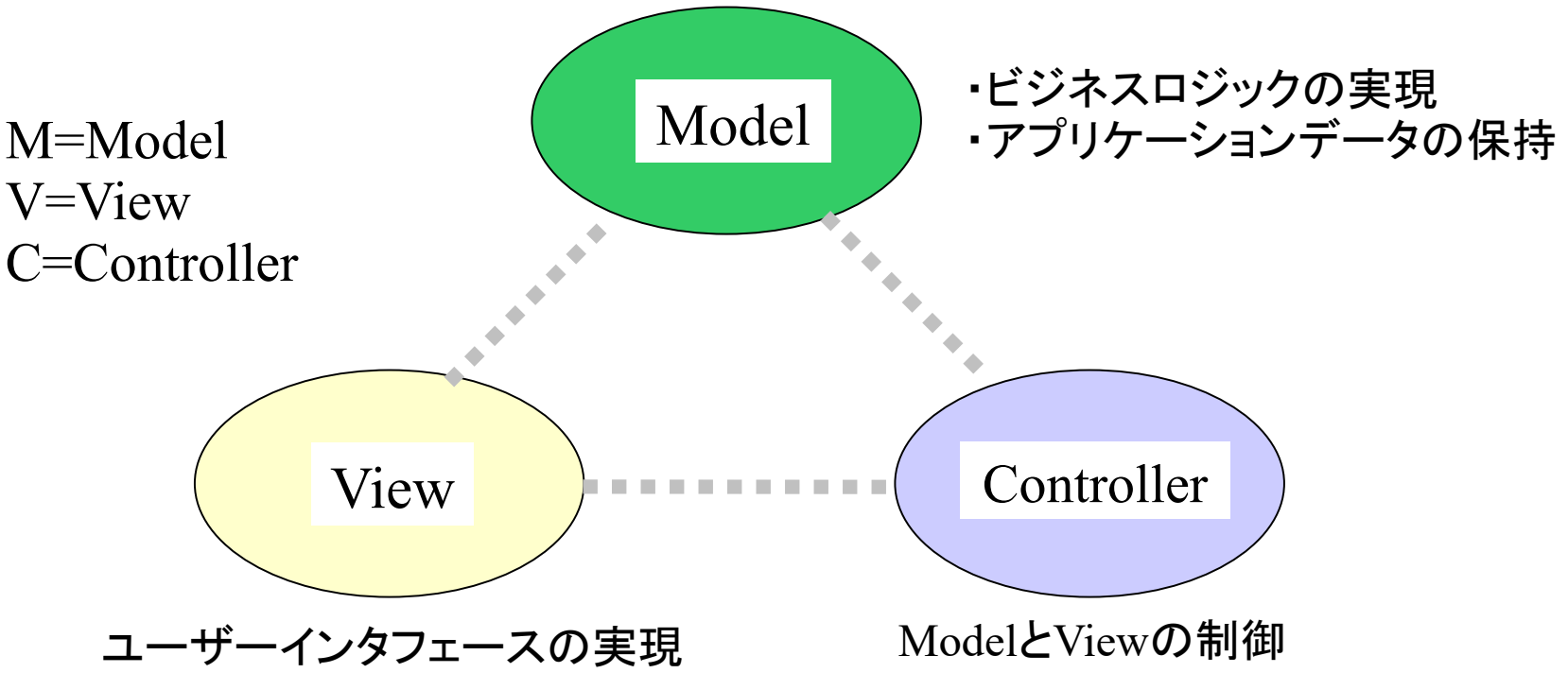
HanteiServlet.java　uranaiInput.jsp　Apache

```
1 @CHARSET "UTF-8";
2
3 h1{
4     color:red;
5 }
```



MVCアーキテクチャ

- アプリケーションを3つの要素に分けて設計するアーキテクチャ

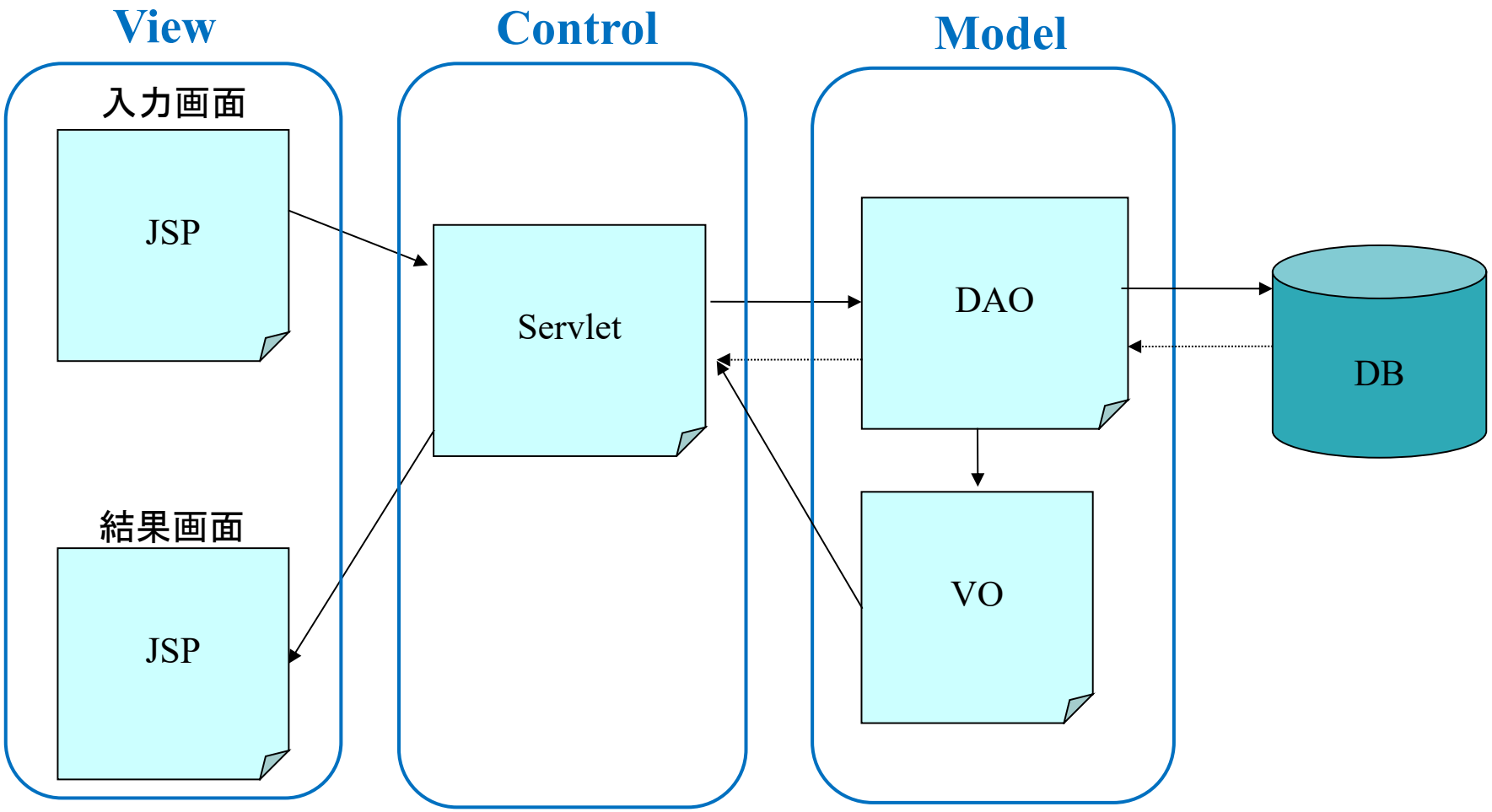


MVCアーキテクチャ

- Model
  - ビジネスロジックを実現する
  - アプリケーションデータを格納・保持する
- View
  - ユーザーインターフェースを実現する
    - 画面の表示を行う
    - 画面からの入力を受け付ける
- Controller
  - ModelとViewを制御する
    - Viewからのリクエストに応じたModelを選択し実行
    - Modelの実行結果に応じたViewを選択し表示



MVC構成図



HTML-基本表記

- <body></body>内に記入した内容が基本的にそのまま画面に表示されます。

- 基本的なタグ

タグ名	内容
<html>	HTML文書の範囲を示す
<head>	ページのヘッダー情報の範囲を示す
<title>	ページのタイトルを記述する。<head>タグ内で利用
<body>	ブラウザに表示する内容を記述する範囲を示す
<p>	段落を示す
 	改行を示す
<h1>,<h2>...	見出しを表示する。<h1>から<h6>までのレベルがある
	まとまりを示す。表示内容をグループ化する
	順番のないリストを表示する
	番号順リストを表示する
<!-- -->	コメントを記述する



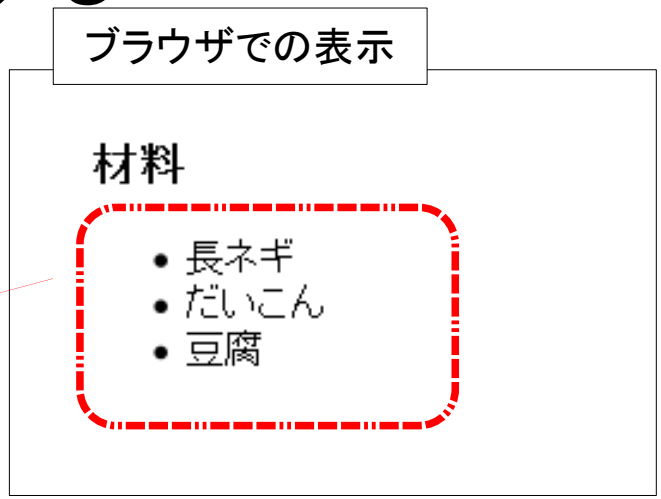
順番のないリスト表示

- ~
内部に
~
を使って、表示したいリスト項目を記述する

コード例

```
<html>
          ・・・
   <body>
     <h3>材料</h3>
     <ul>
       <li>長ネギ</li>
       <li>だいこん</li>
       <li>豆腐</li>
     </ul>
   </body>
</html>
```

ブラウザでの表示



タグは「・」マークをつけて
リスト項目を表示する



番号順リスト表示

- ~
内部に
~
を使って、表示したいリスト項目を記述する

コード例

```
<html>
          ・・・
   <body>
     <h3>手順</h3>
     <ol>
       <li>長ネギを入れる</li>
       <li>だいこんを入れる</li>
       <li>豆腐を入れる</li>
     </ol>
   </body>
</html>
```

ブラウザでの表示

手順

1. 長ネギを入れる
2. だいこんを入れる
3. 豆腐を入れる

タグは番号をつけて
リスト項目を表示する



表示内容をグループ化する

- タグで囲んだグループごとに表示内容の修飾を決めることができる
  - 見出しタグと同様、align属性で表示位置を指定できる

コード例

```
<html>
                :
  <body>
    <p>まず、にんにくと玉ねぎをフライパンで炒めます。<br>
          焦がさないように気をつけてください。</p>
    <div align="center">
      <p>にんにくの香りを引き出します。<br>
          玉ねぎは透き通るまで炒めます。<br>
      </p>
    </div>
    <p>次に、ひき肉を入れて軽く混ぜ合わせます。</p>
  </body>

</html>
```

ブラウザでの表示

まず、にんにくと玉ねぎをフライパンで炒めます。
焦がさないように気をつけてください。

にんにくの香りを引き出します。
玉ねぎは透き通るまで炒めます。

次に、ひき肉を入れて軽く混ぜ合わせます。

タグでグループ化した部分をalign属性（center）で中央表示



フォーム要素とデータの送受信

- HTMLの画面でデータを入力させ送信するには
  - <form>タグで送信先、送信形式（後述）を指定
    - 送信先＝action属性
    - 送信形式＝method属性
  - <form>～</form>内にフォーム要素タグを記述



フォームの基本的な構造(1)

- フォームの基本的な構造

HTML文書

```
<body>
    <form action="http://･･･" method="post">
        ID:
        <input type="text" name="id"> <br>
        パスワード:
        <input type="password" name="pw"> </br>
        <input type="submit" value="ログイン">
    </form>
</body>
```

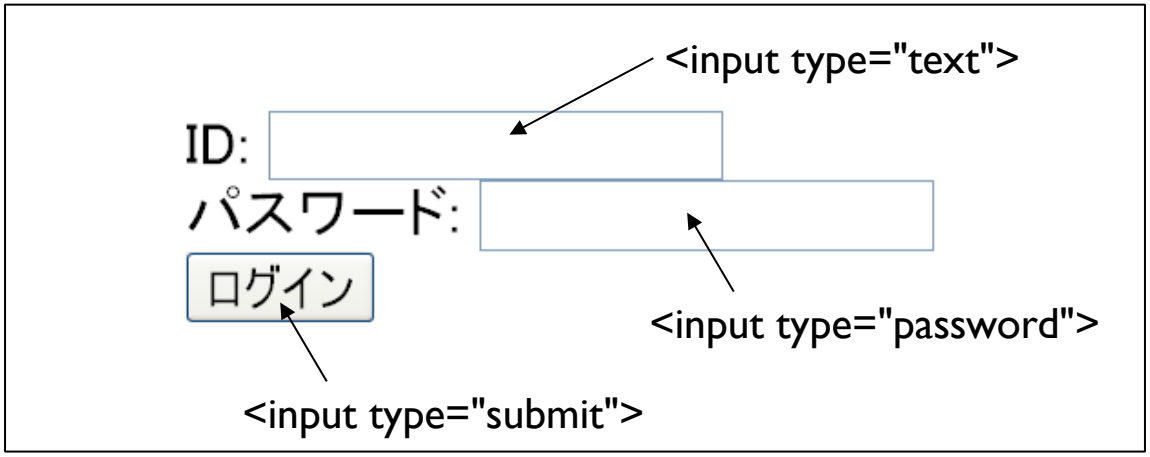
フォーム全体を<form>タグで囲む

フォーム要素を<form>タグ内に記述



フォームの基本的な構造(2)

- 前スライドのコードの表示例

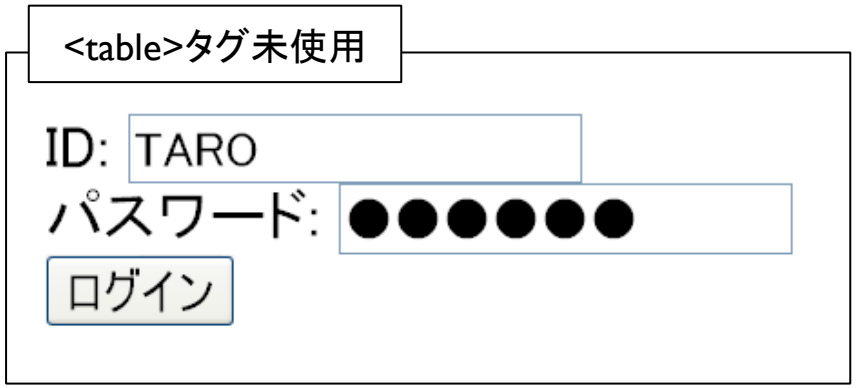


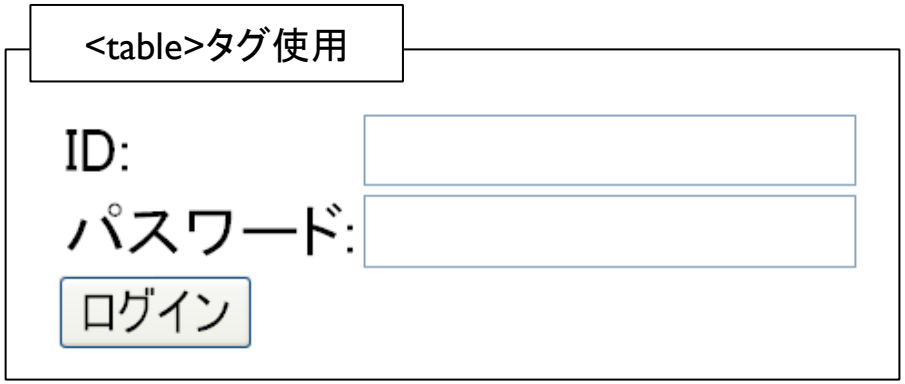
※ <form>それ自体は画面には表示されない



フォームの配置(1)

- HTMLでは入力項目などの位置を整えるには<table>タグを使うことができる
  - テーブル枠は表示、非表示を設定できる

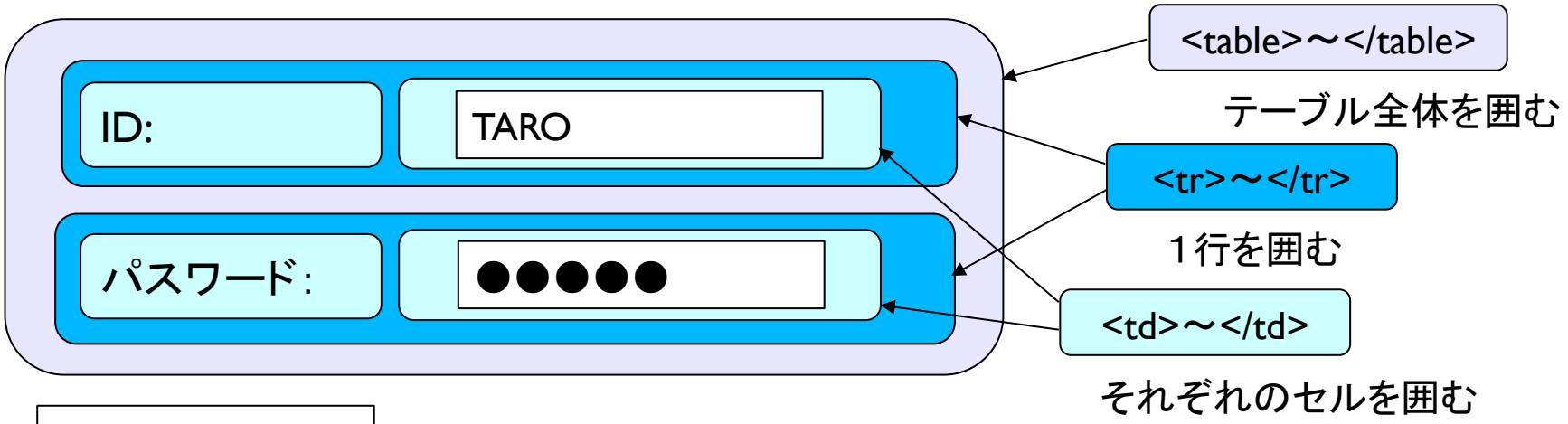






フォームの配置(2)

- 表示とタグの関係



コード例

```
<table>
<tr>
      <td>ID:</td>
      <td><input type="text" name="id"></td>
</tr>
<tr>
      <td>パスワード:</td>
      <td><input type="password" name="pw"></td>
</tr>
</table>
```

※表に枠線をつけるには
<table border>または
<table border="n">(nは枠の太さ)
のように指定します



フォーム要素タグ(1)

- 主なフォーム要素タグ

タグ名(属性)	内容
<input type="text">	テキストボックス（1行入力可能な項目）
<input type="password">	パスワード（入力内容がマスクされる）
<input type="checkbox">	チェックボックス（複数選択可）
<input type="radio">	ラジオボタン（単一選択項目）
<input type="file">	ファイルアップロード用の項目とボタン
<input type="button">	ボタン（JavaScriptなどと組み合わせて利用）
<input type="hidden">	隠しパラメータ（表示されないが送信される）
<input type="submit">	送信ボタン（フォーム内容をサーバに送信）
<input type="reset">	リセットボタン（入力を初期状態に戻す）
<textarea>	複数行入力が可能なテキストエリア
<select>	セレクトボックス（コンボボックス）の親タグ
<option>	選択肢を定義するためのタグ



フォーム要素タグ(2)

- 共通な性質
  - 送信パラメータ名はname属性で指定する

コード例

```
<form action="http://・・・" method="post">
<input type="text" name="address">
<input type="submit" value="送信">
</form>
```

ブラウザでの表示



送信すると

パラメータ名	値
address	東京都

サーバ(送信先)
に送られる内容

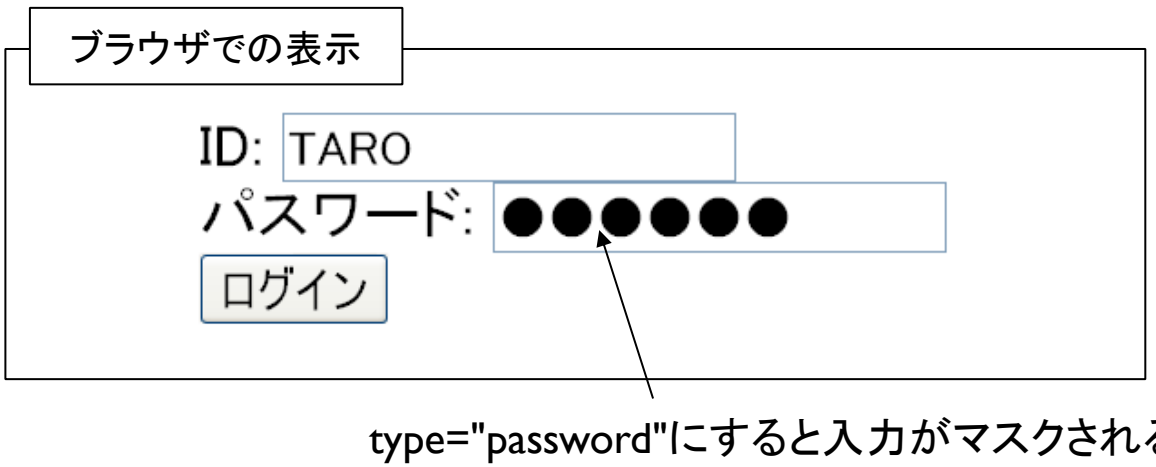


テキストボックス/パスワード(1)

コード例

```
ID:
<input type="text" name="id"><br>
パスワード:
<input type="password" name="pw"></br>
<input type="submit" value="ログイン">
```

ブラウザでの表示



type="password"にすると入力がマスクされる



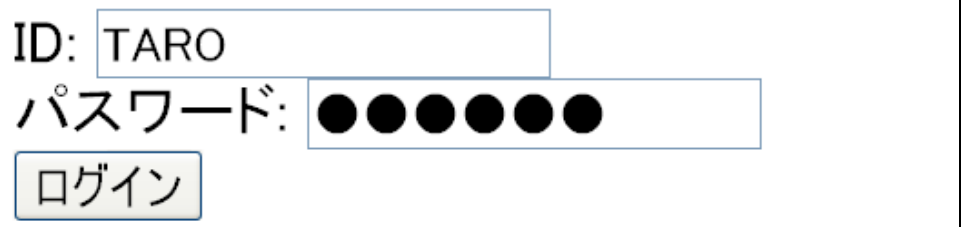
テキストボックス/パスワード(2)

- 初期値を表示する場合はvalue属性を指定

コード例

```
<input type="text" name="id" value="TARO"><br>
<input type="password" name="pw" value="abc"></br>
```

ブラウザでの表示



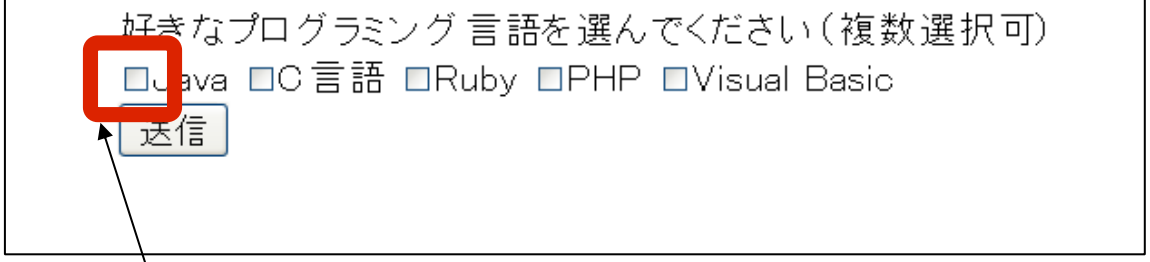


チェックボックス(1)

コード例

```
好きなプログラミング言語を選んでください（複数選択可）<br>
<input type="checkbox" name="fav_lang" value="java">Java
<input type="checkbox" name="fav_lang" value="c">C言語
      :
<input type="checkbox" name="fav_lang" value="vb">Visual Basic
```

ブラウザでの表示



type="checkbox"によるチェックボックスの表示

チェックボックス(2)

- チェックされた値だけが送信される

コード例

```
好きなプログラミング言語を選んでください(複数選択可)<br>
<input type="checkbox" name="fav_lang" value="java">Java
<input type="checkbox" name="fav_lang" value="c">C言語
      ・・・
<input type="checkbox" name="fav_lang" value="vb">Visual Basic
```

ブラウザでの表示



送信すると

パラメータ名	値
fav_lang	java
fav_lang	c
fav_lang	vb

サーバ(送信先)
に送られる内容

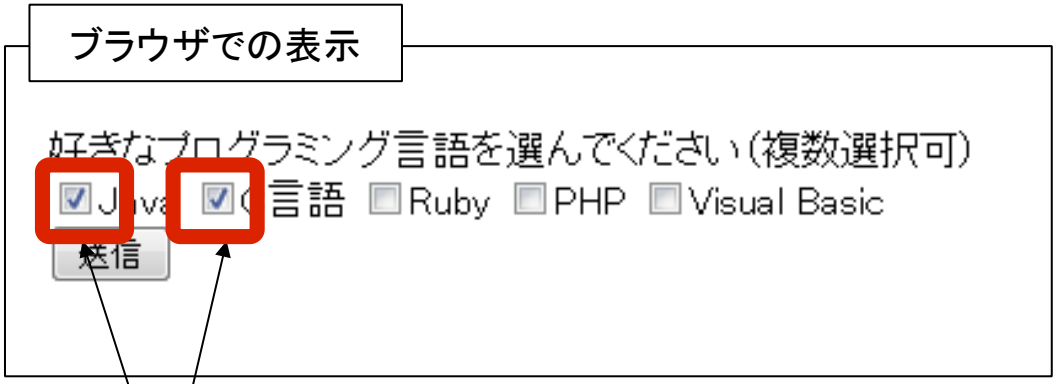
チェックボックス(3)

- デフォルトでチェックをつけるには「checked」を記述する

コード例

```
好きなプログラミング言語を選んでください（複数選択可）<br>
<input type="checkbox" name="fav_lang" value="java" checked>Java
<input type="checkbox" name="fav_lang" value="c" checked>C言語
      ...
<input type="checkbox" name="fav_lang" value="vb">Visual Basic
```

ブラウザでの表示



checkedをつけた項目は初期表示でチェック済みになる



ラジオボタン(1)

- 複数の項目からひとつだけを選択
  - checkedをつけるとデフォルトで選択される

コード例

```
好きなプログラミング言語をひとつだけ選択してください<br>
<input type="radio" name="fav_lang" value="java" checked>Java
<input type="radio" name="fav_lang" value="c">C言語
      :
<input type="radio" name="fav_lang" value="vb">Visual Basic
```

ブラウザでの表示



type="radio"によるチェックボックスの表示
（checkedをつけた項目は初期表示でチェック済みとなる）



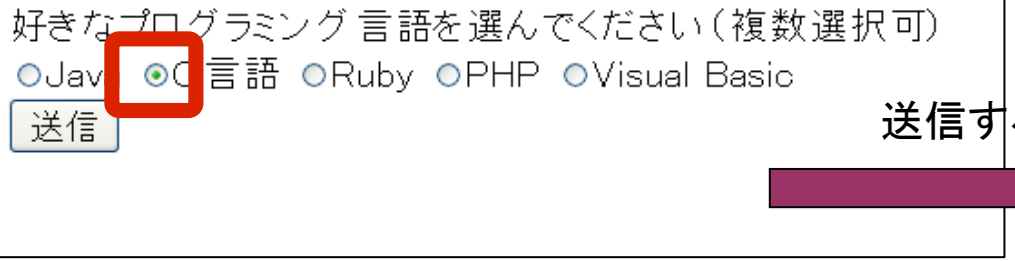
ラジオボタン(2)

- チェックされた値だけが送信される

コード例

```
好きなプログラミング言語をひとつだけ選択してください<br>
<input type="radio" name="fav_lang" value="java">Java
<input type="radio" name="fav_lang" value="c">C言語
      ・・・
<input type="radio" name="fav_lang" value="vb">Visual Basic
```

ブラウザでの表示



送信すると

パラメータ名	値
fav_lang	c

サーバ（送信先）
に送られる内容

ラジオボタン(3)

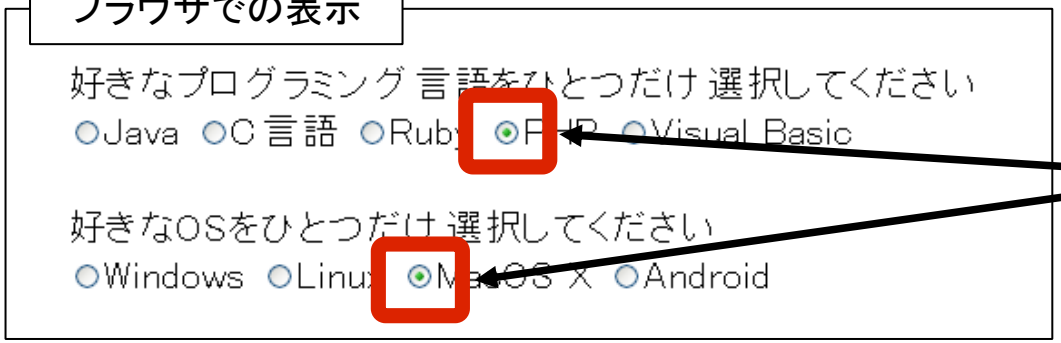
- 同じnameのラジオボタン同士が単一選択となる

コード例

```
好きなプログラミング言語をひとつだけ選択してください<br>
<input type="radio" name="fav_lang" value="java" checked>Java
<input type="radio" name="fav_lang" value="c" >C言語
      ...

好きなOSをひとつだけ選択してください<br>
<input type="radio" name="fav_os" value="win" checked>Windows
<input type="radio" name="fav_os" value="lin">Linux
      ...
```

ブラウザでの表示



同じname属性の値を持つ項目同士のうちからひとつだけ選択できる

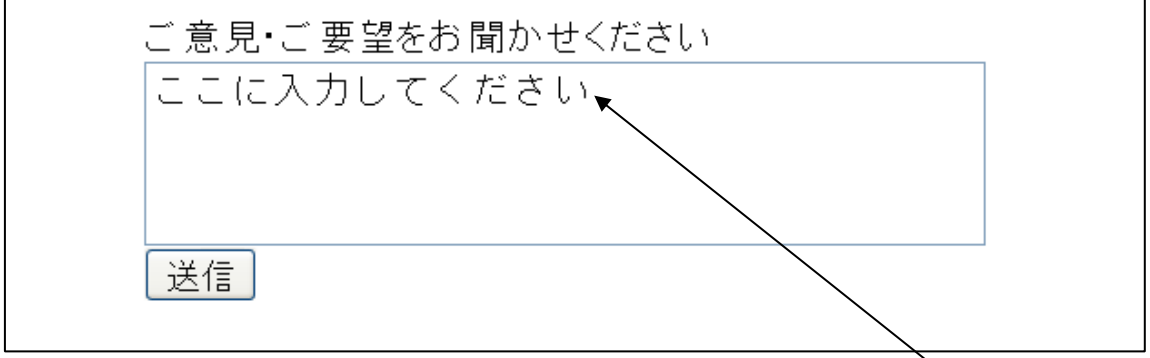
テキストエリア

- 複数行のテキスト入力を受け付ける

コード例

```
ご意見・ご要望をお聞かせください<br>
<textarea name="comment" rows="3" cols="40">
ここに入力してください
</textarea>
<br>
<input type="submit" value="送信">
```

ブラウザでの表示



<textarea>タグのボディが初期値となる

セレクトボックス(1)

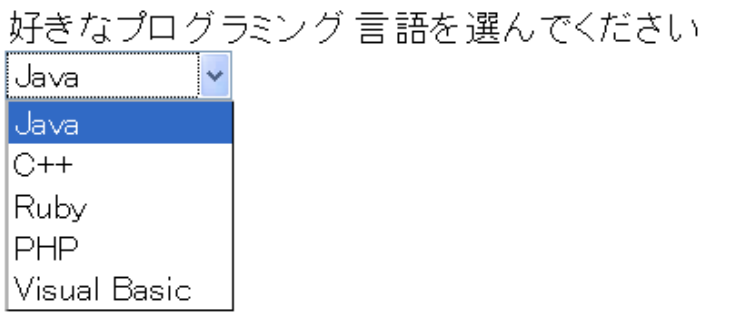
- 複数項目から一つを選択

コード例

好きなプログラミング言語を選んでください

```
<select name="fav_lang">
      <option value="ja">Java</option>
      <option value="cp">C++</option>
            ...
      <option value="vb">Visual Basic</option>
</select>
```

ブラウザでの表示





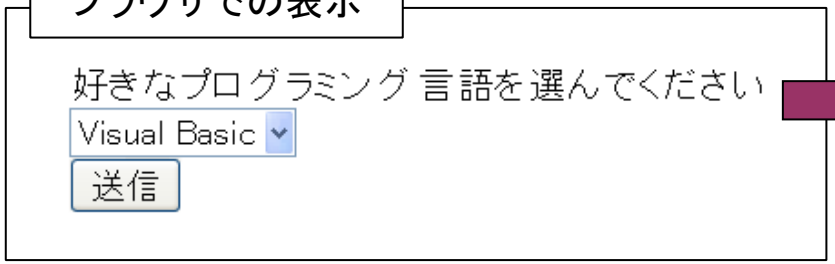
セレクトボックス(2)

- <select>のname属性がパラメータ名、<option>タグのvalue属性がパラメータ値として送信される

コード例

```
好きなプログラミング言語を選んでください<br>
<select name="fav_lang">
      <option value="ja">Java</option>
      <option value="cp">C++</option>
            ・・・
      <option value="vb">Visual Basic</option>
</select>
```

ブラウザでの表示



送信すると

パラメータ名	値
fav_lang	vb

サーバ(送信先)に送られる内容
※表示されている「Visual Basic」という文字列は送信されない

その他のタグ

- 主なタグ

タグ名	内容
<img>	画像を表示する
<a>	文字や画像に対してリンクを設定する
<hr>	区切り線を表示する



画像の表示(1)

- 画像を表示するには
  - <img>タグで画像ファイルを指定する
    - src属性＝画像ファイルの指定
    - alt属性＝画像に対する代替テキストの指定
      - 代替テキストは画像が表示できない時などに代わりに表示される
    - width属性＝画像を表示する幅の指定
      - ピクセル数（単位なしの整数）か画面全体に対する「%値」で指定
    - height属性＝画像を表示する高さの指定
      - ピクセル数（単位なしの整数）か画面全体に対する「%値」で指定



画像の表示(2)

- <img>タグの基本的な構造

HTML文書

```
<body>
    <img src="rose.png" alt="バラの画像"
      width="300" height="200">
</body>
```

src属性で画像ファイルを指定

alt属性で代替テキストを指定

width属性で表示幅を指定

height属性で表示の高さを指定



画像の表示(3)

コード例

```
<html>
    <body>
      <img src="../images/rose.png" alt="バラの画像"
           width="300" height="200">
    </body>


</html>
```

width属性やheight属性で
指定された大きさで表示される

ブラウザでの表示



リンクの設定(1)

- 文字や画像をクリックして別のページに遷移するには
  - <a>タグで遷移先を指定する
    - href属性＝遷移先指定
  - <a>～内に記述した内容がリンク要素として表示される
    - リンク要素は文字だけでなく画像も記述できる



リンクの設定(2)

- <a>タグの基本的な構造

HTML文書

リンク全体を<a>タグで囲む

href属性で遷移先を指定

```
<body>
    <a href="･･･次のページへ      </a>
    <a href="･･･">
    <img src="rose.png" alt="バラ" ... >
    </a>
</body>
```

表示するリンク要素(文字)を<a>タグ内に記述

表示するリンク要素(画像)<a>タグ内に記述



リンクの設定(3)

コード例

```
<html>
  <body>
    <a href="loginPage.html">
          ログインページへ
    </a><br><br>

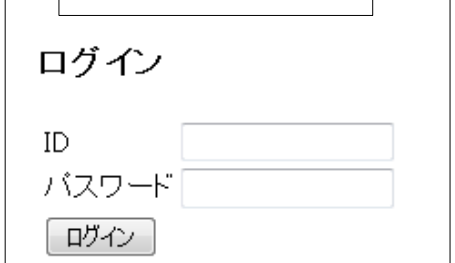
    画像をクリックして詳細確認<br>
    <a href="roseInfo.html">
       <img src="../images/rose.png" alt="バラの画像"
            width="150" height="100">
    </a>
  </body>


</html>
```

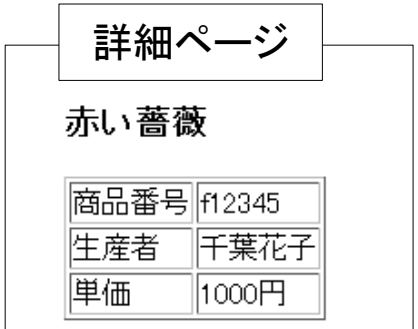
ブラウザでの表示



ログインページ



詳細ページ



それぞれのリンク要素をクリックすると、href属性で指定された遷移先へ移動する



区切り線の表示

- 区切り線（水平線）を表示するには
  - <hr>タグを記述する

コード例

```
<html>
    ・・・
    <body>
    ↓デフォルトの区切り線
    <hr>
    文書が区切られた状態です。
    </body>

</html>
```

ブラウザでの表示

↓デフォルトの区切り線

文書が区切られた状態です。

